

國立臺灣大學電機資訊學院資訊網路與多媒體研究所



碩士論文

Graduate Institute of Networking and Multimedia

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

用於臺灣容許對流天氣預報之少步數 MeanFlow 殘差
生成

Few-Step MeanFlow Residuals for
Convection-Allowing Forecasting over Taiwan

鍾鎮嶸

Desmond Cheong Zheng Yong

指導教授: 周承復博士 博士

Advisor: Cheng-Fu Chou, Ph.D.

中華民國 115 年 6 月

June 2026

國立臺灣大學碩士學位論文
口試委員會審定書



用於臺灣容許對流天氣預報之少步數 MeanFlow
殘差生成

Few-Step MeanFlow Residuals for
Convection-Allowing Forecasting over Taiwan

本論文係鍾鎮嶸君（R13944064）在國立臺灣大學資訊網路與多媒體研究所完成之碩士學位論文，於民國 115 年 6 月 24 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____	_____
_____	_____
_____	_____
_____	_____

所 長：_____





Acknowledgements

I would like to thank my advisor for guidance throughout this thesis, and the members of the lab for the many discussions that shaped this work. I am grateful to the developers of StormCast and PhysicsNeMo for releasing the pretrained checkpoints and training infrastructure that made this study possible, and to the Taiwan Central Weather Administration / NCDR for the RWRF reanalysis data used as training targets. Finally, I thank my family for their support throughout the program.





摘要

近年來，基於擴散模型的生成式天氣預報（例如 GenCast 與 StormCast）已展現出在機率性、銳利化、且可集成的短時預報任務上超越傳統確定性深度學習模型的能力。然而，擴散模型在推論階段需要沿著機率流常微分方程（PF-ODE）執行數十次神經網路前向傳遞（Number of Function Evaluations, NFE），使其在進行長時間自迴歸滾動預測或大規模集成預報時的計算成本過高；同時，擴散殘差在稀疏、重尾的降水通道上仍偏向平滑，難以重現對流尺度的極端降水結構。

本論文以一個以臺灣區域（RWRF）資料集訓練的兩階段 StormCast 模型（凍結的確定性回歸均值加上生成式殘差頭）為基礎，提出並評估 **MeanFlow**（平均速度流匹配）作為其生成式殘差頭：網路直接學習一段完整時間區間上的平均速度 u_θ ，使得單次前向傳遞即可橫越整段噪聲到資料的軌跡，推論僅需 1-2 次網路評估、完全不需數值積分。MeanFlow 與一般擴散蒸餾不同，無需教師、僅需單一訓練階段：它透過 MeanFlow 恆等式（以單次 Jacobian-向量積自舉、並停止梯度的目標）直接從資料學習平均速度場，當區間長度趨於零時其目標恰好退化為標準流匹配。我們將此目標移植到 StormCast 的標準化殘差管線上，並保留兩個對照基準：原生的 EDM 擴散殘差頭（需 35 NFE，即我們所欲加速的對象），以及以條件流匹配（Conditional Flow Matching, I-CFM）學習瞬時直線速度場 v_θ 、需以少數幾步 Euler 積分取樣的 **FlowCast** 殘差頭——MeanFlow 在 75% 的批次上恰好退化為 FlowCast 的目標，故 MeanFlow 對 FlowCast 構成僅針對

訓練目標本身的對照。針對臺灣 RWRF 目標僅有 4 個高解析度輸出通道 ($t2m$, $u10$, $v10$, $qpepre$) 的特性，我們設計逐通道加權的 L2 損失，並在稀疏重尾的降水通道 $qpepre$ 上加入徑向對數功率譜密度 (log-PSD) 正則化項；我們另外比較了 log1p 降水編碼與原始 mm/h 編碼，以及降水通道權重 w_{pr} 的取捨。

我們在 2022 年整年逐小時驗證資料上，以確定性誤差 (RMSE/MAE)、機率分數 (CRPS)、降水類別分數 (CSI、FSS、HSS、誤報率)、徑向功率譜，以及最長 12 小時的自迴歸滾動誤差等指標，在同訓練樣本預算下將 MeanFlow 與 EDM 擴散基準及 FlowCast 進行正面比較。實驗結果顯示，MeanFlow 僅以 1–2 次網路評估 (約為擴散取樣器十分之一、FlowCast 約四分之一的成本) 即可達到與擴散基準同級的確定性與類別技能，並取得全研究最佳的溫度精度；其單步降水 CRPS 優於 FlowCast 在掃描範圍內任何步數的數值。其主要代價為集成離散度偏窄，在自迴歸滾動下表現為小幅的 CRPS 落差 (於僅 1 次評估時最為明顯，第 2 次評估可補回大部分離散度，故以 $K = 2$ 為建議運行點)。本研究為區域對流尺度生成式預報提供了一套可重現的單階段平均速度流匹配流程與降水導向的評估基準。

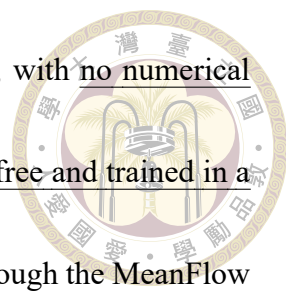
關鍵字：平均速度流匹配、MeanFlow、流匹配、擴散模型、單步生成、生成式天氣預報、對流尺度預報、降水臨近預報、StormCast



Abstract

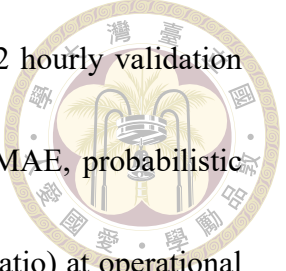
Diffusion-based generative weather forecasters such as GenCast and StormCast have recently surpassed deterministic deep-learning baselines on probabilistic, sharp, and ensemble-capable short-range prediction tasks. However, sampling a diffusion model requires integrating its probability-flow ODE with tens of neural-network function evaluations (NFE) per forecast step, which makes autoregressive rollouts and large ensembles computationally prohibitive for operational regional, convective-scale deployment; moreover, the diffusion residual remains comparatively smooth on the sparse, heavy-tailed precipitation channel, where convective-scale extremes matter most.

This thesis builds on a two-stage StormCast model trained on a Taiwan-domain RWRF dataset — a frozen deterministic regression mean plus a generative residual head — and proposes and evaluates **MeanFlow**, an average-velocity flow-matching objective, as the StormCast generative residual head. The network learns the mean velocity over an entire time interval, so a single forward pass transports the state across the whole noise-to-data



trajectory and sampling needs only one to two network evaluations, with no numerical integration at all. Unlike diffusion distillation, MeanFlow is teacher-free and trained in a single stage: it learns the average-velocity field directly from data through the MeanFlow identity (a single-Jacobian-vector-product, stop-gradient bootstrapped target), and as the interval length shrinks to zero its objective collapses exactly to standard flow matching. We transcribe the objective into StormCast’s standardised-residual pipeline and retain two reference heads for comparison: the native EDM diffusion residual (35 NFE — the model we set out to accelerate), and **FlowCast**, an Independent Conditional Flow Matching (ICFM) head that learns the instantaneous straight-line velocity field v_θ and samples it with a handful of explicit Euler steps. Because MeanFlow’s objective reduces exactly to FlowCast’s on 75% of each batch, the MeanFlow-versus-FlowCast comparison is a controlled A/B test on the training objective alone. Because the Taiwan RWRF target contains only four high-resolution output channels (t2m, u10, v10, qpepre), we can afford channel-specific loss design: a channel-weighted L2 objective plus a radial log power-spectral-density (log-PSD) regularizer on the sparse, heavy-tailed precipitation channel qpepre. We additionally study the log1p versus raw-mm/h precipitation encoding and the precipitation channel-weight w_{pr} .

We compare MeanFlow head-to-head against the EDM diffusion baseline and the



FlowCast head at a matched training-sample budget on the full 2022 hourly validation year, using a precipitation-aware metric suite: per-channel RMSE/MAE, probabilistic CRPS, categorical precipitation scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, radial log-PSD, and autoregressive rollout error out to 12 hours. Our experiments show that MeanFlow reaches the EDM diffusion baseline’s deterministic and categorical skill class at one to two network evaluations — roughly a tenth of the diffusion sampler’s cost and a quarter of FlowCast’s — with the best temperature accuracy in the study and a single-step precipitation CRPS that undercuts FlowCast at any step count. Its principal cost is narrowed ensemble spread, which surfaces as a modest rollout-CRPS gap (largest when exactly one evaluation is used; a second evaluation restores most of the spread, so $K = 2$ is the recommended operating point). The thesis contributes a reproducible single-run average-velocity flow-matching pipeline for the StormCast residual and a precipitation-aware evaluation protocol for regional convective-scale generative forecasting.

Keywords: MeanFlow, Average-Velocity Flow Matching, Flow Matching, Diffusion Models, One-Step Generation, Generative Weather Forecasting, Convective-Scale Forecasting, Precipitation Nowcasting, StormCast





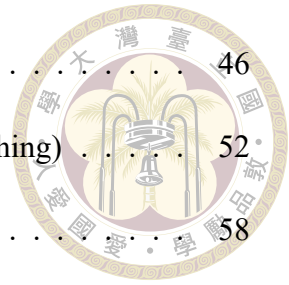
Contents

	Page
Verification Letter from the Oral Examination Committee	i
Acknowledgements	iii
摘要	v
Abstract	vii
Contents	xi
List of Figures	xv
List of Tables	xvii
Denotation	xix
Chapter 1 Introduction	1
1.1 Weather Forecasting and Its Societal Stakes	1
1.2 A Brief History of Weather Prediction Methods	2
1.3 Diffusion Models for Weather: Intuition	4
1.4 The Inference-Cost Problem	4
1.5 From Diffusion to Flow Matching	5
1.6 Contributions	7
1.7 Thesis Outline	9



Chapter 2	Background and Related Work	11
2.1	Numerical Weather Prediction	12
2.2	Data-Driven Global Weather Models	12
2.3	Regional and Convective-Scale Forecasting	13
2.4	GenCast and the Generative Turn	15
2.5	Score-Based Diffusion: A Primer	15
2.5.1	DDPM and the Discrete-Time Construction	16
2.5.2	DDIM and the Deterministic ODE View	17
2.5.3	Continuous-Time View: VE-SDE and PF-ODE	17
2.6	EDM: The Elucidated Formulation	19
2.7	Conditioning in Diffusion Models	21
2.8	Flow Matching, FlowCast, and MeanFlow	22
2.9	Flow Matching versus Diffusion: Why Straight-Line Paths Help	26
2.10	Prior Generative Models for Precipitation Nowcasting	27
2.11	Evaluation of Precipitation Forecasts	28
Chapter 3	Method	31
3.1	Problem Setup	32
3.2	Dataset: Taiwan RWRP	33
3.3	Data Sources and Preprocessing Pipeline	35
3.3.1	Raw data sources	36
3.3.2	Preprocessing pipeline	37
3.3.3	Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior	41
3.4	Regression Mean and the EDM Diffusion Baseline	44

3.5	The FlowCast Baseline (Conditional Flow Matching)	46
3.6	The MeanFlow Residual (Average-Velocity Flow Matching)	52
3.7	Loss Design for the Precipitation Channel	58
3.8	Training Infrastructure	60
Chapter 4	Experiments and Results	61
4.1	Experimental Protocol	61
4.2	Evaluation Metrics	63
4.3	MeanFlow versus Diffusion and FlowCast: Headline Comparison . .	65
4.4	Inference Cost: Wall-Clock and NFE Ablation	70
4.5	Precipitation Skill by Threshold and Neighbourhood	73
4.6	Autoregressive Rollout	76
4.7	Ablation: log1p Precipitation Encoding	78
4.8	Ablation: Precipitation Channel Weight	80
4.9	Comparison Against the Legacy Baseline	81
4.10	Qualitative Case Studies	82
4.11	Failure Modes	86
4.12	Discussion	88
Chapter 5	Conclusion and Future Work	91
5.1	Summary of Contributions	91
5.2	Limitations	92
5.3	Future Work	94
5.4	Closing Remarks	96



References

Appendix A — Hyperparameters and Training Configuration

A.1	EDM Diffusion Baseline Hyperparameters	103
A.2	FlowCast (Conditional Flow Matching)	104
A.3	MeanFlow (Average-Velocity Flow Matching)	106
A.4	Ablation Grids	108
A.5	Per-Channel Dataset Statistics	108

Appendix B — Reproducibility 109

B.1	Code Availability	109
B.2	Dataset and Checkpoints	110
B.3	Commands	111

Appendix C — Detailed Derivations 115

C.1	Tweedie’s Formula: Score and Denoiser	115
C.2	Probability-Flow ODE from the VE-SDE	116
C.3	EDM Preconditioning Derivation	117
C.4	Karras ρ -Spaced Noise Schedule	120
C.5	CFM Theorem: Conditional = Marginal Gradient	120
C.6	Independent CFM: Straight-Line Path and Target	121
C.7	FlowCast’s Standardised-Residual Adaptation	123
C.8	The MeanFlow Average-Velocity Identity	124





List of Figures

Figure 3.1	Per-channel Gaussianity diagnostics for HighRes targets	42
Figure 4.1	Headline scoreboard plot	66
Figure 4.2	Per-channel RMSE and CRPS	69
Figure 4.3	Inference cost and speedup	71
Figure 4.4	Categorical precipitation skill by threshold	74
Figure 4.5	Heavy-rain Fractions Skill Score	76
Figure 4.6	Rollout RMSE versus lead time	78
Figure 4.7	Qualitative precipitation rollout case study	84
Figure 4.8	Qualitative wind rollout case study	85
Figure 4.9	FlowCast training curves	86





List of Tables

Table 3.1	Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.	37
Table 3.2	Gaussianity statistics of HighRes target channels	42
Table 4.1	MeanFlow, FlowCast and EDM diffusion scoreboard	67
Table 4.2	Inference timing ablation (A6000)	70
Table 4.3	Inference timing ablation (H100)	71
Table 4.4	Precipitation skill across pooling windows	75
Table 4.5	Rollout RMSE versus lead time	77
Table 4.6	log1p versus raw-mm/h encoding ablation	79
Table 4.7	Precipitation channel-weight ablation	80
Table 4.8	Legacy versus cleaned pipeline	81





Denotation

NWP	Numerical Weather Prediction
RWRF	Regional Weather Research and Forecasting (Taiwan domain reanalysis)
EDM	Elucidated Diffusion Model (Karras et al., 2022); the reference diffusion baseline
CFM	Conditional Flow Matching
I-CFM	Independent Conditional Flow Matching (the FlowCast baseline objective)
MeanFlow	Average-velocity flow matching (Geng et al., 2025); the method of this thesis
JVP	Jacobian–vector product (forward-mode, used for the MeanFlow target)
ODE	Ordinary Differential Equation
PF-ODE	Probability-Flow Ordinary Differential Equation



SDE	Stochastic Differential Equation
NFE	Number of Function Evaluations (neural-net forward passes per sample)
EMA	Exponential Moving Average
PSD	Power Spectral Density
CSI	Critical Success Index
FSS	Fractions Skill Score
HSS	Heidke Skill Score
FAR	False-Alarm Ratio
CRPS	Continuous Ranked Probability Score
X_t	High-resolution atmospheric state at time t (target). Channels: $t2m$, $u10$, $v10$, $qpepre$.
S_t	Low-resolution conditioning state at time t (24 channels).
I	Time-invariant fields (land-sea mask, orography).
F_ξ	Deterministic regression network producing mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
M_t	Regression mean prediction.
R_t	Residual $R_t = X_t - M_t$ learned by the generative head.
c	Conditioning bundle for the generative head. All three heads (EDM diffusion, FlowCast, MeanFlow) use $\text{concat}(X_{t-1}, M_t, I)$; the orig-

	inal StormCast additionally concatenates the low-resolution background S_t .
D_ψ	EDM diffusion baseline denoiser (EDMPrecond SongUNet).
v_θ	FlowCast (I-CFM baseline) instantaneous velocity field (FlowCast-Precond SongUNet).
u_θ	MeanFlow average-velocity field $u_\theta(z, r, t, c)$ (MeanFlowPrecond SongUNet); the network of this thesis.
σ	EDM noise level.
$\sigma_{\min}, \sigma_{\max}$	Minimum and maximum EDM noise levels (0.002 and 80).
σ_{data}	Data standard-deviation hyperparameter shared by all three heads (0.5).
ρ	Karras schedule exponent (7).
t (or r, t)	Flow time, in $[0, 1]$ (noise at 0, data at 1). MeanFlow uses an interval $[r, t]$ with state time $r \leq t$.
x_0, x_1	Flow endpoints: prior sample $x_0 \sim \mathcal{N}(0, I)$ and standardised residual $x_1 = R_t/\sigma_{\text{data}}$.
v	Instantaneous (I-CFM) velocity field; conditional target $v = x_1 - x_0$.
$u(z_r, r, t)$	Average velocity of the flow over $[r, t]$; the field MeanFlow learns. $\lim_{t \rightarrow r} u = v$.
ρ_{MF}	MeanFlow ratio: fraction of each batch with $r < t$ (the bootstrapped branch); 0.25.

σ_{path}

I-CFM conditional-path standard deviation (0.01; MeanFlow uses the pure interpolant, $\sigma_{\text{path}} = 0$).

K

Number of sampling network evaluations (NFE): Euler steps for Flow-Cast, average-velocity segments for MeanFlow.

β_k

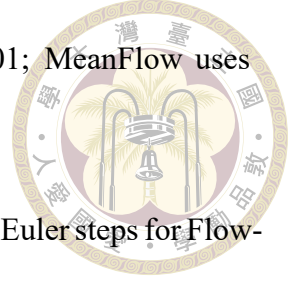
Per-channel loss weight; $w_{\text{pr}} = \beta_{\text{qpepre}}$ is the precipitation weight.

λ_{spec}

Weight of the radial log-PSD spectral regulariser.

$P(\kappa)$

Radial power spectral density at wavenumber κ .





Chapter 1 Introduction

1.1 Weather Forecasting and Its Societal Stakes

Accurate short-range weather forecasting is a public-safety service. In Taiwan, a small, densely populated island lying along the western Pacific typhoon track, the stakes are especially concentrated: a single landfalling typhoon can dump more than 1000 mm of rainfall on steep orography in twelve hours, trigger flash floods and landslides, and disrupt transportation, agriculture, and power. Between typhoons, Mei-yu frontal systems, afternoon convective storms, and orographically forced rainfall drive the remainder of the hydrological cycle. For decision-makers who must issue evacuation orders, schedule reservoir releases, or re-route air traffic, the relevant question is almost always probabilistic and local: how much rain, where, within the next six to twelve hours?

The operationally critical quantity — precipitation at convective scales — is also the hardest for weather models to predict. Precipitation fields are sparse (most pixels are dry), heavy-tailed (the few wet pixels span orders of magnitude), and strongly coupled to terrain and mesoscale dynamics that only emerge at kilometre-scale resolution. This combination means that any forecasting system which produces visually smooth outputs, or which regresses toward the mean, will systematically underpredict the very events that matter most.

1.2 A Brief History of Weather Prediction Methods



Modern weather forecasting has passed through four broad eras, each driven by a different scientific or computational advance.

Numerical Weather Prediction (NWP). Since Richardson's early twentieth-century vision and its post-war realisation by Charney, Phillips, and others, operational forecasting has been dominated by numerical integration of the primitive equations of atmospheric motion. State-of-the-art systems such as the ECMWF Integrated Forecasting System (IFS), the NCEP Global Forecast System (GFS), and regional convection-permitting models like WRF and its Taiwan variant RWRF discretise the atmosphere on a three-dimensional grid, solve the governing fluid-dynamics and thermodynamics equations, and close the unresolved sub-grid processes (convection, microphysics, boundary-layer turbulence, radiation) through physical parameterisations. Uncertainty is quantified by ensemble prediction systems that perturb initial conditions and model physics. NWP is physically consistent, well-understood, and remarkably skilful, but convection-permitting runs are computationally expensive, parameterisation error is non-trivial, and operational latency is a fundamental constraint.

Statistical and classical machine-learning post-processing. Long before deep learning, statistical techniques — model output statistics (MOS), analog methods, kriging, and more recently random forests and gradient-boosted trees — have been used to bias-correct and downscale NWP outputs. These approaches treat the NWP forecast as a feature vector and learn a local correction to each observation site. They set an important precedent:

that learned components can complement physical cores, and that the ultimate criterion is predictive skill at points of interest rather than global physical fidelity.



The deep-learning era. Starting in the early 2020s, a new generation of purely data-driven global weather models began to rival and in some cases surpass operational NWP at synoptic scales. FourCastNet [14] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. Pangu-Weather [2] introduced a three-dimensional hierarchical transformer and demonstrated medium-range skill competitive with IFS. GraphCast [10] adopted an icosahedral-mesh graph neural network and achieved operational-quality ten-day forecasts at a fraction of the inference cost of IFS. At convective scales, MetNet [1, 4, 23] and the Deep Generative Model of Rainfall (DGMR) [17] pushed radar-based nowcasting from deterministic CNNs toward generative models capable of producing sharp, ensemble-like outputs. Most recently, StormCast [15] demonstrated that a convection-permitting diffusion model can learn a km-scale regional forecaster directly from reanalysis data.

The generative turn. A persistent problem of deterministic deep-learning forecasters is that, because they are trained with pixelwise regression losses, they produce fields that are systematically blurry and that underestimate extremes. Generative models resolve this by learning the distribution of plausible forecasts rather than its conditional mean. DGMR showed this qualitatively for radar nowcasting; GenCast [16] made the argument at global scale, demonstrating that a conditional diffusion model produces both sharper and better-calibrated ensemble forecasts than GraphCast. The lesson transfers directly to regional convective forecasting, and it motivates the generative forecaster at the centre of this thesis.

1.3 Diffusion Models for Weather: Intuition



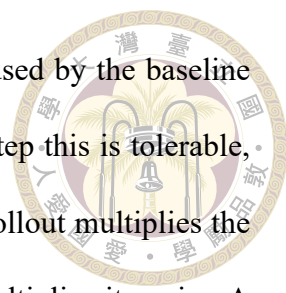
A diffusion model is a generative model trained to reverse a gradual noising process.

Given a clean sample x_0 drawn from some data distribution — say, a high-resolution precipitation field over Taiwan — we progressively add Gaussian noise until after enough steps the result is indistinguishable from pure noise. A neural network is then trained to undo one step of this process: given a noised sample and the current noise level, predict the underlying clean signal (equivalently, the score of the noised distribution). At sampling time we start from pure Gaussian noise and apply the learned denoiser repeatedly until we recover a clean sample.

Three properties make this framework attractive for weather forecasting. First, it is naturally probabilistic: different noise seeds produce different but equally plausible samples, giving an ensemble for free. Second, the samples are sharp: because the model is never asked to average over outcomes, it does not regress toward the mean. Third, the denoiser can be conditioned on auxiliary inputs — the previous atmospheric state, coarse boundary fields, invariant land-surface information — which is exactly the interface a forecaster needs. GenCast at global scale and StormCast at regional scale both exploit these properties.

1.4 The Inference-Cost Problem

The cost of this flexibility is paid at inference time. High-quality diffusion sampling requires integrating the model's probability-flow ordinary differential equation (PF-ODE) with a few tens of neural-network forward passes per generated sample — the so-called



number of function evaluations (NFE); the 18-step Heun schedule used by the baseline in this thesis costs 35 NFE per forecast step. For a single forecast step this is tolerable, but operational use cases compound it. An autoregressive 24-hour rollout multiplies the cost by 24. A 50-member ensemble for probabilistic forecasting multiplies it again. A convection-permitting Taiwan-domain forecast with a 50-member ensemble over a 24-hour horizon therefore requires on the order of $50 \times 24 \times 35 = 42,000$ forward passes of a large U-Net — which places the entire class of generative forecasters near the edge of, or outside, the operational envelope of a regional forecasting centre that must refresh its nowcast every few minutes between radar volumes.

The cost is not the only issue. The reason diffusion needs so many steps is that its PF-ODE trajectory — the path that carries a pure-noise sample to a clean forecast — is curved. A curved trajectory cannot be integrated accurately in one or two steps; coarse step counts incur visible discretisation error, and that error is worst precisely on the sparse, heavy-tailed precipitation channel, where the score of the noised distribution is ill-conditioned near small noise levels. The practical symptom is that an aggressively sub-sampled diffusion residual reverts toward a smooth, “slightly wet everywhere” field that scores acceptably on RMSE but destroys the convective structure operational users care about.

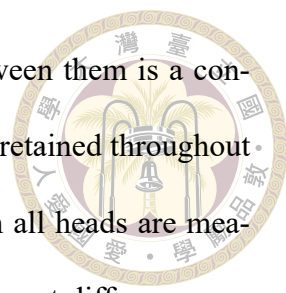
1.5 From Diffusion to Flow Matching

Both problems — the step count and the curvature — point at the same lever: the shape of the generative trajectory. If the path from noise to data were a straight line rather than a curved diffusion ODE, then a handful of explicit Euler steps would integrate it

almost exactly, and a single step would already be a strong approximation.

Conditional Flow Matching (CFM) [11, 28] is a recently-proposed generative framework that learns exactly such a model. Instead of learning a score and integrating a curved PF-ODE, CFM regresses a deterministic velocity field that transports a Gaussian prior to the data along a straight-line probability path, and it does so with a simulation-free mean-squared-error objective that needs no teacher model at all. Ribeiro and Pucur [18] applied this idea to radar precipitation nowcasting under the name **FlowCast** and reported that a 1-to-10-step flow-matching sampler matched or beat a 100-step diffusion sampler on probabilistic precipitation skill at a fraction of the inference cost. A straight-line flow is integrated accurately in a handful of explicit Euler steps — but it is still integrated, and that residual handful of network evaluations is the last cost a flow-matching forecaster leaves on the table.

This thesis pushes the same lever one decisive notch further with **MeanFlow** [5]. Rather than learning the instantaneous velocity of the flow and then integrating it, MeanFlow learns the flow’s average velocity over a whole time interval, so that one network evaluation transports a noise sample across the entire trajectory — there is no ODE solver and no discretisation error to amortise, and sampling collapses to one or two network evaluations. We adapt MeanFlow to the regional convection-permitting StormCast setting on the Taiwan RWRF domain as the generative residual head, keeping StormCast’s two-stage decomposition (a frozen deterministic regression mean plus a generative residual head) intact and replacing only the residual head’s curved EDM diffusion process. MeanFlow is the method of this thesis; FlowCast’s straight-line I-CFM velocity field is retained as the natural flow-matching baseline that MeanFlow generalises — in fact MeanFlow’s objective reduces exactly to FlowCast’s whenever the time interval has zero length, so the two



differ only by the average-velocity bootstrap and a comparison between them is a controlled A/B on the training objective. The EDM diffusion residual is retained throughout as the reference baseline that we set out to accelerate, against which all heads are measured at a matched training-sample budget, so that every quality and cost difference can be attributed to the choice of generative trajectory rather than to data, architecture, or the regression mean, all of which are held fixed. Crucially, MeanFlow is not a distillation of the diffusion teacher: it is trained from scratch in a single run, which makes it a clean test of whether a better-shaped generative target alone — without any teacher signal — already delivers the one-to-two-evaluation speed and the sharpness that operational regional forecasting needs.

1.6 Contributions

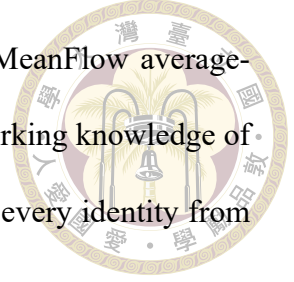
This thesis makes the following contributions:

1. We bring the average-velocity **MeanFlow** objective [5] into weather forecasting — to our knowledge for the first time — and adapt it to the StormCast two-stage regression-plus-residual architecture as the generative residual head: the network learns the displacement of the flow over whole time intervals via the MeanFlow identity (a single-JVP, stop-gradient bootstrapped target), so the residual is sampled in 1–2 network evaluations with no ODE solver, teacher-free and in a single training run, on a four-channel Taiwan regional target.
2. We retain two reference heads on the same backbone and budget so that the MeanFlow comparison isolates the generative trajectory: the native EDM diffusion residual (the 35-NFE model we set out to accelerate), and **FlowCast**, the straight-line

Independent-CFM velocity baseline [11, 18, 28] that MeanFlow generalises. Because MeanFlow’s objective collapses exactly to FlowCast’s at zero interval length, MeanFlow-versus-FlowCast is a controlled A/B test on the training objective alone.

3. We carry out the engineering needed to make all three residual heads share exactly the frozen regression mean and per-channel normalisation: same data, same F_ξ , same σ_{data} , same backbone, only the EDM process versus the I-CFM velocity field versus the MeanFlow average-velocity field.
4. We exploit the four-channel target with a precipitation-aware loss: a channel-weighted L2 objective plus an auxiliary radial power-spectral-density (log-PSD) regulariser on the sparse, heavy-tailed qpepre channel, and we ablate the precipitation channel weight w_{pr} and the log1p-versus-raw-mm/h precipitation encoding.
5. We assemble a precipitation-aware evaluation suite — per-channel RMSE/MAE, probabilistic CRPS, categorical skill scores (CSI, FSS, HSS, false-alarm ratio) at operational thresholds, radial log-PSD, and 1–12-hour rollout errors — and run it on the full 2022 hourly validation year of the Taiwan RWRF dataset.
6. We present a head-to-head comparison of MeanFlow against the EDM diffusion residual and the FlowCast velocity baseline at a matched sample budget, including an NFE/wall-clock-versus-quality trade-off and a comparison against a legacy old-StormCast baseline, on a regional convection-permitting forecaster — to our knowledge the first study of average-velocity flow matching on the Taiwan convective-scale domain.
7. Appendix B.3 provides line-by-line derivations of the theoretical machinery MeanFlow rests on — the score–denoiser (Tweedie) identity, the VE-SDE / PF-ODE,

EDM preconditioning, Conditional Flow Matching, and the MeanFlow average-velocity identity — in a form that should let a reader with a working knowledge of Gaussian algebra and the Fokker–Planck equation reconstruct every identity from scratch.



1.7 Thesis Outline

Chapter 2 surveys weather-prediction methods, a condensed account of the diffusion theory underlying the EDM baseline, and conditional flow matching building up to the average-velocity MeanFlow objective at the centre of this thesis, in enough depth to motivate and define every component used later. Chapter 3 describes the dataset, the frozen regression mean and the EDM diffusion and FlowCast baselines, and — as its core — the MeanFlow average-velocity residual pipeline as implemented in the accompanying codebase. Chapter 4 presents the experimental protocol and the head-to-head results across NFE budgets, precipitation thresholds, and rollout horizons. Chapter 5 summarises findings, limitations, and future work.





Chapter 2 Background and Related Work

This chapter assembles the theoretical background needed to understand the remainder of the thesis. It is organised in four parts: a survey of weather-prediction methods from NWP to data-driven models (Sections 2.1–2.4); a condensed account of score-based diffusion — DDPM and DDIM in summary, the continuous variance-exploding SDE / probability-flow ODE, and the EDM formulation used by our diffusion baseline (Sections 2.5–2.7), given only to the depth needed to understand the baseline we accelerate and the shared data scale σ_{data} ; the theory of Conditional Flow Matching building up to the average-velocity **MeanFlow** objective at the centre of this thesis — with FlowCast’s straight-line velocity field as the flow-matching baseline MeanFlow generalises — together with a direct comparison against the curved diffusion path and a survey of prior generative precipitation models (Sections 2.8–2.10); and an overview of precipitation-aware evaluation metrics (Section 2.11). Throughout the chapter we state the key identities in the main text and defer the derivations the method rests on to Appendix B.3.

2.1 Numerical Weather Prediction



Numerical weather prediction discretises the primitive equations of atmospheric motion — continuity, momentum (Navier–Stokes on a rotating sphere), thermodynamic energy, and water-vapour conservation — onto a three-dimensional grid and advances the state forward in time via explicit or semi-implicit integration. Sub-grid processes not captured by the dynamical core (convection, microphysics, boundary-layer turbulence, radiation, land-surface exchange) are closed through physical parameterisations. Operational systems include ECMWF’s IFS (global spectral, 0.1° horizontal resolution), NCEP’s GFS, and regional limited-area models such as WRF and its Taiwan-tuned variant RWRF, which provide the high-resolution targets used in this thesis.

Uncertainty is quantified by ensemble prediction systems that perturb initial conditions (e.g. via singular-vector or ensemble Kalman methods) and model physics. A convection-permitting (sub-4 km) ensemble with a few tens of members and a 24-hour forecast horizon requires hours of dedicated supercomputer time. The practical consequence is that NWP ensemble size and spatial resolution are traded off against operational latency.

2.2 Data-Driven Global Weather Models

Starting in 2022, a series of deep-learning models demonstrated that a single neural network, trained on the ERA5 reanalysis, can produce global medium-range forecasts that rival operational NWP on many headline metrics at a fraction of the inference cost.

FourCastNet. Pathak et al. [14] used an Adaptive Fourier Neural Operator to produce six-hourly global forecasts at 0.25° resolution. It was the first model to demonstrate that a data-driven system could match IFS on synoptic variables within a plausible compute budget.

Pangu-Weather. Bi et al. [2] introduced a hierarchical three-dimensional transformer with a hierarchical time-aggregation scheme and demonstrated medium-range skill competitive with or exceeding IFS on several variables and lead times.

GraphCast. Lam et al. [10] adopted a multi-mesh graph neural network defined on an icosahedral discretisation of the sphere. It produces ten-day 0.25° forecasts that are competitive with IFS on the majority of evaluated fields and does so in roughly one minute on a single TPU, compared with hours on a supercomputer for NWP.

These models share an important limitation: because they are trained with deterministic pixelwise losses (typically latitude-weighted MSE), their forecasts become progressively blurrier with lead time and systematically underestimate extreme events. For convection and precipitation, which this thesis cares about most, this is disqualifying.

2.3 Regional and Convective-Scale Forecasting

Global models operate at resolutions too coarse to resolve convection and orographic rainfall over Taiwan. Three lines of work fill the gap.

Nowcasting models. MetNet-1/2/3 [1, 4, 23] produce high-resolution short-range precipitation forecasts by ingesting ground-based radar and satellite observations through

large convolutional/attention backbones. DGMR [17] uses a conditional GAN to produce sharp, ensemble-like radar nowcasts and was the first work to make a strong empirical case that generative modelling is essential for precipitation skill.



StormCast. Pathak et al. [15] introduced a convection-permitting regional diffusion forecaster trained on the High-Resolution Rapid Refresh (HRRR) analysis over CONUS. StormCast decomposes forecasting into two stages:

1. A deterministic regression network F_ξ (a StormCastUNet) takes the previous high-resolution state X_{t-1} , a low-resolution conditioning state S_t , and time-invariant fields I , and predicts the conditional mean $M_t = F_\xi(X_{t-1}, S_t, I)$.
2. A residual diffusion network D_ψ (an EDMPrecond SongUNet) models the residual $R_t = X_t - M_t$ conditionally on $c = \text{concat}(X_{t-1}, S_t, M_t, I)$.

At inference time the model is rolled out autoregressively, with the previous diffusion output feeding the next step's regression input. This decomposition has two payoffs: the deterministic network captures what is easy to predict with a single network call, leaving the diffusion model free to focus on the residual uncertainty; and the residual is closer to Gaussian than the raw fields, which stabilises diffusion training.

This thesis uses a Taiwan-adapted StormCast model trained on the RWRF dataset. The adaptation reduces the output state from the 99 channels of the CONUS original to just four: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly precipitation (qpepre). This channel reduction has a useful side-effect: with only four output channels, per-channel loss engineering becomes cheap, and we exploit this in the precipitation-aware losses of Chapter 3.

2.4 GenCast and the Generative Turn

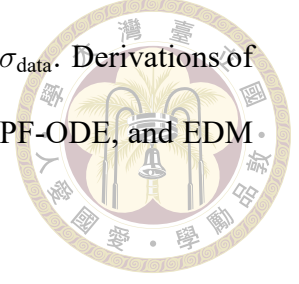


Price et al. [16] introduced GenCast, a global 0.25° conditional diffusion forecaster that samples ensemble members by varying the initial noise seed. GenCast was the first diffusion forecaster to clearly surpass the leading deterministic model (GraphCast) across the majority of its evaluation suite, on both deterministic scores (RMSE) and probabilistic scores (CRPS). Its success is the strongest empirical argument for the claim that motivates this thesis: generative modelling — and diffusion in particular — is the right inductive bias for short-to-medium range weather forecasting, but its operational cost is the obstacle to practical deployment.

2.5 Score-Based Diffusion: A Primer

A diffusion model defines a forward stochastic process that progressively corrupts a data sample $x_0 \sim p_{\text{data}}$ with Gaussian noise until the terminal state is indistinguishable from pure noise, then learns a neural network that reverses the corruption. The apparent difficulty of this programme — fitting the full generative reverse process of a high-dimensional distribution — hides a sequence of algebraic identities that together reduce training to a mean-squared-error regression. Because diffusion enters this thesis only as the reference baseline we set out to accelerate (and as the lineage of the curved generative path that flow matching straightens), we give it a condensed treatment: the discrete-time DDPM construction of Ho et al. [8] and its deterministic DDIM sampler [24] are summarised below, then the continuous variance-exploding SDE and probability-flow ODE that unify them [26], and finally the elucidated EDM parameterisation [9] that the base-

line actually uses and from which all three heads inherit the data scale σ_{data} . Derivations of the continuous-time identities the baseline relies on — Tweedie, the PF-ODE, and EDM preconditioning — are collected in Appendix B.3.



2.5.1 DDPM and the Discrete-Time Construction

The denoising diffusion probabilistic model (DDPM) of Ho et al. [8] defines a T -step Markov chain that adds Gaussian noise to a clean sample x_0 according to a variance schedule $\{\beta_t\}$. Writing $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$, the chain has a closed-form marginal that lets one jump to any noise level in a single line — so the forward process never has to be simulated:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (2.1)$$

The reverse process is a learned Gaussian $p_\theta(x_{t-1} | x_t)$ matched to the (tractable, x_0 -conditioned) forward posterior. After the standard variational-bound algebra and an ϵ -reparameterisation, the per-step KL terms collapse — with no loss-bearing approximation — into a single mean-squared-error objective that simply regresses the added noise:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]. \quad (2.2)$$

The punchline is that an intractable maximum-likelihood problem reduces to a supervised denoising regression, with one network call per training step. The cost lands at inference: DDPM’s ancestral sampler draws $T \approx 1000$ stochastic reverse steps, far too slow for deployment.

2.5.2 DDIM and the Deterministic ODE View



Song et al. [24] observed that the DDPM loss (2.2) depends only on the marginals $q(x_t | x_0)$, not on the particular Markov chain producing them, so the same trained network admits a family of non-Markovian samplers — including a fully deterministic one:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0(x_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \frac{x_t - \sqrt{\bar{\alpha}_t} \hat{x}_0(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}, \quad (2.3)$$

with $\hat{x}_0(x_t, t) = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$ the implied clean-data prediction. Equation (2.3) is exactly one explicit Euler step of an underlying ordinary differential equation, so the same model can be sampled in far fewer steps (typically 20–50 NFE, and toward ≈ 10 with higher-order solvers such as DPM-Solver [13]). DDIM is therefore the simplest first-order integrator of the probability-flow ODE made precise in the next subsection; the persistent curvature of that ODE — which keeps the step count in the 20–50 band for sharp precipitation — is exactly what the straight-line flow-matching path of Section 2.8 removes, and what the average-velocity MeanFlow objective at the centre of this thesis then jumps over entirely.

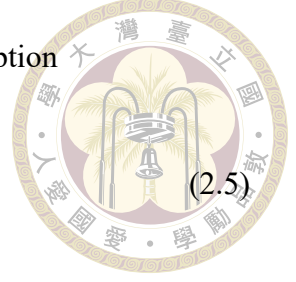
2.5.3 Continuous-Time View: VE-SDE and PF-ODE

Song et al. [26] unified DDPM, noise-conditional score matching, and their continuous-time limits into a single SDE framework. In the variance-exploding (VE) formulation adopted by EDM and inherited by StormCast, the forward SDE is

$$dx = \sqrt{\frac{d\sigma^2(t)}{dt}} dW_t, \quad (2.4)$$

so that the closed-form marginal is the same one-line Gaussian corruption

$$x_\sigma = x_0 + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (2.5)$$



now indexed by a continuous noise level $\sigma \in [0, \sigma_{\max}]$. The remarkable structural fact is that every diffusion SDE admits a deterministic probability-flow ordinary differential equation (PF-ODE) whose marginals coincide with those of the SDE at every time:

$$\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x), \quad (2.6)$$

where $\nabla_x \log p_\sigma(x)$ is the (Stein) score of the noised marginal. Equation (2.6) is derived in Appendix B.3, §C.2 by matching the Fokker–Planck equation of the SDE against the continuity equation of the deterministic flow. The PF-ODE plays a central role here: it defines a deterministic trajectory $\sigma \mapsto x(\sigma)$ that links a pure-noise sample $x(\sigma_{\max})$ to its corresponding clean sample $x(\sigma_{\min})$, and any method that shortens the number of sampling steps — a few-step diffusion sampler, or the straight-line flow of Section 2.8 — is, one way or another, trying to traverse this kind of trajectory with fewer function evaluations.

Score–denoiser equivalence (Tweedie). Training a diffusion model amounts to learning the score, typically indirectly through a denoiser $D(x_\sigma; \sigma)$ whose output predicts $\mathbb{E}[x_0 \mid x_\sigma]$. The score and the denoiser are related by Tweedie’s formula,

$$\nabla_x \log p_\sigma(x) = \frac{D(x; \sigma) - x}{\sigma^2}, \quad (2.7)$$

which we derive from the MMSE-denoiser identity in Appendix B.3, §C.1. The practical consequence is that denoising-score matching reduces to an ℓ_2 regression of the clean data

from its noised version:

$$\mathcal{L}_{\text{DSM}}(\sigma) = \mathbb{E}_{x_0, \epsilon} [\|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2]. \quad (2.8)$$



Equation (2.8) is the continuous-time counterpart of Equation (2.2): the tractable training objective is a weighted mean-squared error over the noise scale, and the network can equivalently predict the clean data x_0 , the noise ϵ , or the $v = \alpha_t \epsilon - \sigma_t x_0$ target of Salimans and Ho [20], the three being linearly interconvertible. The EDM baseline of this thesis uses the x_0 -prediction form through the preconditioning of Section 2.6; flow matching, by contrast, regresses a velocity field directly and uses none of these denoiser parameterisations.

2.6 EDM: The Elucidated Formulation

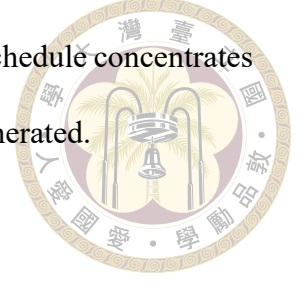
Karras et al. [9] proposed a unified parameterisation of diffusion training and sampling that cleans up a number of ad-hoc choices in earlier DDPM-style models. The EDM diffusion baseline of this thesis uses this formulation in full, and FlowCast borrows its data scale σ_{data} from it (Chapter 3). The EDM preconditioning scalings are not heuristics: they are the unique one-parameter family that makes the denoiser’s input statistics, target statistics, and per- σ loss scale simultaneously uniform across σ . A line-by-line derivation of the $c_{\text{skip}}, c_{\text{out}}, c_{\text{in}}, c_{\text{noise}}, \lambda(\sigma)$ formulas below is given in Appendix B.3, §C.3.

Noise schedule. EDM uses a ρ -spaced discretisation of noise levels between $\sigma_{\min}$ and

$\sigma_{\max}$:

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1} (\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho, \quad (2.9)$$

for $i = 0, \dots, N-1$, with $\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, and $\rho = 7$. The schedule concentrates noise levels near $\sigma_{\min}$, which is where most of the useful signal is generated.



Preconditioning. A naive denoiser $F_\theta(x_\sigma, \sigma)$ must cope with inputs whose magnitudes vary by orders of magnitude. EDM addresses this by wrapping the inner network in skip, input, output, and noise scalings:

$$D_\theta(x_\sigma; \sigma) = c_{\text{skip}}(\sigma) x_\sigma + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x_\sigma, c_{\text{noise}}(\sigma)), \quad (2.10)$$

where

$$\begin{aligned} c_{\text{skip}}(\sigma) &= \sigma_{\text{data}}^2 / (\sigma^2 + \sigma_{\text{data}}^2), & c_{\text{out}}(\sigma) &= \sigma \cdot \sigma_{\text{data}} / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, \\ c_{\text{in}}(\sigma) &= 1 / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}, & c_{\text{noise}}(\sigma) &= \frac{1}{4} \log \sigma. \end{aligned}$$

These scalings ensure that F_θ always sees inputs of unit variance and predicts an unnormalised residual, which is numerically stable across the full range of σ .

Training loss. EDM trains with a weighted denoising-score-matching loss:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{x_0, \sigma, \epsilon} \left[\lambda(\sigma) \|D_\theta(x_0 + \sigma\epsilon; \sigma) - x_0\|^2 \right], \quad (2.11)$$

with a log-normal distribution over σ and a Karras-specified weighting $\lambda(\sigma)$.

Heun sampler. EDM samples via a second-order Heun integrator of the PF-ODE. Given x_i at noise level σ_i , the next sample is

$$d_i = (x_i - D_\theta(x_i; \sigma_i)) / \sigma_i, \quad (2.12)$$

$$x_{i+1}^{(\text{euler})} = x_i + (\sigma_{i+1} - \sigma_i) d_i, \quad (2.13)$$

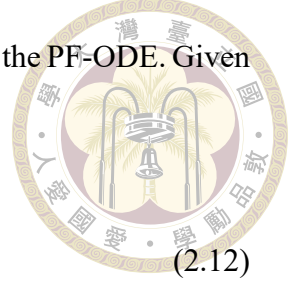
$$d'_i = (x_{i+1}^{(\text{euler})} - D_\theta(x_{i+1}^{(\text{euler})}; \sigma_{i+1})) / \sigma_{i+1}, \quad (2.14)$$

$$x_{i+1} = x_i + (\sigma_{i+1} - \sigma_i) \frac{1}{2} (d_i + d'_i). \quad (2.15)$$

Each step uses two denoiser evaluations except the final step where the correction is skipped. With $N = 18$ steps this yields $2N - 1 = 35$ NFE; the original StormCast paper reports a shorter $N \approx 13$ schedule (≈ 25 NFE). The baseline in this thesis keeps the native $N = 18$ schedule, and its 35-NFE Heun sampler is the cost figure that the straight-line FlowCast sampler of Chapter 3 is benchmarked against: every NFE that FlowCast saves at matched forecast quality is a direct operational speedup.

2.7 Conditioning in Diffusion Models

Conditional diffusion models can be implemented either by concatenating the condition c along the channel axis of the denoiser's input, or by classifier-free guidance (CFG) [7], which trains the network jointly on conditional and unconditional tasks and linearly combines their outputs at inference time. StormCast uses the simpler concatenation approach: the conditioning bundle is stacked with the noised target along the channel dimension and fed to the SongUNet. No classifier-free guidance is used. The original StormCast concatenates $c = \text{concat}(X_{t-1}, S_t, M_t, I)$; all three generative heads in this thesis — the EDM diffusion baseline, FlowCast, and MeanFlow — instead condition on the trimmed



bundle $\text{concat}(X_{t-1}, M_t, I)$, dropping the low-resolution background S_t identically (Section 3.5). Because the trim is applied to every head alike, the fundamental change between the diffusion baseline and the flow-matching heads remains the generative process itself, not the way conditioning enters the network.

2.8 Flow Matching, FlowCast, and MeanFlow

Conditional Flow Matching (CFM) is a recently-proposed alternative to score-based diffusion that learns the same kind of noise-to-data generative model — a deterministic flow integrated from a simple prior to the data distribution — but with a training objective that bypasses score matching entirely. CFM is the foundation on which the method of this thesis is built: the average-velocity **MeanFlow** objective (Section 3.6) generalises the straight-line CFM velocity field of the **FlowCast** baseline (Section 3.5). We therefore develop CFM in full — the simulation-free training principle, the straight-line I-CFM path of FlowCast, and finally the average-velocity identity of MeanFlow — giving it more depth than the condensed diffusion primer above.

Continuous normalising flows, simulation-free training. A continuous normalising flow (CNF; 3) parameterises a deterministic velocity field $v_\theta(x, t)$, $t \in [0, 1]$, and pushes a prior sample $x_0 \sim \mathcal{N}(0, I)$ along the ODE $\dot{x}_t = v_\theta(x_t, t)$ to obtain x_1 , which we would like to distribute as p_{data} . Classical maximum-likelihood CNF training requires evaluating $\log p_\theta(x_1)$ via the Jacobian trace over the whole trajectory — a prohibitive cost at image or video scale. Lipman et al. [11] and Tong et al. [28] instead train v_θ by direct regression against a target vector field $u_t(x \mid z)$ associated with a tractable conditional probability

path $p_t(x | z)$ indexed by a latent z :

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t \sim \mathcal{U}(0,1), z \sim q(z), x \sim p_t(x|z)} \left[\|v_\theta(x, t) - u_t(x | z)\|^2 \right] \quad (2.16)$$



The key theoretical result [11, Thm. 2] is that the gradient of (2.16) with respect to θ equals the gradient of the marginal flow-matching loss with the intractable marginal target field — so training the network against a conditional target yields an unbiased estimator of the true field. A derivation of this identity, along with the constants that are dropped because they do not depend on θ , is given in Appendix B.3, §C.5.

Independent CFM (I-CFM) and the straight-line path. The simplest instantiation, Independent CFM [28], takes $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ independent of $x_1 \sim p_{\text{data}}$, and defines the straight-line Gaussian probability path

$$p_t(x | x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad x_t = (1-t)x_0 + tx_1 + \sigma_{\text{path}} \eta, \quad \eta \sim \mathcal{N}(0, I). \quad (2.17)$$

Differentiating the conditional mean with respect to t gives the corresponding target vector field

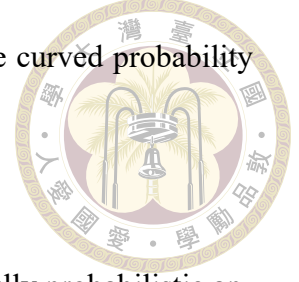
$$u_t(x | x_0, x_1) = x_1 - x_0, \quad (2.18)$$

which is independent of x — exactly the straight-line constant-velocity interpolation between the noise sample and the data sample. The training objective becomes

$$\mathcal{L}_{\text{I-CFM}} = \mathbb{E}_{t, x_0, x_1, \eta} \left[\|v_\theta((1-t)x_0 + tx_1 + \sigma_{\text{path}}\eta, t) - (x_1 - x_0)\|^2 \right]. \quad (2.19)$$

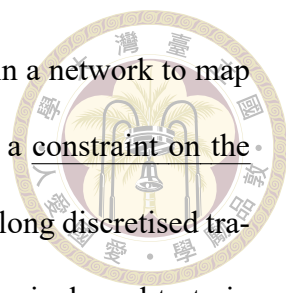
The path noise $\sigma_{\text{path}} > 0$ “thickens” the otherwise-singular conditional paths and regularises the vector-field fit, following Tong et al. [28]; FlowCast uses $\sigma_{\text{path}} = 0.01$. A

derivation of (2.18) from (2.17) and an explicit comparison with the curved probability flow of DDPM are in Appendix B.3, §C.6.



FlowCast. Ribeiro and Pucer [18] introduced FlowCast, the first fully probabilistic application of I-CFM to precipitation nowcasting. FlowCast trains a single U-Net backbone — in the original paper an Earthformer-style cuboid-attention network operating in a VAE latent space — to regress the straight-line vector field $v_\theta(x_t, t, c)$ conditioned on a past-observation bundle c . Sampling uses a 10-step Euler integrator of $\dot{x}_t = v_\theta(x_t, t, c)$. In a direct ablation on the SEVIR benchmark, FlowCast at 1 ODE step beat DDIM at 100 steps on CRPS and CSI, and matched or exceeded the state-of-the-art probabilistic nowcasting baselines CasCast and PreDiff at a fraction of the inference cost. Two features make it the natural flow-matching baseline for this thesis: (i) the straight-line ODE is by construction the trajectory any one-step sampler is trying to approximate, so the performance gap between 1-step and 10-step sampling is much smaller than for curved diffusion ODEs; and (ii) training is simulation-free and requires no teacher — FlowCast learns its velocity field directly from data in a single training run. In Section 3.5 we adapt FlowCast to the StormCast residual pipeline as a baseline, against which the average-velocity MeanFlow head of this thesis — which generalises exactly this straight-line objective — is measured on the same 2022 Taiwan validation year.

MeanFlow: average velocity for one-step generation. Even a straight-line flow is sampled by numerically integrating an ODE, and the integration is what keeps the NFE budget at $K \approx 10$: the marginal velocity field that the network actually learns is curved (the average over crossing conditional paths), so each Euler step still pays a truncation error against that curvature. A separate line of work asks whether the integral can be re-



moved from the sampler altogether. Consistency models [25, 27] train a network to map any point of a trajectory directly to its endpoint, but impose this as a constraint on the network’s behaviour — pairs of network outputs are pulled together along discretised trajectories — which is known to need carefully tuned discretisation curricula and to train unstably. MeanFlow [5] reaches the same goal from a field-theoretic direction: it defines the average velocity $u(z, r, t)$ — the displacement of the flow between two times $r < t$ divided by the interval length — as a new ground-truth field induced by the instantaneous velocity v , and derives an exact identity relating the two, $u = v - (t - r) \frac{d}{dt}u$ in the paper’s data-at- $t=0$ convention. Training regresses the network onto this identity using only the same conditional-velocity samples as CFM (the derivative term is evaluated by a single Jacobian–vector product and treated as a stop-gradient target), so no integral, teacher, curriculum, or auxiliary consistency constraint is needed; at $r = t$ the objective collapses to standard flow matching. Because the network outputs an average velocity, one evaluation transports a prior sample across the whole flow — 1–2 NFE sampling by construction. On ImageNet 256×256 , a from-scratch MeanFlow model reaches an FID of 3.43 at 1 NFE, a 50–70% relative improvement over the best previous one-step diffusion/flow models and close to many-step baselines at 500 NFE [5]. Section 3.6 transcribes the identity into this thesis’s noise-to-data convention and adapts MeanFlow to the StormCast residual pipeline as the generative residual head, trained under hyperparameters identical to FlowCast so that the average-velocity objective can be compared against its straight-line special case like for like.

2.9 Flow Matching versus Diffusion: Why Straight-Line Paths Help

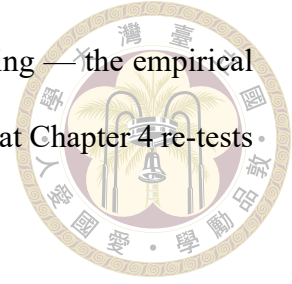


Both score-based diffusion and Conditional Flow Matching learn a deterministic ODE that carries a Gaussian prior sample to a data sample; the difference is the shape of the path that ODE follows, and that shape is what governs how few steps a sampler can use. It is worth stating the contrast explicitly, because it is the single fact that motivates the choice of FlowCast over diffusion in this thesis.

The diffusion path is curved. The EDM PF-ODE of Equation (2.6) has a velocity $-\sigma \nabla_x \log p_\sigma(x)$ that depends on the score of the noised marginal, which itself changes nonlinearly with σ . The trajectory $\sigma \mapsto x(\sigma)$ is therefore curved, and a low-order integrator (Euler, or even Heun) accumulates a truncation error that grows as the step size grows. This is why high-quality diffusion sampling sits in the 20–50-NFE band: below it, the integrator’s curvature error becomes visible, and it shows up first on the sparse, heavy-tailed precipitation channel where the score is most ill-conditioned at small σ .

The I-CFM path is a straight line. Independent CFM (Section 2.8) defines its conditional probability path as the straight-line interpolation $x_t = (1 - t)x_0 + tx_1$ between a prior sample x_0 and a data sample x_1 , with the constant target velocity $u_t = x_1 - x_0$. Along a single such conditional path the trajectory has zero curvature, so an explicit Euler step incurs no truncation error on the conditional path; what error remains comes only from the fact that the learned marginal velocity field averages over many crossing conditional paths. In practice this makes the gap between 1-step and 10-step flow-matching sampling

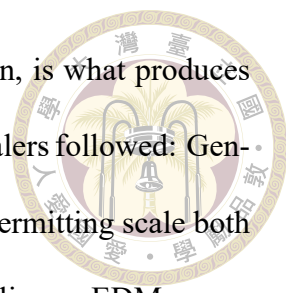
far smaller than the gap between 1-step and 25-step diffusion sampling — the empirical claim that Ribeiro and Pucer [18] verified on radar nowcasting and that Chapter 4 re-tests on the Taiwan RWRF domain.



No teacher, one training run. A third practical difference is organisational. The standard way to make diffusion cheap at inference is distillation: train a fast student to imitate a slow, pretrained teacher’s sampling output (progressive distillation [20], consistency distillation [25, 27], adversarial distillation [21]). Distillation needs a converged teacher, a second training stage, and careful matching of the teacher’s sampler and preconditioning. Rectified flow [12] showed that one can instead reshape the path itself so that straight-line integration suffices. FlowCast takes this route to its simplest conclusion: it trains the straight-line velocity field directly from data in a single run, with no teacher and no second stage. MeanFlow [5] — the method of this thesis — goes one step further along the same axis, learning the flow’s average velocity so that the integral disappears from the sampler entirely while training remains single-stage and teacher-free. This thesis therefore measures a single-run average-velocity forecaster (with its straight-line special case as a flow-matching baseline) against the diffusion model it would otherwise have to distil, rather than comparing distillation schemes among themselves.

2.10 Prior Generative Models for Precipitation Nowcasting

Generative precipitation modelling has converged on the same conclusion from several directions. DGMR [17] used a conditional GAN and was the first to argue convinc-



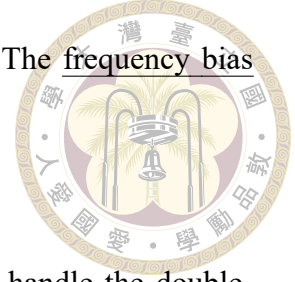
ingly that adversarial/generative training, not deterministic regression, is what produces skilful sharp radar nowcasts. Diffusion-based nowcasters and downscalers followed: GenCast [16] at global scale and StormCast [15] at regional convection-permitting scale both use score-based diffusion residuals, and NVIDIA’s CorrDiff line applies an EDM super-resolution residual to km-scale downscaling. The common cost of all of these is the multi-step diffusion sampler. Flow matching is the most recent entrant: Ribeiro and Pucer [18] introduced FlowCast as the first fully-probabilistic Independent-CFM nowcaster and reported that it matched or beat state-of-the-art diffusion nowcasters (CasCast, PreDiff) at a small fraction of their NFE on the SEVIR radar benchmark. To the best of our knowledge, the present thesis is the first to adapt conditional flow matching to a regional convection-permitting forecaster (as opposed to a radar-to-radar nowcaster), to anchor it to a frozen NWP-style regression mean in the StormCast two-stage decomposition, to bring the average-velocity MeanFlow objective [5] into weather forecasting at all, and to report a three-way diffusion-versus-CFM-versus-MeanFlow comparison with precipitation-specific metrics on the Taiwan domain.

2.11 Evaluation of Precipitation Forecasts

Pixelwise RMSE alone is a poor measure of precipitation-forecast quality because it rewards smooth, low-amplitude predictions that miss extreme events. We adopt a suite of complementary metrics, each introduced briefly here and defined formally in Chapter 4.

Categorical skill scores. For a precipitation threshold τ , every pixel in the forecast and observation is classified as wet ($\geq \tau$) or dry, producing a 2×2 contingency table with hits H , misses M , false alarms F , and correct negatives. The Critical Success Index

$CSI = H/(H + M + F)$ [22] is the standard operational score. The frequency bias $(H + F)/(H + M)$ diagnoses over- or under-prediction.



Fractions Skill Score. Roberts and Lean [19] introduced FSS to handle the double-penalty problem: a forecast that is slightly displaced in space but correct in structure should not be penalised as heavily as one that is wrong everywhere. FSS computes the fraction of pixels exceeding τ in a neighbourhood of radius n around each point and compares forecast and observation fractions.

Spectral diagnostics. The radially averaged logarithmic power spectral density (log-PSD) of the forecast and observation fields quantifies sharpness. A deterministic model that produces blurry outputs will have a log-PSD that drops off too quickly at high wavenumbers; a generative model that matches the data distribution will track the observation spectrum.

Rollout error growth. For an autoregressive forecaster, single-step error understates the operational risk. We therefore report errors at rollout horizons of 1, 3, 6, and 12 hours, which captures the cumulative effect of single-step residual-head error on autoregressive predictions.

Probabilistic scores (optional). For ensemble forecasts we additionally report the continuous ranked probability score (CRPS) and rank histograms for qppepre.





Chapter 3 Method

This chapter describes what was actually built. Section 3.1 states the residual-forecasting problem in precise terms. Section 3.2 describes the Taiwan RWRf dataset and its normalisation. Section 3.3 documents the underlying data sources and the preprocessing pipeline that turns them into the training Zarr store. Section 3.4 recaps the frozen regression mean and the EDM diffusion residual that serves as our reference baseline. Section 3.5 describes the FlowCast Conditional Flow Matching residual — the straight-line flow-matching baseline that the method of this thesis generalises — which we develop first because MeanFlow is defined relative to it. Section 3.6, the core of the chapter, describes **MeanFlow**, the average-velocity flow-matching residual that is the method of this thesis: it pushes the sampling budget down to one or two network evaluations with no ODE integration, while keeping every other design choice identical to the FlowCast baseline. Section 3.7 describes the dataset-specific loss modifications for the precipitation channel, which are shared by the diffusion baseline and both flow-matching heads. Section 3.8 documents the training infrastructure.



3.1 Problem Setup

We adopt the StormCast two-stage decomposition throughout. A frozen deterministic regression network F_ξ produces a conditional mean

$$M_t = F_\xi(X_{t-1}, S_t, I), \quad (3.1)$$

and a generative residual head models the residual $R_t = X_t - M_t$ conditioned on a bundle assembled from the previous state, the regression mean, and the invariants. The regression mean is the same frozen checkpoint for every method in this thesis, so the only object that varies is the generative residual head.

We compare three interchangeable residual heads, holding the dataset, the regression mean M_t , the backbone architecture, and the data scale σ_{data} fixed:

- **EDM diffusion (reference baseline).** A frozen-recipe EDMPrecond SongUNet D_ψ models the residual as a score-based diffusion and samples it by integrating the PF-ODE from σ_{max} to σ_{min} with a Heun sampler ($N = 18$ steps, $2N - 1 = 35$ NFE per hourly step). This is the model we set out to accelerate, and the reference against which the flow-matching heads are measured; it is the residual head of the original StormCast, save for the trimmed conditioning bundle that it shares with them (Section 3.5).
- **FlowCast (flow-matching baseline).** A FlowCastPrecond SongUNet velocity field v_θ models the residual as a straight-line Conditional Flow Matching ODE and samples it with K explicit Euler steps (K NFE per hourly step), where we sweep K from 1 to 50 (the head-to-head scoreboard reports $K \in \{10, 15, 20\}$). It is the straight-

line special case that MeanFlow generalises, retained as the natural flow-matching comparator.

- **MeanFlow (the method of this thesis).** A MeanFlowPrecond SongUNet u_θ models the average velocity of the same flow over an interval $[r, t]$ [5], so that one network evaluation transports the state across the whole interval with no ODE integration. Sampling uses K average-velocity segments at exactly K NFE per hourly step, with $K \in \{1, 2\}$ as the headline operating points (Section 3.6). Training hyperparameters are kept identical to the FlowCast baseline, and MeanFlow’s objective reduces exactly to FlowCast’s at zero interval length, making MeanFlow-versus-FlowCast an A/B test on the training objective alone.

In all cases the full prediction at each step is $\hat{X}_t = M_t + \hat{R}_t$, and the model is rolled out autoregressively, with the previous generated state feeding the next step’s regression input. The central empirical question of the thesis is whether the single-run, teacher-free MeanFlow head matches or beats the diffusion head at a substantially smaller NFE (and wall-clock) budget — $K \in \{1, 2\}$ network evaluations against the diffusion sampler’s 35, with the FlowCast baseline ($K \approx 10$) marking the straight-line flow-matching reference point in between — all at a matched training-sample budget so that the comparison reflects the generative trajectory rather than training compute. Neither flow head distils D_ψ or queries it during training; the three heads are trained independently on the same data.

3.2 Dataset: Taiwan RWRF

The training data live in a Zarr store under the project directory (see Appendix A.5). The store has three top-level groups.

LowRes conditioning S_t . Shape $(T, 24, 224, 128)$, where T is the hourly time index and the 24 channels are mean sea-level pressure, two-metre temperature, ten-metre winds, and the specific humidity, temperature, and zonal and meridional winds at four pressure levels (1000 hPa, 850 hPa, 500 hPa and 250 hPa). These channels are low-resolution re-gridded ERA5 fields that serve as synoptic-scale boundary conditioning.

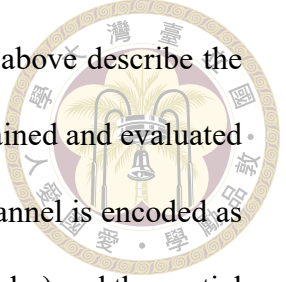
HighRes target X_t . Shape $(T, 4, 224, 128)$, with four channels: two-metre temperature (t2m), ten-metre zonal and meridional winds (u10, v10), and hourly accumulated precipitation (qpepre). The horizontal grid covers Taiwan from 21.6° to 25.6°N and 119.75° to 122.25°E at approximately 2 km resolution. The dataset splits into 21,216 training hours (2019-08 through 2021-12) and 8,760 validation hours (calendar year 2022).

Invariants I . Shape $(2, 224, 128)$, channels land-sea mask and orography. These are broadcast across time and concatenated into the conditioning bundle.

Normalisation. Per-channel means and standard deviations are computed on the training split and stored in the Zarr metadata. The HighRes statistics are

$$\mu_{\text{HR}} = (295.98, -1.58, -2.65, 0.182), \quad \sigma_{\text{HR}} = (5.61, 3.84, 5.66, 9.12), \quad (3.2)$$

which show that qpepre has a near-zero mean with standard deviation an order of magnitude larger than its mean — a direct fingerprint of its heavy-tailed, sparse distribution. Training and validation losses are computed on normalised fields; metrics in Chapter 4 are reported on denormalised fields with physical units.



Cleaned variant used for the reported experiments. The shapes above describe the original store. All FlowCast-versus-diffusion runs in this thesis are trained and evaluated on a cleaned re-export of the same data, in which the precipitation channel is encoded as $\log(1 + x)$ before standardisation (the `qpepre_log1p` option of the loader) and the spatial domain is centre-cropped to a U-Net-friendly 192×96 grid. The cleaned store covers 18,765 training hours (2019-08-01 through 2021-12-31) and 8,759 validation hours (calendar year 2022). The $\log(1 + x)$ encoding is the single most important preprocessing choice for the precipitation channel: it compresses `qpepre`’s dynamic range from a raw standard deviation of 9.12 mm h^{-1} to roughly 0.37 in log space, which makes the standardised residual far better-conditioned for every residual head (Section 4.7). Because the spatial crop changes the field of view, RMSE/CRPS magnitudes are not directly comparable to the legacy 224×128 baseline of Section 4.9; we flag this wherever the two are tabulated together. The regression mean and all three residual heads are trained on this same cleaned, log1p-encoded store, which is what the “train on the same `qpepre` encoding” gotcha in Section 4.12 refers to.

3.3 Data Sources and Preprocessing Pipeline

Section 3.2 described the Zarr store consumed by the training loop. This section describes where the underlying fields come from and how the preprocessing pipeline in `data_preprocessing/` assembles them. The design follows a low-resolution guidance, high-resolution learning strategy: a coarse global reanalysis provides synoptic-scale conditioning, and a convection-permitting regional model plus a radar-derived precipitation product provides the fine-scale target.

3.3.1 Raw data sources



ERA5 (low-resolution input, S_t). The ECMWF ERA5 reanalysis [6] at approximately 25 km spatial resolution supplies the synoptic-scale environmental state. From ERA5 we take four surface variables (mean sea-level pressure `mslp`, two-metre temperature `t2m`, and ten-metre winds `u10`, `v10`) and five upper-air variables (specific humidity `q`, temperature `t`, zonal wind `u`, meridional wind `v`, and geopotential height `z`) at four pressure levels $\{1000, 850, 500, 250\}$ hPa, for a total of $4 + 5 \times 4 = 24$ channels. These are the 24 LowRes channels listed in Section 3.2.

RWRF (high-resolution target, part of X_t). The Radar-assimilated Weather Research and Forecasting (RWRF) numerical model provides convection-permitting (≈ 2 km) analyses over Taiwan. We use hourly RWRF output (`wrfout_d01*_interp`) for the three near-surface dynamical channels: `t2m`, `u10`, `v10`. RWRF contains the fine-scale dynamic and thermodynamic structures that the complex Taiwan terrain imprints on near-surface weather, which is exactly what a regional convective-scale forecaster must learn.

QPEPRE (high-resolution target, precipitation channel). Quantitative Precipitation Estimation (QPEPRE) is a radar-based hourly-rainfall product distributed as fixed-grid text files (`qpepre_YYYYMMDDHHMM-YYYYMMDDHHMM_1_h.txt`), one per hour. Unlike the RWRF fields, which are model outputs, QPEPRE is an observational product and therefore serves as a more faithful ground truth for surface rainfall than any model-forecast precipitation field. It occupies the fourth HighRes channel, `qpepre`.

Table 3.1: Raw data sources used for training. Low-resolution ERA5 provides synoptic conditioning; RWRF and QPEPRE together supply the convection-permitting target.

	ERA5	RWRF	QPEPRE
Type	Reanalysis	Numerical model	Radar observation
Resolution	≈ 25 km	≈ 2 km	≈ 2 km
Role	LowRes input S_t	HighRes target X_t	HighRes target X_t
Channels used	24 (4 sfc + 5×4 levels)	3 (t2m, u10, v10)	1 (qpepre)
Native format	NetCDF	NetCDF (wrfout)	ASCII text

Invariants I . Two static fields are extracted once from an RWRF file and broadcast across time: the land–sea mask `lsm` and orography `orog`. These let the network condition on terrain without re-reading static fields at every time step.

Table 3.1 summarises the three sources.

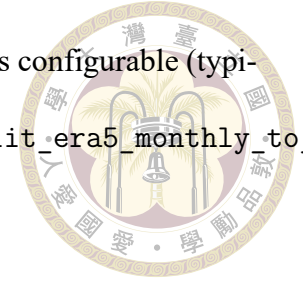
3.3.2 Preprocessing pipeline

The project ships two preprocessing stages under `data_preprocessing/`:

1. `nc_month_to_day/` — split monthly ERA5 NetCDF files into single-hour files.
2. `nc_to_zarr/` — harmonise ERA5, RWRF, and QPEPRE onto a common Taiwan grid and write a StormCast-style Zarr store.

Stage 1: monthly-to-hourly split. ERA5 is distributed as monthly NetCDF files, each containing an hourly time axis. The script `split_era5_monthly_to_daily.py` walks a root directory, opens every file matching a configured pattern, decodes the time axis with `netCDF4.num2date`, and writes one file per hour with a canonical `{variable}_YYYYMMDDTHH.nc` filename. The output uses uniform hours `since 1970-01-01 00:00:00` time units so that downstream indexing does not have to handle per-file calendar differences. The split

runs in parallel via a multiprocessing.Pool whose worker count is configurable (typically 20 processes on a workstation). Driver: nc_month_to_day/split_era5_monthly_to_daily.py; configuration: nc_month_to_day/config.yaml.



Stage 2: unified NetCDF-to-Zarr assembly. The main pipeline is nc_to_zarr/nc_to_zarr_pipeline driven by config.yaml. It is parameterised by four paths and a domain description:

era5-path, rwrf-path, qpepre-path, output-path

domain-size: [224, 128]

lon-bounds: [119.75, 122.25]

lat-bounds: [21.6, 25.6]

train-ranges / valid-ranges

For each requested split (train, valid, or combined), the pipeline performs the following steps.

1. **Time-stamp generation.** The configured date ranges are expanded into a dense hourly list between the start and end of each range.
2. **File indexing.** Three in-memory indices are built — one per source — that map timestamps to absolute file paths. For ERA5 this is a nested {variable → {date-key → path}} dictionary built by a regex on filenames; for RWRF it is {folder-name → path}; for QPEPRE it is {time-key → path} derived from the qpepre_{start}-{end}_1_h.txt filename convention. When multiprocessing is enabled the indices are serialised to a single JSON file .shared_index.json in the output directory so that worker processes can mmap-read the index instead of receiving a pickled copy.

3. **Combined validity precomputation.** Before processing anything, the pipeline walks the timestamp list once and flags every hour for which any of the required ERA5 variables, RWRf file, or QPEPRE file is missing. Only timesteps that pass this joint check are processed, and the Zarr store carries an explicit per-timestamp valid mask so that downstream code can skip gaps without re-scanning the raw files.

4. **Per-timestep field extraction.**

- ERA5: open the NetCDF, select the target time index, select the correct pressure level from the level dimension (for upper-air variables), crop to the latitude/longitude bounds, and resample to the configured 224×128 target grid.
- RWRf: read XLAT and XLONG to cache the native latitude/longitude, find the target Time index from the Times character variable, slice each requested variable to the domain, optionally apply per-variable transforms, and resample to the target grid.
- QPEPRE: load the text file, parse it into a {lat, lon, rain} scatter, grid it onto the native QPEPRE resolution, and resample to the 224×128 target grid. Because QPEPRE is already an observational product, we use it directly as the qpepre HighRes channel rather than the RWRf precipitation field.

5. **Grid harmonisation.** All three sources are resampled to a shared $H \times W = 224 \times 128$ grid covering lat $[21.6, 25.6]^\circ\text{N}$, lon $[119.75, 122.25]^\circ\text{E}$ at approximately 2 km spacing. This guarantees that the LowRes, HighRes, and invariant arrays share a pixel-aligned coordinate system, which is essential because the diffusion network conditions on LowRes and invariants by channel-wise concatenation.

6. **Streaming statistics.** As each training sample is filled into the data cube, a `StreamingStats` accumulator updates per-channel running means and standard deviations in single-pass Welford form. After the train split finishes, the stats are written to `stats/means.npy` and `stats/stds.npy` alongside the Zarr store. These are the normalisation statistics quoted in Equation 3.2.

7. **Zarr writing.** The assembled cube is written as an `xarray.Dataset` with dimensions (time, channel, y, x), chunked along time and space for efficient random access, and consolidated (`zarr.consolidate_metadata`) so that the training loader can open the store with a single metadata read. Invariants are written once, without a time dimension, as a separate Zarr store under `invariants/invariants.zarr`.

Output layout. A successful run produces the tree

<output-path>/

LowRes/

stormcast_test_train.zarr # (T_train, 24, 224, 128)

stormcast_test_valid.zarr # (T_valid, 24, 224, 128)

stats/{means.npy, stds.npy, channels.txt}

HighRes/

stormcast_test_train.zarr # (T_train, 4, 224, 128)

stormcast_test_valid.zarr # (T_valid, 4, 224, 128)

stats/{means.npy, stds.npy, channels.txt}

invariants/

invariants.zarr # (2, 224, 128)

which matches the layout documented in `zarr_metadata.json` and consumed by `data_loader_rwrf_era`. For the experiment used in this thesis the pipeline produces $T_{\text{train}} = 21,216$ hourly samples (2019-08-01 through 2021-12-31) and $T_{\text{valid}} = 8,760$ hourly samples (calendar year 2022).

Why this matters here. Three features of this pipeline are load-bearing for the work in this thesis:

- Pixel-aligned multi-source grids. The diffusion baseline and FlowCast both consume conditioning and target on the same grid, so residual losses can be computed pixel-wise without any on-the-fly re-gridding.
- Explicit valid mask. Roughly 10–15 % of hours fail the joint-validity check (usually a missing QPEPRE file). Both loaders and metrics respect the mask, so downstream scores are never contaminated by NaN-filled cells.
- Frozen training-split statistics. Because the Zarr store commits to a single pair of $(\mu_{\text{HR}}, \sigma_{\text{HR}})$ at write time, all residual heads see identical normalised inputs — a prerequisite for the σ_{data} -matching discussion in Section 3.4.

3.3.3 Target Distributions: Sparsity, Gaussianity, and the Diffusion Prior

Diffusion models place a Gaussian prior on their target: the forward process is $x_{\sigma} = x_0 + \sigma \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$, and the denoiser is trained under the implicit assumption that the marginals of x_0 are reasonably well-behaved relative to that noise scale. How closely each of the four HighRes channels actually conforms to this assumption directly shapes how the

generative residual head must be regularised. Figure 3.1 shows per-channel histograms (with a fitted Gaussian overlay) and Gaussian QQ plots computed over the full training split ($n = 6.08 \times 10^8$ pixels per channel). The streaming statistics are summarised in Table 3.2.

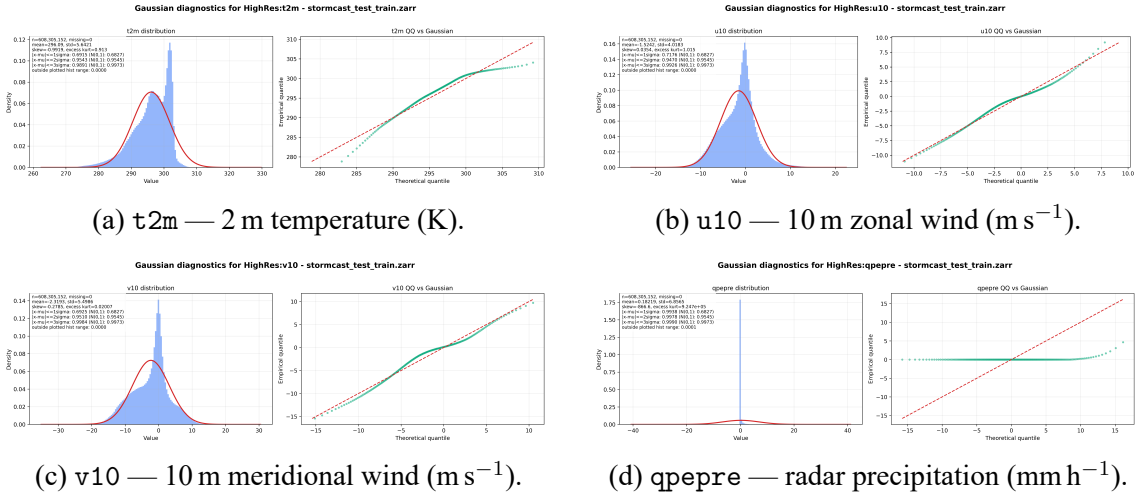
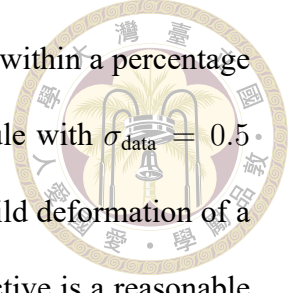


Figure 3.1: Per-channel distribution and Gaussian QQ plots for the four HighRes target channels over the 2.5-year training split. Left subpanels: empirical density (red) with a Gaussian fit of matched (μ, σ) (blue). Right subpanels: empirical quantiles against theoretical Gaussian quantiles; the red dashed line is $y = x$ for a perfect Gaussian. Generated by `data_preprocessing/inspect_tools/plot_high_res_vars.py`.

Table 3.2: Summary statistics of the four HighRes target channels over the training split. A standard Gaussian has skewness = 0, excess kurtosis = 0, and concentrates 68.3/95.5/99.7 % of its mass within $1/2/3\sigma$.

Channel	Mean	Std	Skewness	Excess kurt.	% in 1σ	% in 3σ
$t2m$	296.09	5.64	-0.99	0.91	69.2	98.9
$u10$	-1.52	4.02	0.04	1.01	71.8	99.3
$v10$	-2.32	5.50	-0.28	0.02	69.3	99.8
$qppepre$	0.18	6.86	-866.6	9.25×10^5	99.4	99.9

The three near-surface state channels are approximately Gaussian. $u10$ is the closest: almost zero skewness, mild leptokurtosis, and a QQ plot that tracks the diagonal out to roughly $\pm 4\sigma$ before fanning into the heavy tails associated with gust events. $v10$ is almost symmetric with essentially zero excess kurtosis. $t2m$ is the least symmetric of the three: left-skewed (skewness = -0.99) with a secondary mode around 280 K corresponding to winter cold surges over northern Taiwan and the surrounding sea. For all three channels the



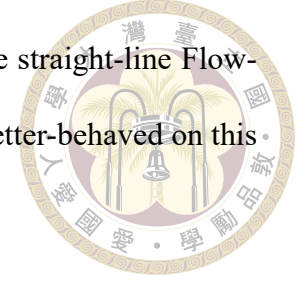
empirical $1\sigma/2\sigma/3\sigma$ coverage agrees with the Gaussian reference to within a percentage point, which is exactly the regime for which the EDM noise schedule with $\sigma_{\text{data}} = 0.5$ was designed: the generative residual head sees each channel as a mild deformation of a unit-variance Gaussian after normalisation, and the standard ℓ_2 objective is a reasonable fit.

qpepre is a different object entirely. Its marginal is dominated by an atom at zero — the overwhelming majority of pixel-hours are dry — with a long, very sparse tail of heavy rainfall events. The raw statistics make this quantitative: the coverage inside $\pm 1\sigma$ is 99.4 % rather than the Gaussian 68.3 %, which is the signature of a near-delta distribution whose standard deviation is inflated entirely by the tail. The empirical moments reported in Table 3.2 (skewness ≈ -867 , excess kurtosis $\approx 10^6$) are formally enormous because the raw qpepre field uses a -9999 sentinel for missing observations that is only filtered at the next pipeline stage; the physical tail above zero is still extreme (excursions beyond 100 mm h^{-1} occur during typhoon landfalls), but the numbers headline the practical message: the qpepre histogram looks nothing like a Gaussian, and the QQ plot sits on a flat shelf near zero that peels almost vertically at the upper end.

This mismatch has three consequences for residual generative modelling that recur throughout the rest of this thesis:

1. Score matching on raw qpepre is ill-posed at small σ . When the target distribution is almost a point mass at zero, the diffusion score $\nabla_x \log p_\sigma(x)$ diverges as $\sigma \rightarrow \sigma_{\min}$ — one reason the EDM baseline needs many small- σ steps to render precipitation. Learning the residual relative to the regression mean M_t partially absorbs this, but the residual inherits the same atom-plus-tail shape, and the $\log(1+x)$ encoding of

Section 3.7 is what finally compresses its dynamic range. The straight-line Flow-Cast velocity field has no score term and is correspondingly better-behaved on this channel.



2. Pixel-wise losses undersell the tail. Because dry pixels vastly outnumber rainy ones, an ℓ_2 loss converges to a “slightly wet everywhere” solution that scores well on RMSE but destroys the convective structure we actually care about. This is the motivation for the radial log-PSD spectral regulariser applied specifically to qppepre in the FlowCast loss (Section 3.7), and for the precipitation-specific metrics (CSI, FSS, HSS, frequency-of-exceedance at fixed rainfall thresholds) reported in Chapter 4.
3. The channel budget matters. Because only one of the four output channels is pathological, per-channel loss weights β_k are a natural knob: the three near-Gaussian fields can be trained with a standard denoising objective while qppepre receives heavier regularisation and, if needed, an asinh or $\log(1 + x)$ transform to compress its dynamic range before the noise is added. Having only four target channels (versus the 99 of global StormCast) makes such per-channel bespoke treatment computationally trivial.

In short, Figure 3.1 is the quantitative justification for every “qppepre is the hard channel” design decision that follows: three channels live comfortably inside the Gaussian prior that diffusion assumes, and one does not.

3.4 Regression Mean and the EDM Diffusion Baseline

The reference model is the two-stage StormCast described in Section 2.3, with the EDM hyperparameters of Section 2.6. It supplies both the frozen regression mean that

FlowCast reuses unchanged and the diffusion residual that FlowCast is compared against.

- **Regression F_ξ .** A StormCastUNet taking (X_{t-1}, S_t, I) as input and predicting M_t .

The checkpoint is used without modification or further training, and is shared by the diffusion baseline, FlowCast, and MeanFlow so that all three residual heads model a residual relative to the same mean.

- **Diffusion baseline D_ψ .** An EDMPrecond SongUNet trained to denoise the residual

$R_t = X_t - M_t$ conditioned on $c_t = \text{concat}(X_{t-1}, S_t, M_t, I)$. EDM hyperparameters:

$\sigma_{\min} = 0.002$, $\sigma_{\max} = 80$, $\sigma_{\text{data}} = 0.5$, $\rho = 7$; sampling is the Heun integrator of

Section 2.6. The baseline is trained from scratch on the same cleaned dataset as

FlowCast and evaluated at the same ≈ 2 M-training-sample budget (the nearest saved checkpoint; see Section 4.1).

Both networks are instantiated through the factory `get_preconditioned_architecture` in `stormcast/utils/nn.py`, which selects the EDM or FlowCast preconditioning wrapper by name. The conditioning bundle is assembled by `build_network_condition_and_target`, which handles the channel-wise concatenation and the one-hot time-offset encoding used in the rollout.

Why the baseline matters for a fair comparison. The value of the comparison in Chapter 4 rests on holding everything except the generative trajectory fixed. The diffusion baseline, FlowCast, and MeanFlow share the dataset, the frozen F_ξ , the SongUNet backbone (`channel_mult = [1, 2, 2, 2, 2]`), and the data scale $\sigma_{\text{data}} = 0.5$ that anchors the residual statistics. The EDM data scale matters here for a subtle reason: both flow heads standardise their residual by exactly the same σ_{data} (Sections 3.5 and 3.6), so the per-channel loss

scaling and the regression-mean anchoring are numerically identical across all heads. A mismatched σ_{data} would silently shift the residual distribution one head sees relative to the others and make any quality difference uninterpretable.



3.5 The FlowCast Baseline (Conditional Flow Matching)

FlowCast is the straight-line flow-matching baseline of this thesis, and we develop it first because the MeanFlow method of Section 3.6 is defined relative to it — MeanFlow reuses FlowCast’s conditioning, standardised-residual pipeline, backbone, and losses unchanged, and its objective reduces exactly to FlowCast’s at zero interval length. FlowCast replaces the diffusion baseline’s curved PF-ODE with a simulation-free straight-line flow: a single network is trained from scratch to regress the target vector field $u_t = x_1 - x_0$ of Independent CFM (Section 2.8), and sampling is a handful of Euler steps. FlowCast is not a distillation method — it never queries the EDM diffusion model and uses no teacher at all — it is a one-training-run residual head that learns its velocity field directly from data. The same StormCast residual decomposition is preserved: the deterministic regression mean M_t is produced by the frozen F_ξ , and FlowCast learns only the residual $R_t = X_t - M_t$ conditioned on a bundle c_t assembled by `build_network_condition_and_target` in `stormcast/utils/nn.py`.

Conditioning bundle. FlowCast and the EDM diffusion baseline are conditioned on exactly the same channel bundle. Both share the config setting

```
regression_conditions: ["state", "background", "invariant"]
diffusion_conditions: ["state", "regression", "invariant"]
```

which is identical in `stormcast/config/model/flowcast.yaml` and `stormcast/config/model/diffusion.yaml`: the frozen regression network F_ξ is fed the full (X_{t-1}, S_t, I) , but the generative network — diffusion or flow — is conditioned only on

$$c_t = \text{concat}(X_{t-1}, M_t, I). \quad (3.3)$$

The low-resolution background S_t is deliberately omitted from the direct conditioning path of every head (a deviation from the original StormCast, which concatenates S_t alongside M_t); the synoptic-scale signal in S_t enters the residual only through the regression mean $M_t = F_\xi(X_{t-1}, S_t, I)$, which already summarises what the residual needs to know about it. Trimming S_t removes 24 input channels at no measured cost, and — crucially for a controlled comparison — the trim is applied identically to all heads (the MeanFlow config carries the same two condition lists verbatim). Keeping "regression" in the diffusion-conditions list anchors every residual to the same M_t , so the only thing that differs between the EDM, FlowCast, and MeanFlow residuals is the generative trajectory, not what they condition on. This is what makes the head-to-head comparison of Chapter 4 fair.

Standardised-residual pipeline. FlowCast operates on the residual in a standardised space. The loss (`stormcast/utils/flowcast_loss.py`) divides the raw residual by $\sigma_{\text{data}} = 0.5$ before building the flow trajectory, and the sampler (`stormcast/utils/nn.py:flowcast_model_forward`) multiplies the integrated state by σ_{data} on output. Keeping σ_{data} matched to the EDM baseline means the regression-mean anchoring statistics and the per-channel loss scaling are identical across the residual heads. Concretely, with $x_1 = R_t/\sigma_{\text{data}}$ and $x_0 \sim \mathcal{N}(0, I)$, the I-CFM interpolated state is

$$x_t = (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta, \quad t \sim \mathcal{U}(t_\epsilon, 1 - t_\epsilon), \quad \eta \sim \mathcal{N}(0, I), \quad (3.4)$$

with $\sigma_{\text{path}} = 0.01$ from Tong et al. [28] and $t_\varepsilon = 10^{-5}$ clamping the endpoints. The small $\sigma_{\text{path}}\eta$ perturbation thickens the otherwise-Dirac conditional path $p_t(x_t | x_0, x_1)$ into a narrow Gaussian tube, separating it from the deterministic-interpolant ($\sigma_{\text{path}} = 0$) limit. The “Independent” in I-CFM refers instead to the coupling: x_0 and x_1 are drawn independently [28], as opposed to the minibatch optimal-transport coupling of OT-CFM.

Velocity predictor. The FlowCast network is the FlowCastPrecond module at `stormcast/`
`utils/flowcast_precond.py`, instantiated through the factory call `get_preconditioned_architecture(...)` at `stormcast/utils/nn.py`. The factory wraps a SongUNet with `channel_mult=[1, 2, 2, 2, 2]`, identical to the EDM baseline’s backbone. Unlike EDMPrecond there are no $c_{\text{skip}}/c_{\text{in}}/c_{\text{out}}$ scalings — the network output is the velocity estimate:

$$v_\theta(x_t, t, c_t) = \text{SongUNet}(\text{concat}[x_t, c_t], t \cdot \tau), \quad (3.5)$$

where $\tau = 1000$ (the `time_scale` field in `stormcast/config/model/flowcast.yaml`) rescales the flow time $t \in [0, 1]$ before it enters the SongUNet’s positional-embedding layer. The SongUNet positional embedding was designed for diffusion-timestep-valued inputs with dynamic range of order 10^2 – 10^3 ; feeding raw $t \in [0, 1]$ would collapse the embedding to a nearly-constant signal. $\tau = 1000$ restores enough dynamic range for the embedding to carry useful time information without changing the SongUNet itself, so no architectural changes from the diffusion baseline’s backbone are needed.

Loss. The training loss is the standardised-space I-CFM regression

$$\mathcal{L}_{\text{FC}} = \mathbb{E}[\|v_\theta(x_t, t, c_t) - (x_1 - x_0)\|_{2,\beta}^2] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}(\hat{x}_1, x_1), \quad (3.6)$$

Algorithm 1 FlowCast training step (I-CFM on the standardised residual)

Require: frozen F_ξ , FlowCast velocity field v_θ , schedule $(\sigma_{\text{data}}, \sigma_{\text{path}}, t_\varepsilon)$, channel weights β , spectral weight λ_{spec}

- 1: sample mini-batch $\{(X_{t-1}, S_t, X_t)\}$; form $M_t \leftarrow F_\xi(X_{t-1}, S_t, I)$, $R_t \leftarrow X_t - M_t$
- 2: build conditioning $c_t \leftarrow \text{concat}(X_{t-1}, M_t, I)$ $\triangleright$ note: S_t reaches v_θ only via M_t
- 3: standardise: $x_1 \leftarrow R_t / \sigma_{\text{data}}$
- 4: sample $x_0 \sim \mathcal{N}(0, I)$, $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, $\eta \sim \mathcal{N}(0, I)$
- 5: build interpolant: $x_t \leftarrow (1 - t) x_0 + t x_1 + \sigma_{\text{path}} \eta$
- 6: target field: $u_t \leftarrow x_1 - x_0$
- 7: predict: $\hat{v} \leftarrow v_\theta(x_t, t, c_t)$
- 8: pointwise: $\mathcal{L}_{\text{pw}} \leftarrow \|\hat{v} - u_t\|_{2, \beta}^2$
- 9: implied clean: $\hat{x}_1 \leftarrow x_t + (1 - t) \hat{v}$
- 10: spectral: $\mathcal{L}_{\text{spec}} \leftarrow \frac{1}{|\mathcal{K}|} \sum_{\kappa} |\log P_{\hat{x}_1^{\text{qpepre}}}(\kappa) - \log P_{x_1^{\text{qpepre}}}(\kappa)|$
- 11: $\mathcal{L} \leftarrow \mathcal{L}_{\text{pw}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}$
- 12: AdamW step on θ ; EMA update $\theta^- \leftarrow 0.999 \theta^- + 0.001 \theta$

where $\|\cdot\|_{2, \beta}$ is the per-channel-weighted L2 norm from Section 3.7 with $\beta = (1, 1, 1, 2)$ (the `channel_weights` field), $\hat{x}_1 = x_t + (1 - t) v_\theta$ is the flow’s implied clean-residual estimate at the current time, and $\mathcal{L}_{\text{spec}}$ is the radial log-PSD regulariser of Section 3.7 with `spectral_channels` = {qpepre} and $\lambda_{\text{spec}} = 0.1$. Operating the spectral term on the implied-clean field $\hat{x}_1$ rather than on the raw velocity keeps the spectrum interpretable. Algorithm 1 summarises the per-step update.

Optimisation. Following the FlowCast paper we use the AdamW optimiser with $\beta_{\text{Adam}} = (0.9, 0.999)$, learning rate 5×10^{-4} , and weight decay 10^{-4} on all parameters. The learning-rate schedule is a 4,000-step linear warmup ($\sim 1\%$ of the training budget) into a cosine decay to $0.01 \times 5 \times 10^{-4} = 5 \times 10^{-6}$ over the remaining steps, implemented inline in `trainer_flowcast.py`. Gradients are globally clipped at 1.0, and per-parameter NaN gradients are sanitised with `torch.nan_to_num` (a safety net borrowed from the EDM trainer). Training runs for a total of 400,000 optimiser steps at a global batch size of 96 (set by the launcher `stormcast/zettabyte_scripts/train_flowcast.sh`, which overrides the config default of 64), split across the available GPUs through PyTorch’s

DistributedDataParallel; the per-GPU micro-batch is `batch_size_per_gpu = "auto"`, which the trainer resolves to $96/W$ for W workers. This batch size is what fixes the step-to-sample conversion of the matched-budget comparison in Section 4.1 (FlowCast step 20,833 $\approx$ 2 M samples at batch 96, versus EDM step 31,250 at batch 64).

EMA shadow network. A second FlowCastPrecond instance is kept as an exponential-moving-average shadow of the online weights, with a fixed decay of 0.999. The shadow is updated after every optimiser step by `ExponentialMovingAverage.update` and copied into the shadow network by `apply_shadow(stormcast/utils/ema.py)`. A single fixed decay suffices throughout training because the FlowCast target — the straight-line velocity field — is stationary; there is no shrinking time discretisation for the decay to track. The shadow weights, not the online weights, are used at validation and inference time, and are saved to `ema_state.pt` alongside each regular checkpoint. Loading the raw `checkpoint_*.pt` instead of `ema_state.pt` yields visibly noisier samples.

Sampling. Generation integrates the learned ODE $\dot{x}_t = v_\theta(x_t, t, c_t)$ from $t = 0$ to $t = 1$ with a fixed-step solver. The implementation `flowcast_model_forward` in `stormcast/`
`utils/nn.py` exposes two choices. The default is explicit Euler with K steps:

$$x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, c_t), \quad \Delta t = 1/K, \quad x_{t=0} \sim \mathcal{N}(0, I), \quad (3.7)$$

which costs K NFEs. The alternative `solver="midpoint"` is a single-stage Runge–Kutta second-order scheme:

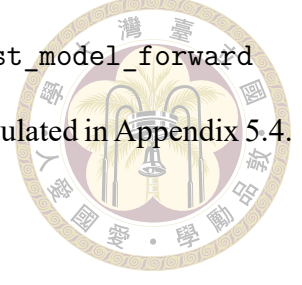
$$v_{1/2} = v_\theta(x_t, t, c_t), \quad \tilde{x} = x_t + \frac{1}{2}\Delta t v_{1/2}, \quad x_{t+\Delta t} = x_t + \Delta t v_\theta(\tilde{x}, t + \frac{1}{2}\Delta t, c_t), \quad (3.8)$$

which costs $2K$ NFEs but is globally second-order accurate ($O(\Delta t^2)$ global error, versus explicit Euler's $O(\Delta t)$). The sampler finishes by de-standardising: $\hat{R}_t = \sigma_{\text{data}} x_{t=1}$ and $\hat{X}_t = M_t + \hat{R}_t$. Validation inside the training loop uses $K = \text{valid_num_steps} = 10$ Euler steps, while the main experiments report the scoreboard at $K \in \{10, 15, 20\}$ and sweep K from 1 to 50, placing each cost point against the EDM Heun baseline on the NFE/wall-clock–quality trade-off of Chapter 4.

Why a straight-line flow is the right residual head. Three properties of I-CFM make FlowCast attractive against the diffusion baseline rather than merely a faster imitation of it. First, the straight-line probability path is by construction what a one-step deterministic generator is trying to approximate, so the FlowCast $K = 1$ and $K = 2$ numbers are a strong reference point for the best a few-step ODE sampler trained on the same data can do — and the gap to the $K = 10$ sampler is small (Chapter 4), unlike the steep degradation a diffusion sampler suffers below ≈ 20 NFE. Second, the training is teacher-free and single-stage, so FlowCast avoids the second training run, the sampler/preconditioning matching, and the EMA-coupling subtleties that a distillation of the diffusion baseline would require. Third, and specific to precipitation, Ribeiro and Pucer [18] report that I-CFM is noticeably more stable than score matching on sparse heavy-tailed fields — a claim directly relevant to the qppepre channel and one we re-test on the Taiwan domain.

Implementation pointers. Entry point `stormcast/train_flowcast.py`; training loop `stormcast/utils/trainer_flowcast.py(flowcast_training_loop)`; loss `stormcast/`
`utils/flowcast_loss.py(FlowCastLoss)`; preconditioner `stormcast/utils/flowcast_precond.py`
`(FlowCastPrecond)`; EMA helper `stormcast/utils/ema.py(ExponentialMovingAverage)`;
`configs/stormcast/config/flowcast.yaml`, `stormcast/config/model/flowcast.yaml`,

`stormcast/config/training/flowcast.yaml`; `sampler flowcast_model_forward` in `stormcast/utils/nn.py`. The numerical hyperparameters are tabulated in Appendix 5.4.



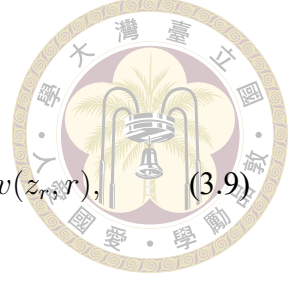
3.6 The MeanFlow Residual (Average-Velocity Flow Matching)

FlowCast still pays for its samples with $K \approx 10$ Euler steps, because what it learns is the instantaneous velocity of a marginal field whose trajectories are curved (Section 2.8): every Euler step incurs truncation error against that curvature, and the error only vanishes as K grows. MeanFlow [5] removes the integration from the sampler instead of refining it. It trains the network to predict the average velocity of the flow over a whole interval $[r, t]$, so that a single evaluation transports the state across the interval exactly — no ODE solver, and therefore no discretisation error to amortise. We adapt MeanFlow to the StormCast residual pipeline as the generative residual head of this thesis: it reuses the conditioning bundle c_t , the standardised-residual convention, the SongUNet backbone, the per-channel loss weights, and the spectral regulariser of the FlowCast baseline unchanged, and differs from it only in the training objective and the (correspondingly cheaper) sampler. MeanFlow is likewise teacher-free and trained from scratch in a single run.

Average velocity. Work in the standardised residual space of Section 3.5: $x_1 = R_t/\sigma_{\text{data}}$ is the data endpoint, $x_0 \sim \mathcal{N}(0, I)$ the prior, and the linear path $z_s = (1-s)x_0 + sx_1$ runs from noise at $s = 0$ to data at $s = 1$ with conditional velocity $v = x_1 - x_0$. (MeanFlow uses the pure interpolant, $\sigma_{\text{path}} = 0$: the identity below differentiates along the exact trajectory, so the Gaussian thickening used by FlowCast is dropped.) For two times $r \leq t$ the average

velocity is the displacement over $[r, t]$ divided by the interval length,

$$u(z_r, r, t) \triangleq \frac{1}{t-r} \int_r^t v(z_\tau, \tau) d\tau, \quad \lim_{t \rightarrow r} u(z_r, r, t) = v(z_r, r), \quad (3.9)$$



where $v(\cdot, \tau)$ is the marginal velocity field of Section 2.8. Like v , the field u is a property of the underlying flow, not of any network. Its defining property is that it makes transport across an interval a single algebraic step,

$$z_t = z_r + (t-r) u(z_r, r, t), \quad (3.10)$$

exactly, with no solver: if u is known, one evaluation at $(r, t) = (0, 1)$ maps noise to data.

The cost has been moved from inference into training: Eq. (3.9) contains an intractable path integral, so u cannot be regressed directly.

The MeanFlow identity. The integral is eliminated by differentiation rather than quadrature. Rewrite Eq. (3.9) as $(t-r) u(z_r, r, t) = \int_r^t v d\tau$ and differentiate with respect to the state time r , holding t fixed, with z_r moving along the trajectory ($dz_r/dr = v(z_r, r)$, $dt/dr = 0$). The product rule on the left and the fundamental theorem of calculus on the right give $-u + (t-r) \frac{d}{dr} u = -v(z_r, r)$, i.e.

$$\boxed{u(z_r, r, t) = v(z_r, r) + (t-r) \frac{d}{dr} u(z_r, r, t)}, \quad (3.11)$$

with the total derivative expanding into partials along the trajectory tangent,

$$\frac{d}{dr} u(z_r, r, t) = v(z_r, r) \partial_z u + \partial_r u. \quad (3.12)$$

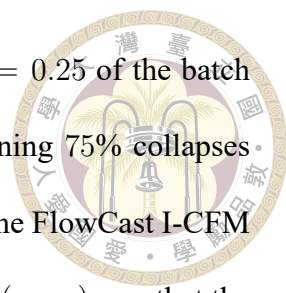
Equation (3.11) is the MeanFlow identity of Geng et al. [5] transcribed to this thesis's noise-to-data convention: Geng et al. place data at $s = 0$ and differentiate with respect to t , obtaining $u = v - (t - r) \frac{d}{dt}u$; here data sits at $s = 1$, so the differentiation runs through the lower limit and the sign flips. The identity relates the field we want (u) to the field we can sample (v) using only quantities available at a single point of a single trajectory — the path integral is gone. At $t = r$ it collapses to $u = v$, recovering plain flow matching as a boundary condition.

Training objective. A network $u_\theta(z_r, r, t, c_t)$ is encouraged to satisfy Eq. (3.11) by regressing onto the identity's right-hand side with the network's own derivative bootstrapped in:

$$\mathcal{L}_{\text{MF}} = \mathbb{E} \left[\text{sg}(w) \left\| u_\theta(z_r, r, t, c_t) - \text{sg}(u_{\text{tgt}}) \right\|_{2,\beta}^2 \right] + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}, \quad u_{\text{tgt}} = v + (t - r)(v \partial_z u_\theta + \partial_r u_\theta), \quad (3.13)$$

where $v = x_1 - x_0$ is the conditional I-CFM velocity standing in for the marginal field (the same substitution FlowCast makes, valid in expectation), sg is stop-gradient, and $\|\cdot\|_{2,\beta}$ is the channel-weighted norm of Section 3.7 with the same $\beta = (1, 1, 1, 2)$ as FlowCast. Three implementation choices follow Geng et al. [5]:

- **JVP bootstrap with stop-gradient.** The derivative in u_{tgt} is evaluated by a single forward-mode Jacobian–vector product through u_θ with tangent $(v, 1, 0)$ for (z, r, t) — one extra forward-mode pass, no higher-order gradients, because the whole target is stop-gradient (`torch.func.jvp` under `no_grad`). A network achieving zero loss satisfies the identity, and hence the definition Eq. (3.9).
- **Two-time sampling with partial collapse.** Each sample draws two i.i.d. times from



$\mathcal{U}(t_\epsilon, 1 - t_\epsilon)$ and sorts them into $r \leq t$. Only a fraction $\rho_{\text{MF}} = 0.25$ of the batch keeps $r < t$ and trains on the bootstrapped target; the remaining 75% collapses $t \leftarrow r$, for which $u_{\text{tgt}} = v$ and the objective reduces exactly to the FlowCast I-CFM regression. The collapsed subset pins the boundary condition $u(\cdot, r, r) = v$ that the $r < t$ subset propagates outward; Geng et al. [5] find $\rho_{\text{MF}} = 0.25$ optimal, and the JVP overhead is confined to a quarter of the batch. The prediction pass itself runs once over the full batch, which keeps distributed training simple.

- **Adaptive weighting.** The per-sample weight $w = (\|\Delta\|_{2,\beta}^2 + \varepsilon_w)^{-p}$ with $p = 1$, $\varepsilon_w = 10^{-3}$, applied with stop-gradient, implements the adapted-loss-weight scheme that Geng et al. [5] found necessary for stable bootstrapped training (their Tab. 1e); $p = 0$ recovers plain weighted MSE.

The spectral term $\mathcal{L}_{\text{spec}}$ is the radial log-PSD regulariser of Section 3.7, applied — as in FlowCast — on the implied clean residual $\hat{x}_1 = z_r + (1 - r)u_\theta$, but restricted to the collapsed ($t = r$) subset of the batch where that extrapolation is the flow-matching one and the construction stays directly comparable to Eq. (3.6). Algorithm 2 summarises the update.

Network parameterisation: two times into one backbone. The network is the MeanFlowPrecond module (`stormcast/utils/meanflow_precond.py`), wrapping the same SongUNet backbone (`channel_mult = [1, 2, 2, 2, 2]`) as the EDM and FlowCast heads. The only new requirement is injecting two times instead of one. The state time r takes FlowCast’s route: it is scaled by $\tau = 1000$ and fed through the SongUNet’s diffusion-time positional embedding (Eq. (3.5)). The interval length $\Delta = t - r$ enters through the SongUNet’s otherwise-unused augmentation-label pathway as a fixed 32-dimensional $\sin / \cos$ Fourier feature

Algorithm 2 MeanFlow training step (average-velocity matching on the standardised residual)

Require: frozen F_ξ , average-velocity network u_θ , scale σ_{data} , clamp t_ε , **MeanFlow ratio** ρ_{MF} , adaptive (p, ε_w) , channel weights β , spectral weight λ_{spec}

- 1: sample mini-batch $\{(X_{t-1}, S_t, X_t)\}$; form $M_t \leftarrow F_\xi(X_{t-1}, S_t, I)$, $R_t \leftarrow X_t - M_t$
- 2: build conditioning $c_t \leftarrow \text{concat}(X_{t-1}, M_t, I)$; standardise $x_1 \leftarrow R_t / \sigma_{\text{data}}$
- 3: sample $x_0 \sim \mathcal{N}(0, I)$; set $v \leftarrow x_1 - x_0$
- 4: sample (r, t) : two i.i.d. $\mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$ draws, sorted so $r \leq t$; with prob. $1 - \rho_{\text{MF}}$ collapse $t \leftarrow r$
- 5: interpolate state: $z_r \leftarrow (1 - r) x_0 + r x_1$
- 6: **if** $r < t$: $\frac{d}{dr} u \leftarrow \text{jvp}(u_\theta(\cdot, \cdot, \cdot, c_t); (z_r, r, t); (v, 1, 0))$ $\triangleright$ forward-mode, no grad
- 7: $u_{\text{tgt}} \leftarrow v + (t - r) \frac{d}{dr} u$ **else:** $u_{\text{tgt}} \leftarrow v$
- 8: predict (single forward over the full batch): $\hat{u} \leftarrow u_\theta(z_r, r, t, c_t)$
- 9: per-sample error: $\Delta^2 \leftarrow \|\hat{u} - \text{sg}(u_{\text{tgt}})\|_{2, \beta}^2$; weight $w \leftarrow (\Delta^2 + \varepsilon_w)^{-p}$
- 10: pointwise: $\mathcal{L}_{\text{pw}} \leftarrow \text{mean}(\text{sg}(w) \Delta^2)$
- 11: spectral (collapsed subset only): $\hat{x}_1 \leftarrow z_r + (1 - r) \hat{u}$; $\mathcal{L}_{\text{spec}}$ per Section 3.7
- 12: $\mathcal{L} \leftarrow \mathcal{L}_{\text{pw}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}$
- 13: AdamW step on θ ; EMA update $\theta^- \leftarrow 0.999 \theta^- + 0.001 \theta$

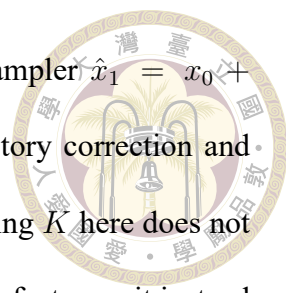
vector with log-spaced frequencies in $[1, 10^3]$:

$$u_\theta(z, r, t, c_t) = \text{SongUNet}(\text{concat}[z, c_t], r \cdot \tau, \text{FF}(t - r)). \quad (3.14)$$

This is the (r, Δ) variant of the two-time embeddings ablated by Geng et al. [5], who find time-plus-interval conditioning the most effective; the added parameters (one linear map from the 32 Fourier features) are negligible against the 78.7 M backbone. At $\Delta = 0$ the parameterisation reduces to FlowCast’s up to a constant augmentation vector, which is what makes the per-parameter comparison between the two students clean.

Sampling. Generation replaces numerical integration with Eq. (3.10) applied over K equal segments of $[0, 1]$ (`meanflow_model_forward` in `stormcast/utils/nn.py`):

$$z_{(i+1)/K} = z_{i/K} + \frac{1}{K} u_\theta\left(z_{i/K}, \frac{i}{K}, \frac{i+1}{K}, c_t\right), \quad i = 0, \dots, K - 1, \quad z_0 \sim \mathcal{N}(0, I), \quad (3.15)$$



at exactly one NFE per segment. $K = 1$ is the pure one-step sampler $\hat{x}_1 = x_0 + u_\theta(x_0, 0, 1, c_t)$; $K = 2$ trades one extra evaluation for a mid-trajectory correction and is the validation default. Unlike the FlowCast Euler sampler, increasing K here does not reduce a truncation error — each segment is already exact under a perfect u — it instead shortens the intervals over which the learned u_θ must be accurate. The sampler finishes identically to FlowCast: $\hat{R}_t = \sigma_{\text{data}} x_{t=1}$ and $\hat{X}_t = M_t + \hat{R}_t$.

Optimisation and checkpoints. Everything FlowCast-side is kept: AdamW with $\beta_{\text{Adam}} = (0.9, 0.999)$, learning rate 5×10^{-4} , weight decay 10^{-4} , a 4,000-step linear warmup into cosine decay, global gradient clipping at 1.0, and a fixed-decay 0.999 EMA shadow whose weights are saved to `ema_state.pt` and used by the in-training validation sampler. The production run trains on 7 GPUs at a global batch of 112; its step-to-sample conversion places the Chapter 4 checkpoint (step 20,000, ≈ 2.24 M samples) just above the ≈ 2 M-sample anchor shared by the EDM and FlowCast checkpoints — within 12%, which is small against the snapshot noise quantified in Section 4.1. In the head-to-head harness the FlowCast and MeanFlow students are loaded from their online checkpoints in exactly the same way, so the A/B comparison is symmetric end to end.

Why average velocity for this residual. First, it attacks the only cost FlowCast leaves on the table. The straight-line path made each Euler step cheap to take; MeanFlow removes the need to take many of them, compressing the residual head to 1–2 NFE per forecast hour — the regime where large ensembles between radar volumes stop being a budget question. Second, it is the right shape for a controlled study: because the $t = r$ subset of the objective is FlowCast’s objective, MeanFlow is a strict generalisation trained under identical hyperparameters, and any skill difference in Chapter 4 is attributable to the

average-velocity bootstrap alone. Third, the principled alternative to it — consistency-style models — impose network-behaviour constraints with curriculum schedules that are notoriously unstable on heavy-tailed targets; MeanFlow’s target is a property of the underlying field, needs no curriculum, and trains with the same loss machinery (channel weights, spectral term) already validated on qpepre.

Implementation pointers. Entry point `stormcast/train_meanflow.py`; training loop `stormcast/utils/trainer_meanflow.py`; loss `stormcast/utils/meanflow_loss.py` (`MeanFlowLoss`); preconditioner `stormcast/utils/meanflow_precond.py` (`MeanFlowPrecond`); configs `stormcast/config/meanflow.yaml`, `stormcast/config/model/meanflow.yaml`, `stormcast/config/training/meanflow.yaml`; sampler `meanflow_model_forward` in `stormcast/utils/nn.py`. The numerical hyperparameters are tabulated in Appendix 5.4.

3.7 Loss Design for the Precipitation Channel

The four-channel RWRF target makes per-channel loss engineering cheap, and we use this budget to attack the hardest channel, qpepre, from two directions. Both terms below are shared by the EDM baseline, FlowCast, and MeanFlow where applicable; the two flow heads use the channel-weighted L2 form on their velocity regression plus the spectral term on the implied clean residual (for MeanFlow, on the $t = r$ subset of the batch; Section 3.6).

Channel-weighted L2. Let $\beta = (\beta_{t2m}, \beta_{u10}, \beta_{v10}, \beta_{qpepre})$ be a vector of non-negative per-channel weights. The weighted L2 norm used by the FlowCast velocity loss is

$$\|a\|_{2,\beta}^2 = \sum_{k=1}^4 \beta_k \|a_k\|^2, \quad (3.16)$$

where the sum runs over the four output channels and the norm inside each term is the ordinary spatial L2. The default configuration uses $\beta = (1, 1, 1, w_{pr})$, with the precipitation weight w_{pr} exposed as the last entry of `channel_weights` in `stormcast/config/model/flowcast.yaml`. The headline FlowCast run uses $w_{pr} = 2$; Section 4.8 sweeps $w_{pr} \in [1.0, 2.4]$ to locate the precipitation/wind trade-off. A robust Pseudo-Huber variant $\rho_{c_h,\beta}(a, b) = \sum_k \beta_k (\sqrt{\|a_k - b_k\|^2 + c_h^2} - c_h)$ with a small c_h is also available for bounded gradients on extreme tail events, but is not used in the headline run.

Radial log-PSD regulariser. Because RMSE-like losses reward smooth, low-amplitude precipitation outputs, we add a spectral-matching regulariser that penalises the L1 distance between the radial log-PSD of the predicted and target `qpepre` fields:

$$\mathcal{L}_{\text{spec}} = \frac{1}{|\mathcal{K}|} \sum_{\kappa \in \mathcal{K}} |\log P_{\text{pred}}(\kappa) - \log P_{\text{target}}(\kappa)|, \quad (3.17)$$

where $P(\kappa)$ is the radially averaged 2D power spectral density at wavenumber κ and $\mathcal{K}$ is the set of non-zero radial wavenumbers on the spatial grid. The regulariser is active only on channels listed in `spectral_channels` and weighted by `spectral_weight` = λ_{spec} (default 0.1). By default we apply it to `qpepre` only, on the flow's implied clean residual $\hat{x}_1$ rather than the raw velocity (Algorithm 1).

Monitoring for mode collapse. Even with the above terms, sparse heavy-tailed precipitation is an invitation to mode collapse: the model can minimise the weighted L2 by producing a slightly-wet constant field. During training we therefore log the wet-pixel fraction at thresholds $\{0.1, 1.0, 5.0\} \text{ mm h}^{-1}$ and the 95th and 99th percentiles of the output distribution, and compare them to the same quantities on the training batch. A large divergence is a collapse warning.

3.8 Training Infrastructure

All runs use PyTorch with `torchrun --standalone --nnodes=1 --nproc_per_node=N` for multi-GPU training. Configuration is managed with Hydra; experiment overrides are passed on the command line as key=value pairs. Checkpoints are written every M iterations (default $M = 1000$) under

```
StormCast_{method}/{experiment}/0/checkpoints_{method}/  
checkpoint_*.pt
```

together with CSV loss logs and, every V iterations, validation heatmap and power-spectrum images for the four output channels. Auto-resume restores both the online checkpoint and, for FlowCast and MeanFlow, the EMA shadow (`ema_state.pt`) by a `checkpoint_latest.pt` convention in `trainer_flowcast.py` / `trainer_meanflow.py`; the EDM baseline restores from `checkpoints_diffusion/`. A run can therefore be restarted with the same command line and will pick up from the last written checkpoint.



Chapter 4 Experiments and Results

This chapter presents the empirical evaluation of the MeanFlow residual of Chapter 3 against its two reference heads — the EDM diffusion baseline it sets out to accelerate and the FlowCast straight-line flow-matching baseline it generalises. Section 4.1 defines the experimental protocol and the matched-sample-budget convention. Section 4.2 defines the evaluation metrics. Section 4.3 reports the headline comparison on the 2022 validation year. Section 4.4 reports the inference-cost (wall-clock and NFE) ablation, where MeanFlow’s one-to-two-evaluation operating point is the central result. Section 4.5 drills into precipitation skill by threshold and neighbourhood size; Section 4.6 reports autoregressive rollout error growth. Sections 4.7 and 4.8 report the $\log_{10}$ -encoding and precipitation-weight ablations. Section 4.9 places the heads against a legacy old-StormCast baseline. Section 4.10 presents qualitative case studies, Section 4.11 catalogues failure modes, and Section 4.12 concludes with a discussion.

4.1 Experimental Protocol

Validation split. All reported metrics are computed on the cleaned 2022 full-year RWRf validation store (8,759 hourly samples; Section 3.2). This contrasts with the stale `valid_dates` shipped in `stormcast/config/inference/stormcast.yaml`, which we override ex-

plicitly in every run.

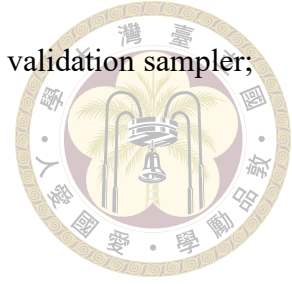


Matched training-sample budget. The fair-comparison unit is training samples seen, not optimiser steps, because the residual heads use different global batch sizes. The EDM baseline trains at batch 64, FlowCast at batch 96, and MeanFlow at batch 112, so ≈ 2 M samples corresponds to step 31,250 for EDM and step 20,833 for FlowCast; we evaluate each head at the nearest saved checkpoint to this anchor (EDM EDMPrecond.0.31000, FlowCast FlowCastPrecond.0.20000, MeanFlow MeanFlowPrecond.0.20000). The MeanFlow checkpoint lands at ≈ 2.24 M samples, 12 % above the anchor — small against the 10–30 % snapshot noise below, but noted for completeness. Holding training samples fixed means any quality difference reflects the generative trajectory and not a training-compute advantage.

Single-snapshot noise. Each validation point is a single snapshot. Adjacent validation logs drift by 10–30 %, so differences below ≈ 5 % on a single metric should not be over-interpreted; we call out only gaps that are large relative to this noise floor.

NFE budgets and samplers. The EDM baseline samples with its native 18-step Heun schedule (two model evaluations per step except the last, $2N - 1 = 35$ NFE per hourly step). FlowCast samples with K explicit Euler steps, K NFE; we use $K = 10$ in the headline scoreboard, the point at which the straight-line ODE is essentially converged (Section 4.4), and report the $K \in \{10, 15, 20\}$ cost ladder in the timing ablation. MeanFlow samples with K average-velocity segments (Eq. (3.15)), also exactly K NFE; its headline operating points are $K \in \{1, 2\}$. The scoreboard harness loads the two flow students from their online checkpoints in the same way, so the FlowCast-versus-MeanFlow comparison

is symmetric end to end (the EMA shadow is used by the in-training validation sampler; Section 3.5).



Rollouts and ensembling. The scoreboard metrics are computed from autoregressive rollouts driven by `compare_diffusion_vs_flowcast.py`, which denormalises every output (undoing standardisation and the $\log_{1p}$ encoding) so that `qpepre` is scored in mm h^{-1} regardless of the on-disk encoding. Following the StormCast ensemble design [15], CRPS uses a **five-member** per-hour ensemble of independent noise seeds propagated autoregressively. Deterministic and categorical scores use the ensemble mean. Ten initial times are spaced evenly across the validation year, each rolled out autoregressively over a **six-hour** horizon (+1 h . . . + 6 h), matching StormCast’s headline 1–6 h skill window.

Hardware. Training: multi-GPU single node. Evaluation: single GPU. The “Time/Seq.” column in the scoreboards is the wall-clock cost of generating one five-member six-hour rollout sequence on the evaluation GPU; the dedicated inference-timing ablation of Section 4.4 reports the operationally relevant cost at a 50-member, 24-hour configuration on a single NVIDIA RTX A6000.

4.2 Evaluation Metrics

All metrics are computed on denormalised fields with physical units (K, m s^{-1} , mm h^{-1}).

RMSE and MAE. For channel k ,

$$\text{RMSE}_k = \sqrt{\frac{1}{|\mathcal{T}|HW} \sum_{t \in \mathcal{T}} \sum_{i,j} (\hat{X}_{t,i,j}^{(k)} - X_{t,i,j}^{(k)})^2}, \quad (4.1)$$



with MAE defined analogously.

CRPS. The continuous ranked probability score evaluates the full predictive distribution of the small per-hour ensemble against the observation; lower is better. It rewards a forecast that is both sharp and calibrated, and is our primary probabilistic score.

Categorical precipitation scores. For threshold τ (mm h^{-1}), classify each pixel as wet ($\geq \tau$) or dry. With hits H , misses M , false alarms F :

$$\text{CSI}(\tau) = \frac{H}{H + M + F}, \quad \text{FAR}(\tau) = \frac{F}{H + F}, \quad \text{HSS}(\tau) = \frac{2(H \text{CN} - M F)}{(H + M)(M + \text{CN}) + (H + F)(F + \text{CN})}, \quad (4.2)$$

where CN is the correct-negative count. We report $\tau \in \{0.1, 1.0, 5.0, 10.0, 16.0\} \text{ mm h}^{-1}$; “-M” suffixes denote the mean over thresholds and “-P16” the $\tau = 16 \text{ mm h}^{-1}$ heavy-rain point. CSI and HSS reward correct detection (higher is better); FAR penalises over-prediction (lower is better). To relax the single-pixel double-penalty, CSI/POD/FAR/HSS are scored with neighbourhood-max pooling: an event counts as forecast if any pixel within an $n \times n$ window exceeds τ . We evaluate at the four StormCast pooling windows [15] on the $\approx 2 \text{ km}$ RWRf grid — 3 km (grid scale, $1 \times 1 \text{ px}$), 15 km ($7 \times 7 \text{ px}$), 27 km ($13 \times 13 \text{ px}$) and 45 km ($23 \times 23 \text{ px}$) — and take the 15 km window as the headline.

Fractions Skill Score. With neighbourhood window n ,

$$\text{FSS}(\tau, n) = 1 - \frac{\sum_{i,j} (p_{i,j}^{\text{fc}} - p_{i,j}^{\text{ob}})^2}{\sum_{i,j} (p_{i,j}^{\text{fc}})^2 + \sum_{i,j} (p_{i,j}^{\text{ob}})^2}, \quad (4.3)$$



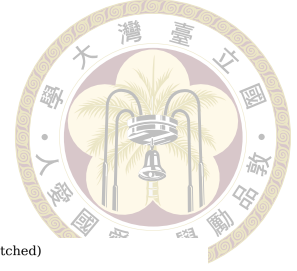
where $p_{i,j}$ is the fraction of τ -exceeding pixels in the $n \times n$ box centred at (i, j) [19]. FSS mitigates the double-penalty problem and is reported at the same four pooling windows; the heavy-rain point $\tau = 16 \text{ mm h}^{-1}$ is denoted FSS-P16.

Radial log-PSD. We compute the 2D FFT of each qpepre field, square its magnitude, radially average to obtain $P(\kappa)$, and compare $\log P(\kappa)$ of forecast and observation. A model that blurs precipitation has a log-PSD that decays too fast at high wavenumbers.

Rollout metrics. Per-channel RMSE is additionally reported as a function of lead time $\ell \in \{1, \dots, 6\} \text{ h}$ to expose autoregressive error growth.

4.3 MeanFlow versus Diffusion and FlowCast: Headline Comparison

Table 4.1 is the central result of the thesis: MeanFlow (at one and two average-velocity segments) set against its two reference heads — the EDM diffusion baseline and the FlowCast straight-line baseline at $K = 10$ Euler steps — on the 2022 validation year, all at the matched $\approx 2 \text{ M}$ -sample budget, sharing the same frozen regression mean and cleaned log1p dataset. Categorical scores are reported at the 15 km headline pooling window; Section 4.5 reports all four windows. Figure 4.1 visualises the same scoreboard.



Three-way scoreboard: EDM diffusion vs. FlowCast vs. MeanFlow (2022 validation; best per metric hatched)



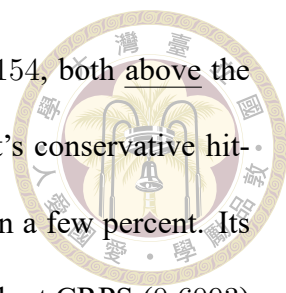
Figure 4.1: Per-metric comparison of the EDM diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$ on the 2022 validation year (five-member ensembles, 15 km categorical window). FlowCast attains its skill at roughly a quarter of the diffusion wall-clock cost, MeanFlow at roughly a tenth.

Table 4.1: Headline comparison on the 2022 validation year (cleaned 192×96 , $\log_{10}$ qpepre, ≈ 2 M training samples; five-member ensembles over ten initial times, +6 h roll-outs). Categorical scores (15 km neighbourhood) and precipitation RMSE are in mm h^{-1} after denormalisation. “-M” = mean over thresholds. $\uparrow/\downarrow$ indicate the better direction; **bold** marks the best in each column. t_{2m} RMSE is omitted for width and discussed in the text (Figure 4.2 (a)). Time/Seq. is the wall-clock cost of one five-member six-hour rollout sequence on a single NVIDIA RTX A6000.

Method	Time/Seq. (s)	CRPS $\downarrow$	CSI-M $\uparrow$	HSS-M $\uparrow$	FAR-M $\downarrow$	RMSE u10 $\downarrow$	RMSE v10 $\downarrow$	RMSE qpepre $\downarrow$
EDM diffusion (35 NFE)	11.23	0.5537	0.2622	0.2926	0.3417	1.3513	1.6082	1.0292
FlowCast ($K=10$)	3.07	0.5518	0.2504	0.2889	0.3504	1.2838	1.4085	0.9590
MeanFlow ($K=1$)	0.93	0.6710	0.2443	0.2861	0.2985	1.3767	1.6915	0.9834
MeanFlow ($K=2$)	1.16	0.6093	0.2690	0.3154	0.3738	1.2986	1.4887	1.0054

Read. At $K = 10$ FlowCast runs in 3.07 s per five-member sequence versus 11.23 s for the 35-NFE EDM Heun sampler — a **$3.7\times$ speedup** — and at that budget it matches or beats the diffusion baseline on most columns. CRPS is in fact marginally better (0.5518 vs 0.5537, a 0.3 % gap, i.e. statistically indistinguishable), and FlowCast is clearly ahead on u10 RMSE (-5%), v10 RMSE (-12%), and qpepre RMSE (-7%). The diffusion baseline retains an edge on the hit-based categorical scores at the headline window — CSI-M (0.2622 vs 0.2504) and HSS-M (0.2926 vs 0.2889) — and on t_{2m} RMSE (diffusion 0.8023 vs FlowCast 0.8498; the one channel where FlowCast trails, omitted from Table 4.1 for width but shown in Figure 4.2 (a)). The two FAR figures are within noise at 15 km (0.3504 vs 0.3417), but FlowCast is the lower-false-alarm head at the 3, 27 and 45 km windows (Section 4.5). Pushing FlowCast beyond $K = 10$ buys no headline skill, only wall-clock (Section 4.4), consistent with the straight-line ODE being essentially converged by ≈ 10 steps.

Read (MeanFlow rows). MeanFlow compresses the cost axis by another large factor: 1.16 s at $K = 2$ — $\approx 9.6\times$ cheaper than the diffusion sampler and $2.6\times$ cheaper than FlowCast’s $K = 10$ — and 0.93 s at $K = 1$. At $K = 2$ it posts the best categorical scores



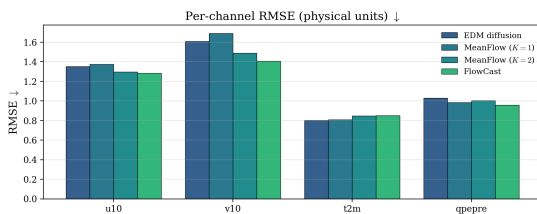
in the table at the headline window (CSI-M 0.2690 and HSS-M 0.3154, both above the diffusion baseline's 0.2622/0.2926), i.e. it does not inherit FlowCast's conservative hit-rate deficit, while matching FlowCast's u_{10} and v_{10} RMSE to within a few percent. Its concessions are probabilistic and on $qpepre$ RMSE: the six-hour rollout CRPS (0.6093) trails the ODE heads by $\approx 10\%$, FAR-M at $K = 2$ (0.3738) is the highest in the table (more hits bought with more false alarms), and $qpepre$ RMSE (1.0054) sits at the diffusion baseline's level rather than FlowCast's. The $K = 1$ row makes the mechanism legible: deterministic t_{2m} (0.8087, second only to diffusion) and the lowest FAR-M in the table (0.2985, a narrow ensemble raises few alarms) sit alongside the worst CRPS of the cleaned heads (0.6710) and the worst v_{10} RMSE (1.6915). The CRPS degradation from $K = 2$ to $K = 1$ ($0.6093 \rightarrow 0.6710$, $+10\%$) is the under-dispersion failure mode of Section 4.11: a single average-velocity evaluation maps each noise draw almost onto the conditional mean, narrowing ensemble spread, and the deficit compounds autoregressively. One extra segment restores most of the spread, which is why $K = 2$ is the recommended MeanFlow operating point.

Interpretation. The categorical picture is a coherent trade rather than a wash. FlowCast is the most conservative precipitation forecaster — lowest false-alarm ratio across the wider neighbourhoods but slightly fewer hits at the headline window — whereas the diffusion baseline and MeanFlow- $K2$ are more aggressive in placing wet pixels and are rewarded by CSI/HSS at some cost in FAR. Which trade is preferable is an operational choice; the unambiguous wins for FlowCast are the wind fields, precipitation RMSE, and a $\sim 3.7\times$ lower inference cost at statistically indistinguishable CRPS. MeanFlow occupies a third corner of the trade space: the best categorical hit rates of any head at the headline window and near-best temperature, at a tenth of the diffusion cost, paid for with

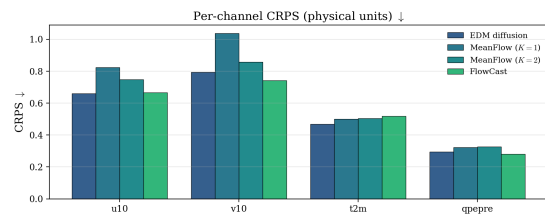
a single-snapshot-scale rollout-CRPS gap whose origin (narrowed ensemble spread under autoregression) is diagnosed in Section 4.11; restoring rollout spread at $K \leq 2$ is listed as future work in Chapter 5.



Per-channel RMSE and CRPS. Figures 4.2 (a) and 4.2 (b) break the deterministic and probabilistic errors out by channel. FlowCast’s advantage is largest and consistent across both metrics on the meridional wind v10 (RMSE 1.41 vs 1.61; CRPS 0.742 vs 0.793) and on the precipitation channel (qpepre CRPS 0.281, the best of any head). On u10 the diffusion baseline holds the lower CRPS (0.659 vs 0.667) despite FlowCast’s lower RMSE, and on t2m the diffusion baseline leads both metrics, FlowCast’s temperature residual being the noisiest at this budget. MeanFlow narrows the t2m gap: at $K = 1$ its temperature RMSE (0.8087 K) is the best of the flow heads and within 1 % of the diffusion baseline — evidence that FlowCast’s t2m weakness belongs to its learned trajectory rather than to the shared backbone, loss weights, or residual pipeline, since MeanFlow shares all three — while paying for its narrowed spread in the wind CRPS columns, where the $K = 1$ row is the weakest in the study.



(a) Per-channel RMSE (physical units).



(b) Per-channel CRPS.

Figure 4.2: Per-channel deterministic (RMSE) and probabilistic (CRPS) errors for the diffusion baseline, FlowCast, and MeanFlow on the 2022 validation year. FlowCast wins on the winds and precipitation; MeanFlow at $K=1$ holds the best temperature of the flow heads while its narrowed spread shows up in the wind CRPS.

4.4 Inference Cost: Wall-Clock and NFE Ablation



The defining property of a straight-line flow is that it integrates accurately in very few steps; the defining property of an average-velocity model is that it does not integrate at all. Tables 4.2 and 4.3 report the inference-timing ablation at an operationally realistic configuration — a 50-member, 24-hour ensemble forecast on a single NVIDIA RTX A6000 (the deployment GPU) and on a single NVIDIA H100 80GB — across the EDM Heun sampler, FlowCast at $K \in \{10, 15, 20\}$, and MeanFlow at $K \in \{1, 2\}$. This is the cost figure that matters for deployment: an ensemble convection-permitting cycle re-run every forecast hour. Figure 4.3 plots both devices’ wall-clock on a log axis with each head’s speedup over the EDM baseline annotated.

Table 4.2: Inference timing on a single NVIDIA RTX A6000 (50-member ensemble, 24 h autoregressive rollout, one CUDA warm-up forecast discarded). “s / forecast” is the wall-clock for one full 50-member 24 h ensemble forecast — the headline operational cost. “Speedup” is relative to the EDM diffusion sampler. “s / NFE” divides the per-forecast-hour cost by the per-step NFE; MeanFlow’s higher per-NFE figure is fixed per-forecast overhead (I/O, denormalisation) amortised over only one–two evaluations, not a heavier network. **Bold** = best per column.

Method	NFE/step	s / forecast↓	Speedup↑	s / fcst-hr↓	s / member↓	s / NFE
EDM diffusion	35	1898.0	1.0×	1.582	37.96	0.044
FlowCast ($K=10$)	10	645.2	2.9×	0.538	12.90	0.054
FlowCast ($K=15$)	15	826.4	2.3×	0.689	16.53	0.046
FlowCast ($K=20$)	20	1086.3	1.7×	0.905	21.73	0.045
MeanFlow ($K=1$)	1	191.9	9.9×	0.160	3.84	0.160
MeanFlow ($K=2$)	2	239.1	7.9×	0.199	4.78	0.100

Read. The ordering tracks the per-step NFE almost exactly, because wall-clock is dominated by network evaluations. FlowCast at $K = 10$ generates a full 50-member 24-hour ensemble in 645 s (≈ 10.8 min) against the diffusion sampler’s 1898 s (≈ 31.6 min), a $2.9\times$ speedup; the $K = 15$ and $K = 20$ rungs scale the cost linearly (826 s, 1086 s) for no headline-skill gain (Section 4.3), which is why $K = 10$ is the FlowCast operating

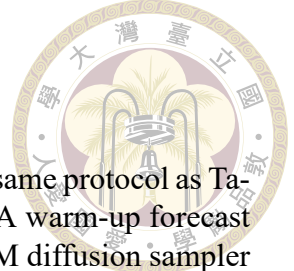


Table 4.3: Inference timing on a single NVIDIA H100 80GB HBM3 (same protocol as Table 4.2: 50-member ensemble, 24 h autoregressive rollout, one CUDA warm-up forecast discarded). Columns as in Table 4.2; “Speedup” is relative to the EDM diffusion sampler on the same device. **Bold** = best per column. The H100 roughly halves every wall-clock against the A6000 ($A6000/H100 \approx 1.8\text{--}2.1\times$; see Figure 4.3), and the relative speedups over EDM grow slightly because the diffusion baseline’s many evaluations dominate more on faster silicon.

Method	NFE/step	s / forecast↓	Speedup↑	s / fcst-hr↓	s / member↓	s / NFE
EDM diffusion	35	986.6	1.0×	0.822	19.73	0.023
FlowCast ($K=10$)	10	321.2	3.1×	0.268	6.42	0.027
FlowCast ($K=15$)	15	451.0	2.2×	0.376	9.02	0.025
FlowCast ($K=20$)	20	577.8	1.7×	0.482	11.56	0.024
MeanFlow ($K=1$)	1	91.1	10.8×	0.076	1.82	0.076
MeanFlow ($K=2$)	2	116.4	8.5×	0.097	2.33	0.048

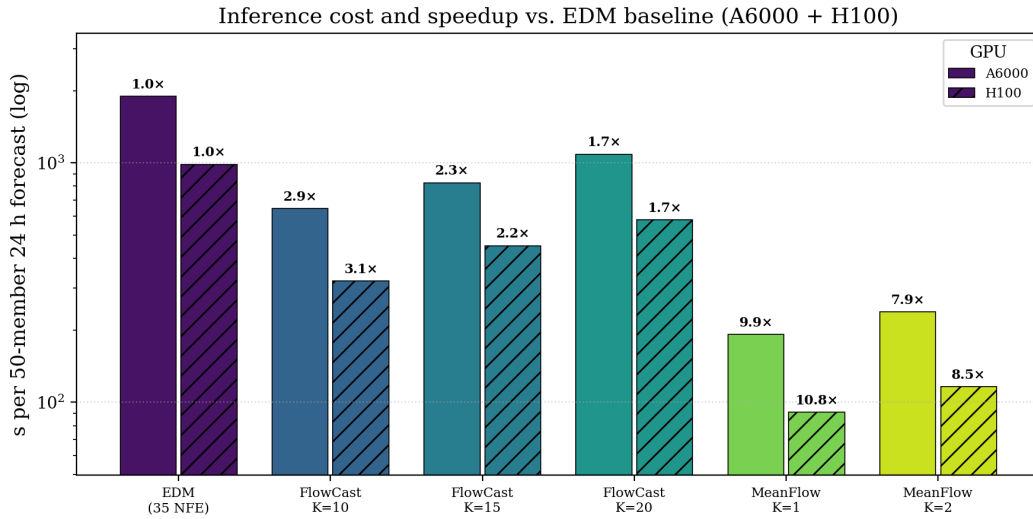
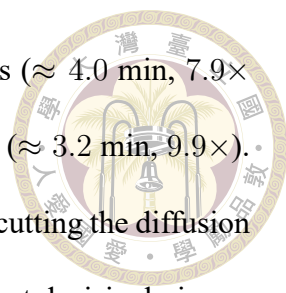


Figure 4.3: Wall-clock for one 50-member 24-hour ensemble forecast on a single NVIDIA RTX A6000 (solid) and a single NVIDIA H100 80GB (hatched), log axis, with each head’s speedup over the 35-NFE EDM diffusion baseline (on the same device) annotated above its bar. On the A6000 FlowCast- $K=10$ is $2.9\times$, MeanFlow- $K=2$ $7.9\times$, and MeanFlow- $K=1$ $9.9\times$ faster than EDM; on the H100 the corresponding factors are $3.1\times$, $8.5\times$, and $10.8\times$. The bars sample the viridis ramp from baseline (cool) to the average-velocity student (warm).



point. MeanFlow at $K = 2$ cuts the same ensemble forecast to 239 s (≈ 4.0 min, $7.9\times$ over diffusion and $2.7\times$ over FlowCast- $K10$), and at $K = 1$ to 192 s (≈ 3.2 min, $9.9\times$). Per network evaluation, the ODE heads cost ≈ 0.045 s– 0.054 s, undercutting the diffusion Heun sampler's ≈ 0.044 s only marginally on a per-evaluation basis but decisively in aggregate because they need so many fewer evaluations and each is a single forward pass with no second-order corrector. MeanFlow's larger nominal s/NFE (0.10 – 0.16 s) is an amortisation artefact: with only one or two evaluations, the fixed per-forecast overhead (data movement, denormalisation, metric I/O) is divided over a tiny NFE count rather than reflecting a more expensive forward pass — the network itself is the same SongUNet as FlowCast's.

Operational consequence. This is the cost that motivated the thesis (Section 1.4). A 50-member, 24-hour ensemble cycle issues 50×24 sequence-steps per forecast; at 35 NFE per step the diffusion sampler performs $50 \times 24 \times 35 = 42,000$ network evaluations per cycle, against 12,000 for FlowCast- $K10$ and just 2,400 for MeanFlow- $K2$ (1,200 at $K = 1$). Converting a configuration that the diffusion sampler places at half an hour of GPU time per cycle into one that completes in four minutes is what moves an ensemble convection-permitting forecast back inside an operational between-radar-volume budget. Crucially, these speedups are obtained without a second distillation training stage: they are intrinsic to the straight-line and average-velocity models.¹

Cross-hardware. The H100 leg (Table 4.3) confirms that the ranking is hardware-independent: the H100 roughly halves every wall-clock relative to the A6000 ($A6000/H100 = 1.8$ – $2.1\times$

¹The timing harness counts the EDM Heun sampler at 36 evaluations per step (two per Heun step, last corrector retained); the 35-NFE convention used elsewhere drops the final corrector. The one-evaluation difference does not affect any conclusion.

across all six configurations), but the relative speedups over the EDM baseline are preserved and even grow slightly — FlowCast- $K10$ goes from $2.9\times$ on the A6000 to $3.1\times$ on the H100, and MeanFlow- $K2$ from $7.9\times$ to $8.5\times$ ($K = 1: 9.9\times \rightarrow 10.8\times$). The mechanism is that the diffusion baseline’s 35 evaluations per step keep it compute-bound on faster silicon, while the one-to-two-evaluation MeanFlow head amortises a larger fixed-overhead fraction, so each hardware generation widens rather than narrows the gap. In absolute terms the H100 brings the MeanFlow- $K2$ cost to 116 s (≈ 1.9 min) and MeanFlow- $K1$ to 91 s (≈ 1.5 min) per 50-member 24-hour forecast, against the diffusion sampler’s 987 s (≈ 16.4 min).

4.5 Precipitation Skill by Threshold and Neighbourhood

Mean categorical scores hide both the threshold dependence and the neighbourhood dependence that matter operationally. Figure 4.4 plots CSI, FAR, and HSS against the rainfall threshold at the headline window, and Figure 4.5 the heavy-rain FSS across the four pooling windows.

Neighbourhood dependence. Every categorical score improves monotonically as the pooling window widens from 3 km to 45 km, because the grid-scale double penalty relaxes. At $\tau = 0.1 \text{ mm h}^{-1}$ the EDM baseline’s CSI rises from 0.533 (3 km) to 0.720 (45 km) and FlowCast’s from 0.546 to 0.738; the wet/dry discrimination the models actually have is a few-pixel-displaced one, which the grid-scale score under-credits. The 3 km (grid-scale) categorical numbers collapse to near-zero at the heavy thresholds for all heads — this is the single-pixel artefact the StormCast pooling windows are designed to relax, not a model statement. Table 4.4 reports the threshold-averaged categorical scores across all

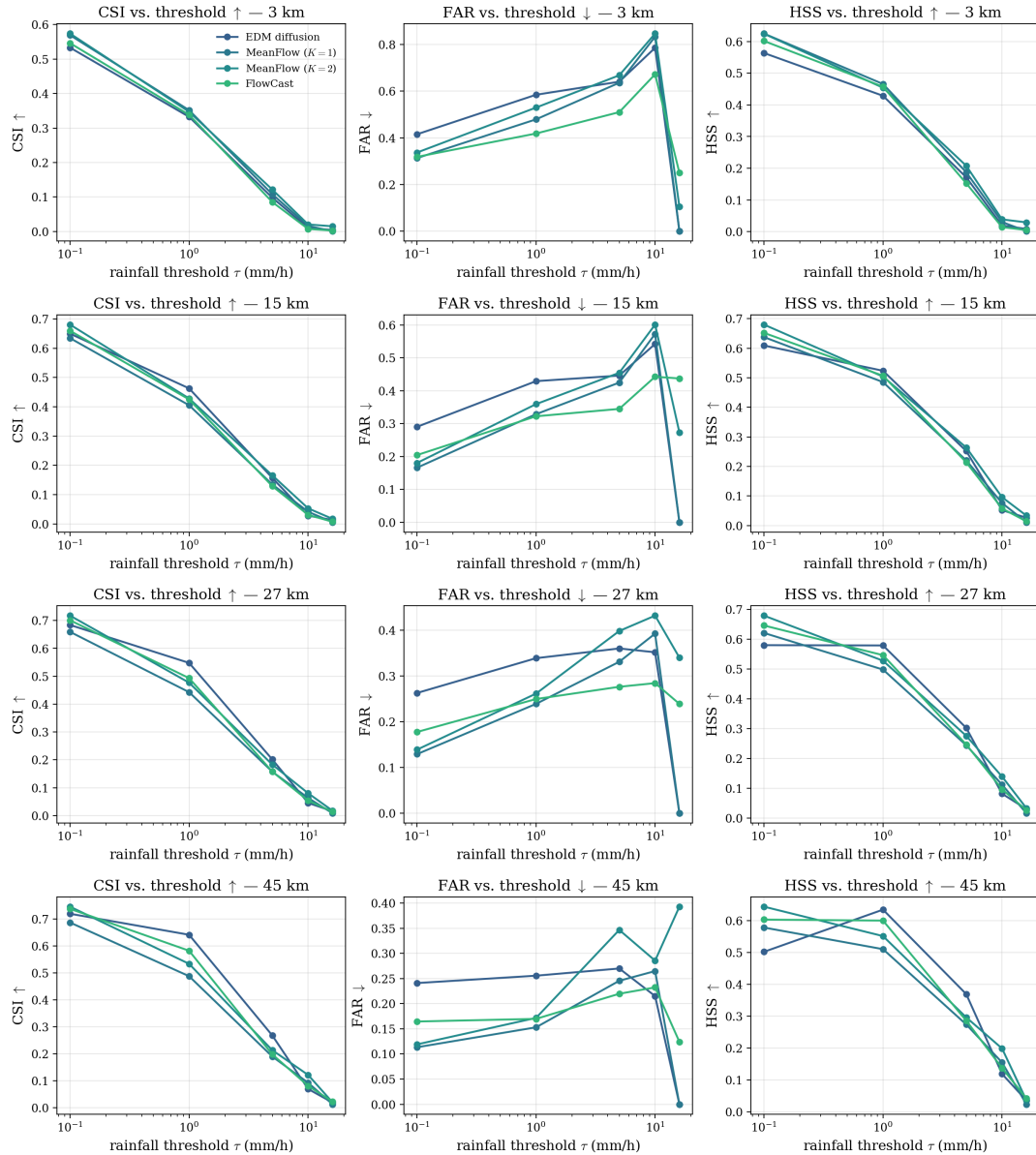


Figure 4.4: CSI, FAR, and HSS as a function of rainfall threshold for the diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$ (five-member ensemble means, 15 km neighbourhood). All heads collapse toward zero CSI above $\approx 5 \text{ mm h}^{-1}$ at this budget; the flow heads hold a false-alarm advantage at the light-to-moderate thresholds.

four windows for every head.

Table 4.4: Threshold-averaged categorical precipitation skill (mean over $\tau \in \{0.1, 1, 5, 10, 16\} \text{ mm h}^{-1}$) at the four StormCast neighbourhood windows, with the heavy-rain FSS point (FSS-P16, $\tau = 16 \text{ mm h}^{-1}$) alongside, for the legacy and cleaned heads (five-member ensemble means, 2022 validation year). Every score improves as the window widens; the 15 km row is the headline of Table 4.1. FSS-P16 is near-zero for all heads (ensemble-mean smoothing of $\geq 16 \text{ mm h}^{-1}$ extremes; Figure 4.5).

Method	Window	CSI-M $\uparrow$	HSS-M $\uparrow$	FAR-M $\downarrow$	FSS-P16 $\uparrow$
Legacy EDM (224 $\times$ 128)	3 km	0.1008	0.1256	0.5768	0.0198
	15 km	0.1448	0.1397	0.4110	0.0273
	27 km	0.1791	0.1472	0.3349	0.0256
	45 km	0.2312	0.1571	0.2570	0.0230
EDM diffusion	3 km	0.1956	0.2384	0.4853	0.0088
	15 km	0.2622	0.2926	0.3417	0.0076
	27 km	0.2989	0.3151	0.2628	0.0065
	45 km	0.3434	0.3320	0.1963	0.0053
FlowCast ($K=10$)	3 km	0.1960	0.2462	0.4340	0.0053
	15 km	0.2504	0.2889	0.3504	0.0038
	27 km	0.2839	0.3129	0.2454	0.0040
	45 km	0.3242	0.3337	0.1821	0.0043
MeanFlow ($K=1$)	3 km	0.2092	0.2621	0.4525	0.0018
	15 km	0.2443	0.2861	0.2985	0.0017
	27 km	0.2663	0.2987	0.2184	0.0014
	45 km	0.2938	0.3081	0.1553	0.0011
MeanFlow ($K=2$)	3 km	0.2156	0.2709	0.4974	0.0295
	15 km	0.2690	0.3154	0.3738	0.0264
	27 km	0.2950	0.3315	0.3142	0.0228
	45 km	0.3267	0.3452	0.2631	0.0184

Light to moderate rain. At $\tau = 0.1 \text{ mm h}^{-1}$ (15 km) the heads are close on CSI (diffusion 0.650, FlowCast 0.660, MeanFlow- $K=2$ 0.681), but FlowCast’s false-alarm ratio is markedly lower (0.204 vs the diffusion baseline’s 0.290). The same ordering holds at $\tau = 1.0 \text{ mm h}^{-1}$ (FAR: FlowCast 0.322 vs diffusion 0.429). This is the quantitative form of the “more conservative” behaviour: FlowCast does not paint speculative light rain across dry pixels, and MeanFlow- $K=1$ is the most conservative of all (FAR 0.166 at $\tau = 1.0$, 15 km).

Heavy rain. At $\tau = 5.0$ and 10.0 mm h^{-1} the heads converge to low CSI (≈ 0.13 – 0.17 at 5 mm h^{-1} , 15 km , with MeanFlow- $K2$ marginally ahead), reflecting how hard intense convective cells are to place exactly at the $\approx 2 \text{ M}$ -sample budget. Heavy-rain skill is the clearest target for the longer-training and tail-loss experiments listed in the project to-do. At the 16 mm h^{-1} point the comparison degenerates: with five-member ensemble means, almost no pixel of any head exceeds 16 mm h^{-1} , so FSS-P16 is at or below 0.03 for all heads at every neighbourhood size (Figure 4.5; MeanFlow- $K2$ is the relative best of the cleaned heads at $\text{FSS-P16} \leq 0.030$, still negligible). This is an ensemble-mean artefact — averaging members suppresses extremes — rather than a statement about individual members, and it is precisely why per-member (probabilistic) heavy-rain scoring over larger ensembles is listed as future work in Chapter 5.

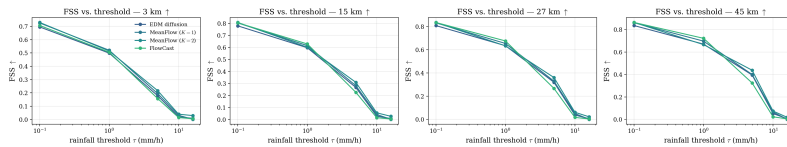


Figure 4.5: FSS at $\tau = 16 \text{ mm h}^{-1}$ for the four pooling windows (3, 15, 27, 45 km), computed on five-member ensemble means, for all heads. The score is near-zero across the board: averaging members suppresses $\geq 16 \text{ mm h}^{-1}$ extremes almost entirely, so this threshold mainly diagnoses ensemble-mean smoothing rather than model skill. Heavy-rain assessment at the member level requires probabilistic scoring (future work).

4.6 Autoregressive Rollout

Single-step skill understates operational risk because errors compound under autoregression. Figure 4.6 plots per-channel RMSE against lead time out to six hours, and Table 4.5 tabulates the same numbers at every lead.

Read. All heads show the expected monotone error growth. For v10 FlowCast stays well below the diffusion baseline at every lead time (1.68 m s^{-1} vs 2.23 m s^{-1} at $+6 \text{ h}$),



Table 4.5: Per-channel RMSE (physical units: m s^{-1} for winds, K for $t2m$, mm h^{-1} for $qppepre$) as a function of autoregressive lead time, $+1 \text{ h} \dots +6 \text{ h}$, on the 2022 validation year (five-member ensemble means over ten initial times). **Bold** marks the best value per lead and channel. [†]Legacy $qppepre$ is scored on the legacy 224×128 grid (raw mm h^{-1} encoding) and is not directly comparable to the cleaned 192×96 heads. [‡]Corrupt sample, excluded from the best-value bolding.

Lead (h)	Method	u10↓	v10↓	t2m↓	qppepre↓
1	Legacy EDM (224×128)	1.1364	1.1229	0.4899	1.0441 [†]
	EDM diffusion	1.0035	1.0766	0.4259	1.1022
	FlowCast ($K=10$)	1.0094	1.0895	0.4708	1.0838
	MeanFlow ($K=1$)	0.9578	1.0892	0.4433	1.0612
	MeanFlow ($K=2$)	0.9577	1.0343	0.4258	1.0472
2	Legacy EDM (224×128)	1.5537	1.4332	0.6811	1.1622 [†]
	EDM diffusion	1.2816	1.3563	0.5876	1.2361
	FlowCast ($K=10$)	1.2573	1.2927	0.6250	1.2254
	MeanFlow ($K=1$)	1.2558	1.4062	0.5927	1.2640
	MeanFlow ($K=2$)	1.2229	1.2896	0.5761	1.2774
3	Legacy EDM (224×128)	1.7302	1.6452	0.8474	1.1179 [†]
	EDM diffusion	1.4005	1.6570	0.7477	1.2397
	FlowCast ($K=10$)	1.3141	1.4876	0.7839	1.2055
	MeanFlow ($K=1$)	1.3860	1.7085	0.7465	1.2415
	MeanFlow ($K=2$)	1.2954	1.5578	0.7565	1.2677
4	Legacy EDM (224×128)	1.8454	1.7200	0.9645	10.7571 [‡]
	EDM diffusion	1.4003	1.7498	0.8679	1.3480
	FlowCast ($K=10$)	1.3180	1.5556	0.8998	1.2718
	MeanFlow ($K=1$)	1.4564	1.8145	0.8572	1.3385
	MeanFlow ($K=2$)	1.3245	1.6101	0.8781	1.3660
5	Legacy EDM (224×128)	2.0156	1.8685	1.0533	1.1137 [†]
	EDM diffusion	1.5313	1.9133	0.9898	1.3238
	FlowCast ($K=10$)	1.3689	1.5426	1.0542	1.2434
	MeanFlow ($K=1$)	1.6049	1.9838	0.9440	1.2937
	MeanFlow ($K=2$)	1.4759	1.7131	1.0065	1.3334
6	Legacy EDM (224×128)	2.1399	2.1199	1.6099	1.1005 [†]
	EDM diffusion	1.6711	2.2335	1.6008	1.2994
	FlowCast ($K=10$)	1.5574	1.6805	1.7272	1.2189
	MeanFlow ($K=1$)	1.7741	2.3286	1.6157	1.2370
	MeanFlow ($K=2$)	1.6502	2.0141	1.7727	1.2749

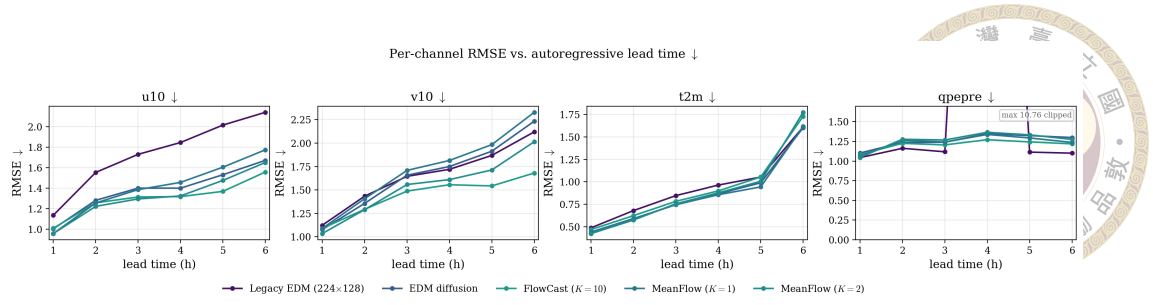


Figure 4.6: Per-channel RMSE as a function of autoregressive lead time (1–6 h) for the legacy baseline, the EDM diffusion baseline, FlowCast at $K = 10$, and MeanFlow at $K \in \{1, 2\}$. FlowCast tracks or improves on the diffusion baseline for the winds and precipitation across the horizon; MeanFlow at $K=1$ holds the lowest t2m error of the flow heads, and neither flow head destabilises under rollout.

with MeanFlow- $K2$ between the two (2.01 m s^{-1}); for u10 the heads are close, FlowCast slightly ahead at the longest lead (+6 h: 1.56 m s^{-1} vs the diffusion baseline’s 1.67 m s^{-1}). The qpepre rollout RMSE is comparable between the cleaned heads and does not diverge ($\approx 1.2\text{--}1.3 \text{ mm h}^{-1}$ throughout) — the conservative flow precipitation does not destabilise under rollout. The one adverse signal is t2m: FlowCast’s temperature error grows fastest, reaching 1.73 K at +6 h against 1.60 K for the diffusion baseline, consistent with its weaker single-step t2m number; MeanFlow at $K = 1$ holds t2m closest to the diffusion baseline (1.62 K) while $K = 2$ drifts to FlowCast’s level (1.77 K), so the t2m rollout ordering follows the single-step ordering of Section 4.3 exactly. The legacy baseline shows an isolated qpepre blow-up at +4 h ($\approx 10.8 \text{ mm h}^{-1}$), the kind of unphysical excursion none of the cleaned flow heads exhibits — which we attribute to the straight-line ODE’s stability.

4.7 Ablation: $\log_{1p}$ Precipitation Encoding

The single most consequential preprocessing decision is whether to store qpepre as raw mm h^{-1} or as $\log(1 + x)$ before standardisation. Table 4.6 reports the within-architecture A/B test (single-step standardised validation RMSE at $\approx 2 \text{ M}$ samples) for

the EDM and FlowCast heads. MeanFlow is absent from this table by necessity: only a log1p MeanFlow was trained, so its raw-mm/h cell does not exist. Because the mechanism identified below is head-agnostic — the badly-scaled raw channel contaminates the shared U-Net trunk — and MeanFlow shares that trunk, loss weighting, and the FlowCast objective on 75 % of its batch, we adopt log1p for MeanFlow without a separate ablation.

Table 4.6: Single-step standardised validation RMSE at ≈ 2 M samples, log1p versus raw-mm/h qpepre encoding. Values are dimensionless standard scores. **Bold** = better within each architecture pair on the non-qpepre channels. The qpepre column is not directly comparable across the encoding boundary (different units in standardised space); see the note.

Run	RMSE t2m	RMSE u10	RMSE v10	RMSE qpepre [†]
diffusion, log1p	0.0994	0.3523	0.2762	0.7865
diffusion, raw mm/h	0.1245	0.3955	0.3115	0.4156
FlowCast, log1p	0.0835	0.2931	0.2463	0.3851
FlowCast, raw mm/h	0.1118	0.3079	0.2331	0.3471

Read. The log1p encoding improves both heads on t2m and u10 (~ 20 – 25 % on t2m, ~ 5 – 11 % on u10) and improves the diffusion baseline’s v10 as well; the lone exception is FlowCast’s v10, where the two encodings differ by less than the 10–30 % snapshot noise and so are effectively tied. The mechanism is that the badly-scaled raw qpepre channel (standard deviation 9.12 mm h^{-1}) propagates through the shared U-Net trunk and degrades the other channels; compressing it to a standard deviation of ≈ 0.37 in log space removes that pressure. The net benefit is similar for both architectures, so the encoding choice is essentially independent of the head, and we adopt log1p as canonical throughout. The qpepre column ([†]) cannot be compared directly across the encoding boundary: in standardised space the two encodings carry different units, and inverting log1p on a scalar RMSE is invalid because $\exp m1$ does not commute with the root-mean-square. The mm h^{-1} -correct comparison is the denormalised scoreboard of Section 4.3, where `compare_diffusion_vs_flowcast.py` inverts the encoding per pixel before scoring.

4.8 Ablation: Precipitation Channel Weight



The precipitation weight w_{pr} (the last entry of `channel_weights`) trades precipitation skill against wind/temperature skill. Table 4.7 reports the full eight-point FlowCast sweep $w_{\text{pr}} \in [1.0, 2.4]$ (single-step standardised validation RMSE, $\log 1p, \approx 2$ M samples). Because `channel_weights` is applied only inside the training loss and not at evaluation, these RMSEs are directly comparable across rows without rescaling by w_{pr} . The sweep is FlowCast-only: w_{pr} is a training-time parameter, so a MeanFlow sweep would require seven additional training runs. MeanFlow instead adopts the sweep’s winning all-round setting $w_{\text{pr}} = 2.0$ unchanged, which keeps the MeanFlow-versus-FlowCast headline comparison an A/B on the objective alone.

Table 4.7: FlowCast precipitation-weight sweep, single-step standardised validation RMSE ($\log 1p, \approx 2$ M samples). **Bold** = column minimum. $w_{\text{pr}} = 2.0$ is the headline run.

w_{pr}	RMSE t2m	RMSE u10	RMSE v10	RMSE qpepre
1.0	0.0994	0.3188	0.2326	0.5464
1.2	0.1032	0.2912	0.2110	0.3771
1.4	0.0890	0.2733	0.2050	0.3996
1.6	0.0930	0.3282	0.2391	0.6796
1.8	0.0983	0.2838	0.2075	0.4831
2.0 (headline)	0.0835	0.2931	0.2463	0.3851
2.2	0.0990	0.3178	0.2499	0.5178
2.4	0.0991	0.3075	0.2355	0.5515

Read. There is no clean monotone trend: both $w_{\text{pr}} = 1.4$ and $w_{\text{pr}} = 2.0$ beat their neighbours, while $w_{\text{pr}} = 1.6$ and $w_{\text{pr}} \geq 2.2$ are poor across the board. $w_{\text{pr}} = 1.4$ is the per-channel sweet spot for the wind fields (lowest u10 and v10 RMSE in the sweep), whereas $w_{\text{pr}} = 2.0$ is best on t2m; on qpepre the minimum is a near-tie among $w_{\text{pr}} \in \{1.2, 1.4, 2.0\}$ that sits inside the 10–30 % single-snapshot noise, so we do not read fine ordering between those three. The robust conclusions are (i) some up-weighting of qpepre helps but the

floor-to-ceiling spread is large and real (e.g. $w_{\text{pr}} = 1.4$ at 0.3996 vs $w_{\text{pr}} = 1.6$ at 0.6796 is a $\sim 70\%$ jump on qpepre RMSE); (ii) $w_{\text{pr}} \approx 2.0$ is a reasonable all-round default that wins t2m and is competitive on qpepre, which is why the canonical run adopts it; and (iii) the wind/precipitation trade-off is genuine and would justify dropping to $w_{\text{pr}} = 1.4$ if the winds were the priority.

4.9 Comparison Against the Legacy Baseline

To place the cleaned-pipeline heads in context, Table 4.8 adds the legacy old-StormCast EDM baseline (the original 224×128 raw-qpepre model on the legacy dataset). Because the legacy grid and encoding differ, this is a pipeline-level rather than a controlled head-to-head comparison, and the absolute magnitudes are not directly comparable to the cleaned rows (the field of view differs by the centre crop).

Table 4.8: Legacy old-StormCast EDM baseline versus the cleaned-pipeline diffusion, FlowCast, and MeanFlow heads (categorical scores at the 15 km window, mm h^{-1}). The legacy leg uses a different grid/encoding, so absolute values are comparable only in magnitude (see Section 3.2).

Method	Time/Seq. (s)	CRPS↓	CSI-M↑	FAR-M↓	RMSE qpepre↓
Legacy EDM (raw, 224×128)	19.66	0.6348	0.1448	0.4110	1.4827
Cleaned EDM (35 NFE)	11.23	0.5537	0.2622	0.3417	1.0292
Cleaned FlowCast ($K=10$)	3.07	0.5518	0.2504	0.3504	0.9590
Cleaned MeanFlow ($K=2$)	1.16	0.6093	0.2690	0.3738	1.0054

Read. The cleaning + $\log 1p$ + retraining pipeline is a large step up over the legacy baseline on every metric (CRPS $0.63 \rightarrow 0.55$, CSI-M $0.14 \rightarrow 0.26$ at the headline window, qpepre RMSE $1.48 \rightarrow 0.96\text{--}1.03$), independent of the generative head. On top of that, FlowCast adds the inference-cost and precipitation-RMSE wins at near-parity-or-better on CRPS, and MeanFlow compresses the cost axis to 1.16 s per evaluation sequence —

a $17\times$ end-to-end speedup over the legacy system and $\approx 9.6\times$ over the cleaned diffusion baseline — while posting the best categorical hit rate of any head; its rollout CRPS (0.6093) gives back part of FlowCast’s parity but still improves on the legacy level. The legacy-to-cleaned and cleaned-EDM-to-flow gaps are therefore largely orthogonal: most of the headline improvement is the pipeline, and the flow heads’ specific contribution is to deliver comparable-or-better skill at a fraction of the cost without a teacher — FlowCast at CRPS parity, MeanFlow at the lowest cost.

4.10 Qualitative Case Studies

To complement the aggregate metrics we examine the single most precipitation-active initial time of the 2022 validation year — $t_0 = 2022-10-15\ 21:00\ \text{UTC}$, a heavy convective episode with a peak RWRF rain rate of $\approx 80\ \text{mm h}^{-1}$ and $\approx 48\%$ of the domain wet at $\geq 1\ \text{mm h}^{-1}$ — and roll every head forward autoregressively over the same $+1\ \text{h} \dots +6\ \text{h}$ horizon used for the rollout metrics. Figure 4.7 stacks the precipitation fields (RWRF truth, legacy EDM, cleaned EDM diffusion, FlowCast, MeanFlow), and Figure 4.8 the zonal wind, all denormalised to physical units; the model rows load the matched $\approx 2\ \text{M}$ -sample checkpoints used throughout the chapter.

Precipitation. The case renders the categorical trade-offs of Section 4.5 visible. The cleaned EDM diffusion head paints the broadest, most diffuse precipitation — widespread moderate rain that bleeds into nominally dry pixels, the spatial signature of its higher false-alarm ratio at the fine windows. FlowCast is the most conservative head: it keeps the dry regions dry but visibly damps the most intense convective cores, and the heavy-rain signal washes out as the rollout advances — the qualitative form of its low FAR bought at the cost

of heavy-rain hits. MeanFlow at two evaluations occupies the middle ground the metrics predict: it retains markedly more high-intensity structure than FlowCast, re-forming bright $\geq 20 \text{ mm h}^{-1}$ cells late in the rollout (consistent with its best CSI-M/HSS-M hit rates), while still suppressing the diffusion baseline's speculative light rain. Crucially MeanFlow's two-evaluation fields are not visibly smoother than FlowCast's ten-step fields — the average-velocity sampler does not pay the blurring penalty an aggressively truncated ODE integration would — and none of the cleaned flow heads destabilises over the six hours (no unphysical excursions), echoing the bounded rollout RMSE of Table 4.5. The legacy EDM panel sits on the wider 224×128 grid (a different field of view) and is visibly grainier.

Wind. On u_{10} (Figure 4.8) all heads track the broad-scale flow closely — the wind channels are far easier than precipitation — with the cleaned flow heads' fields lying closest to the RWRF target, consistent with their lower wind RMSE (Section 4.3) and the absence of wind-field drift under rollout.

Training dynamics. Figure 4.9 shows the FlowCast training and validation loss curves over the canonical run. The spectral-matching regulariser on q_{pepre} (the radial log-PSD term of Section 3.5) keeps the predicted precipitation spectrum from rolling off too early at high wavenumbers, which is the spectral signature of a sharp (non-blurry) generative forecast; both the training and validation curves descend smoothly with no divergence, and the EMA-shadow validation loss tracks below the online loss as expected.



6 h hourly autoregressive rollout — qpepre ($t_0 = 2022-10-15\ 21:00\ \text{UTC}$)

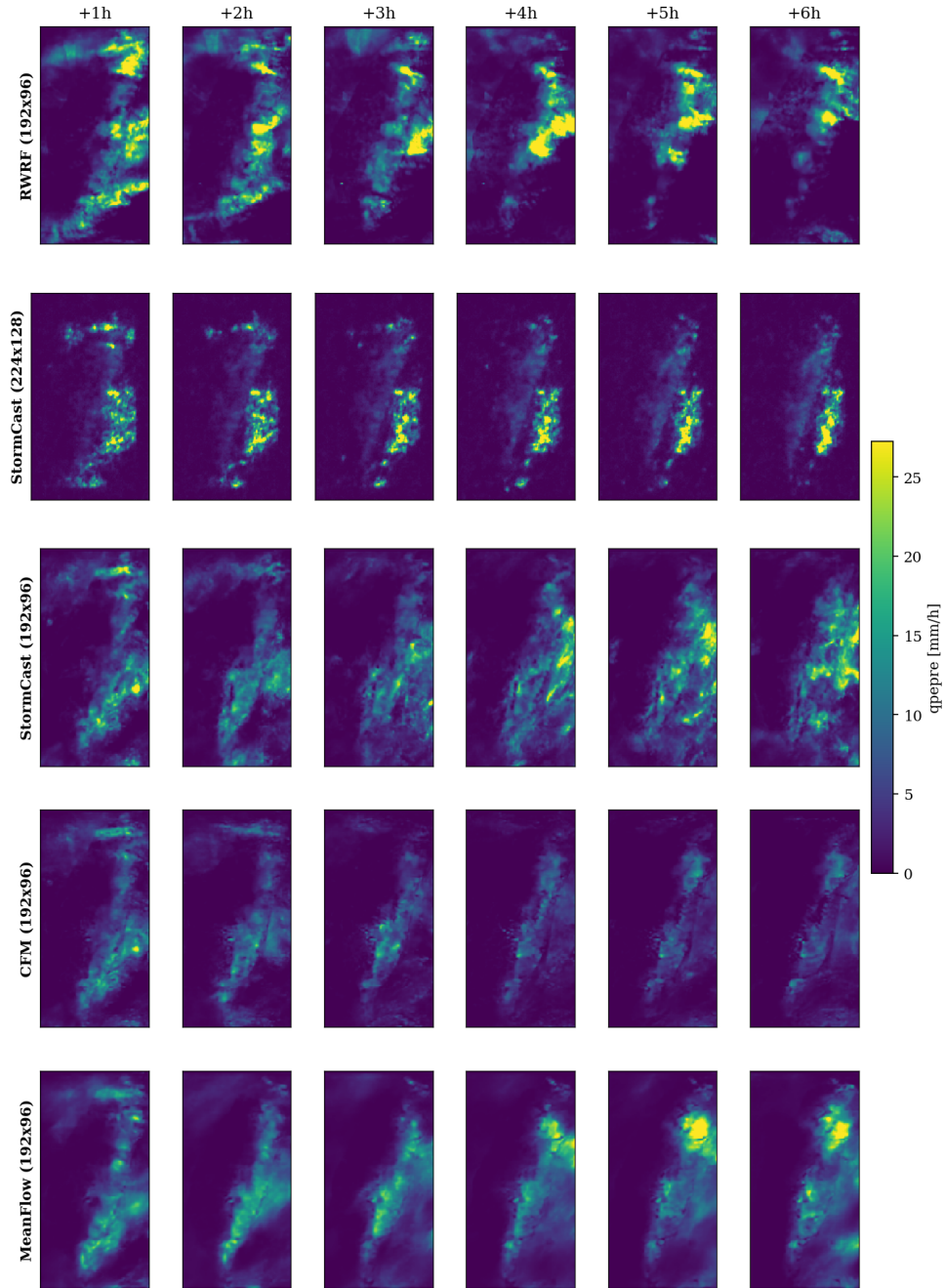


Figure 4.7: Six-hour autoregressive rollout of $qpepre$ (mm h^{-1}) for the heavy-rain initial time $t_0 = 2022-10-15\ 21:00\ \text{UTC}$. Rows: RWRF/QPEPRE truth, legacy EDM (224×128), cleaned EDM diffusion (35 NFE), FlowCast ($K=10$), MeanFlow ($K=2$); columns: lead times $+1\ \text{h} \dots +6\ \text{h}$. The diffusion head spreads broad moderate rain, FlowCast damps the intense cores under rollout, and MeanFlow retains the most high-intensity structure of the flow heads at two network evaluations.



6 h hourly autoregressive rollout — u_{10} ($t_0 = 2022-10-15$ 21:00 UTC)

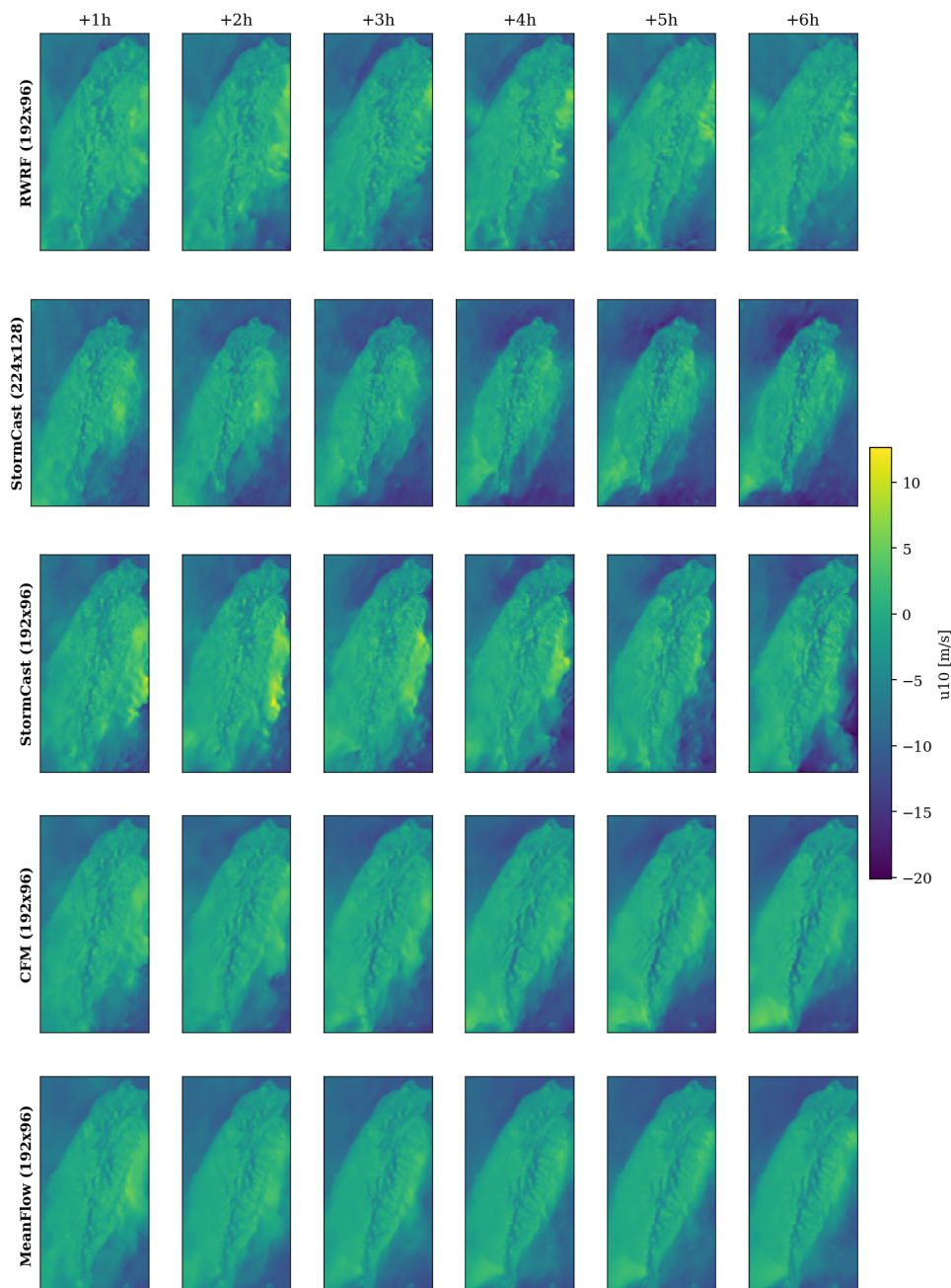


Figure 4.8: Six-hour autoregressive rollout of the 10 m zonal wind u_{10} (m s^{-1}) for the same initial time and row/column layout as Figure 4.7. All heads track the broad-scale wind field; the cleaned flow heads lie closest to the RWRF target with no rollout drift.

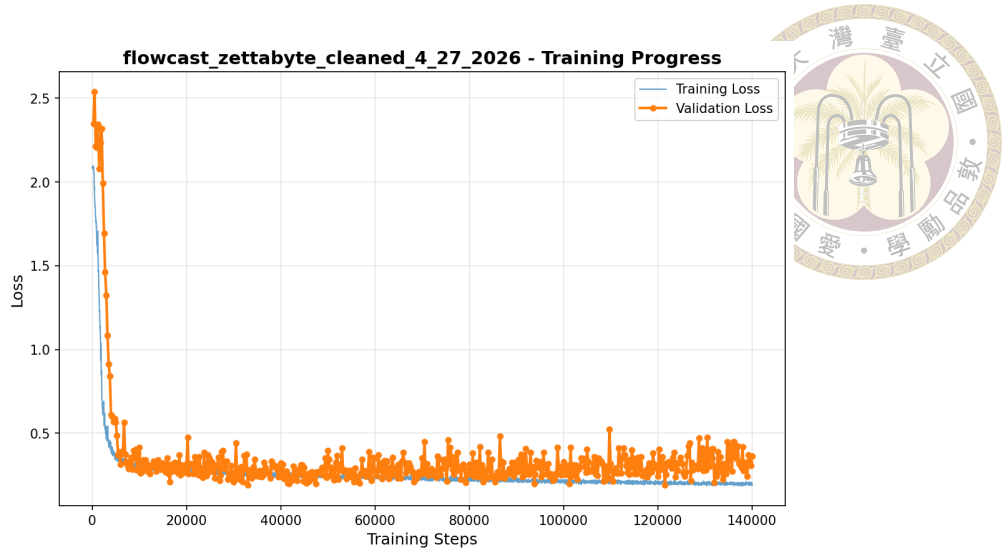


Figure 4.9: FlowCast training and validation loss over the canonical run. The smooth descent and the EMA-shadow validation curve confirm stable optimisation under the spectral-regularised CFM objective.

4.11 Failure Modes

We document four failure modes observed during development.

Heavy-rain under-detection. The flow heads’ weakest metric is pixel-exact CSI at the heavier thresholds ($5\text{--}10\text{ mm h}^{-1}$): their conservative, low-false-alarm behaviour costs hits on intense cells, and at the $\approx 2\text{ M}$ -sample budget all heads converge to low heavy-rain CSI. Diagnosis: the per-threshold CSI curve (Figure 4.4) flattening toward zero above 5 mm h^{-1} , partly recovered by the wider-neighbourhood FSS. Mitigation candidates (deferred to future work): longer training, a higher w_{pr} paired with the spectral term, or an explicit tail-quantile loss on qpepre.

t2m rollout drift. FlowCast’s temperature error grows fastest of the cleaned heads under rollout, reaching 1.73 K at $+6\text{ h}$ against the diffusion baseline’s 1.60 K (Figure 4.6). This is the autoregressive signature of its weaker single-step t2m number. Notably, MeanFlow

at $K = 1$ does not inherit this drift despite sharing FlowCast's backbone and loss weights — its τ_{2m} rollout error stays at the diffusion baseline's level ($1.62 K$) — which suggests the drift is a property of the learned trajectory rather than of the architecture. Mitigation for FlowCast: a per-channel weight that does not starve τ_{2m} , or a short teacher-forcing warmup.

One-evaluation under-dispersion (MeanFlow). At $K = 1$ MeanFlow's deterministic skill is intact (its τ_{2m} is the best of the flow heads and its FAR the lowest in the study) but its CRPS lags its own $K = 2$ setting: a single average-velocity evaluation maps each noise draw almost straight onto the conditional mean, narrowing ensemble spread and costing calibration. The effect compounds under autoregression because each hour's narrowed ensemble seeds the next, so the rollout CRPS gap is the visible symptom: 0.6710 at $K = 1$ versus 0.6093 at $K = 2$ (a 10 % deficit; Table 4.1), with the wind CRPS columns of Figure 4.2 (b) showing the same signature. One extra segment ($K = 2$) restores most of the spread at one additional NFE, which is why $K = 2$ is the recommended MeanFlow operating point and the validation default. Diagnosis: the CRPS columns of Table 4.1 and Figure 4.2; per-member rank histograms and rollout spread restoration are future work (Chapter 5).

EMA dependence. Loading the online checkpoint_*.pt training-state dump instead of the proper weights produces visibly noisier flow-head samples. This is not a model failure but an evaluation pitfall. The in-training validation samplers use the EMA shadow (ema_state.pt); the head-to-head harness of this chapter loads the online .mdlus checkpoints for FlowCast and MeanFlow symmetrically, so the A/B comparison between the two students is unaffected by the convention, and the EMA-versus-online gap itself shrinks

as training proceeds.

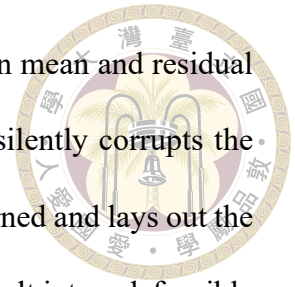


4.12 Discussion

The headline question of the thesis is whether a single-run, teacher-free average-velocity residual can replace the EDM diffusion residual in a regional convection-permitting forecaster without losing the features that matter for Taiwan precipitation — and do so cheaply enough to deploy. The evidence at the matched ≈ 2 M-sample budget is a qualified yes: at $K \in \{1, 2\}$ network evaluations — roughly a tenth of the diffusion sampler’s cost — MeanFlow reaches the diffusion baseline’s deterministic and categorical skill class, posts the best categorical hit rates of any head at the headline neighbourhood and near-best temperature, and runs a full 50-member 24-hour ensemble forecast in four minutes against the diffusion sampler’s half hour, conceding only a spread-driven rollout-CRPS gap (largest at $K = 1$, mostly closed by $K = 2$). The FlowCast straight-line baseline charts the intermediate point on the same trajectory axis: at $K = 10$ Euler steps it matches the diffusion baseline’s CRPS, beats it on wind and precipitation RMSE, and runs $\approx 3.7\times$ faster — confirming that the straight-line path already recovers most of the diffusion skill before the average-velocity jump removes the last of the integration cost.

Three caveats bound these conclusions. First, all numbers are at a single ≈ 2 M-sample snapshot with 10–30 % validation noise; the FlowCast canonical run continues to ≈ 13.4 M samples (step 140,000) and a final comparison at converged budgets is part of the remaining work (Chapter 5 and the project to-do). Second, both heads are anchored to the same frozen regression mean and the same cleaned log1p dataset, so the comparison is clean but specific to this Taiwan RWRF setup; transfer to other domains is untested. Third,

the precipitation-encoding gotcha of Section 4.7 means the regression mean and residual head must always share the $\log_1 p$ encoding — mixing encodings silently corrupts the residual distribution. The next chapter summarises what we have learned and lays out the concrete experiments and data still needed to turn this provisional result into a defensible thesis claim.



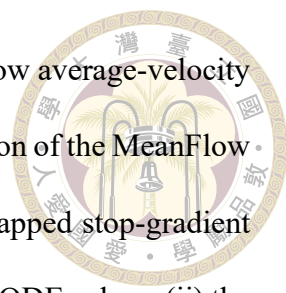




Chapter 5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis asked whether a single-run, teacher-free average-velocity residual can replace the EDM diffusion residual in a regional, convection-permitting StormCast forecaster over the Taiwan domain — delivering the speed a curved diffusion ODE cannot, without sacrificing the sharpness and probabilistic calibration that motivated generative forecasting in the first place — and how few network evaluations such a residual ultimately needs. Starting from the StormCast two-stage decomposition (a frozen deterministic regression mean plus a generative residual head) trained on the Taiwan RWRf dataset, we adapted **MeanFlow** — which learns the average velocity over a whole time interval and transports the state across it in one or two network evaluations, with no integration at all — as the generative residual head. As baselines we retained the EDM diffusion residual it sets out to accelerate and **FlowCast**, a Conditional-Flow-Matching head that learns the straight-line instantaneous velocity field and samples it with a handful of explicit Euler steps and that MeanFlow generalises exactly. All three were compared head-to-head on the full 2022 hourly validation year.



The concrete contributions are: (i) a single-training-run MeanFlow average-velocity residual pipeline for StormCast — to our knowledge the first application of the MeanFlow objective to a weather forecaster — trained with the identity-bootstrapped stop-gradient loss, sampling the residual in one or two network evaluations with no ODE solver; (ii) the supporting engineering and two reference heads (EDM diffusion and the FlowCast I-CFM baseline) anchored to exactly the same frozen regression mean, backbone, and σ_{data} , so the comparison isolates the generative trajectory and the MeanFlow-versus-FlowCast leg is an A/B on the training objective alone (MeanFlow’s loss reducing to FlowCast’s at zero interval length); (iii) a precipitation-aware loss — channel-weighted L2 plus a radial log-PSD regulariser on qpepre — together with log1p-encoding and precipitation-weight ablations; (iv) a precipitation-aware evaluation suite spanning deterministic (RMSE/MAE), probabilistic (CRPS), categorical (CSI/FSS/HSS/FAR), spectral, and rollout metrics; and (v) the resulting finding that, at a matched ≈ 2 M-sample budget, MeanFlow reaches the diffusion baseline’s deterministic and categorical skill class at $K \in \{1, 2\}$ evaluations — roughly a tenth of the diffusion sampler’s cost — posting the best categorical precipitation hit rates (CSI/HSS) of any head at the headline neighbourhood and the best temperature of the flow heads (within 1 % of the diffusion baseline), conceding only a spread-driven rollout-CRPS gap (≈ 10 % at $K = 2$, larger at $K = 1$); the FlowCast baseline meanwhile matches the diffusion CRPS, beats it on wind and precipitation RMSE and on false-alarm ratio at the wider neighbourhoods, and runs $\approx 3.7\times$ faster at 10 Euler steps, marking the intermediate point on the same trajectory axis.

5.2 Limitations

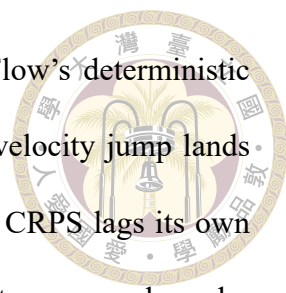
Several limitations should be kept in mind when interpreting these results.

Single training snapshot. The headline comparison is taken at a single ≈ 2 M-sample checkpoint per head, where single-snapshot validation noise is 10–30 % (MeanFlow’s checkpoint sits 12 % above the anchor, inside that noise; Section 4.1). The canonical FlowCast run continues to ≈ 13.4 M samples, and the diffusion baseline likewise has more training available; a converged-budget comparison is not yet in hand.

Single regression mean and single domain. All three heads are anchored to one frozen regression mean trained on one regional domain (Taiwan RWRf). We cannot assess transfer to other convection-permitting domains (HRRR/CONUS StormCast) without retraining, nor disentangle how much of the flow heads’ behaviour depends on the specific quality of this regression mean.

Heavy-rain detection. The flow heads’ conservative, low-false-alarm behaviour costs pixel-exact hits at the heavier rainfall thresholds. The neighbourhood FSS recovers part of this, suggesting a displacement rather than a pure miss, but the deterministic heavy-rain CSI is a genuine weakness at the current budget — and at the heaviest threshold, five-member ensemble-mean scoring degenerates for every head (averaging suppresses extremes), so the experiment cannot yet rank heads there at all.

Limited probabilistic evaluation. CRPS is computed from five-member per-hour ensembles — a small, operationally realistic ensemble in the StormCast tradition — but a large-ensemble calibration study (rank histograms, spread–skill at 50–100 members), which a cheap few-step sampler makes affordable, has not been run.



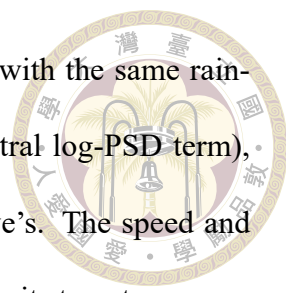
Under-dispersion at low NFE (MeanFlow). At $K = 1$ MeanFlow’s deterministic skill is intact but its ensemble spread narrows — a single average-velocity jump lands each noise draw nearly on the conditional mean — so its aggregate CRPS lags its own $K = 2$ setting, most visibly on the wind channels, and the deficit compounds under autoregression: over the six-hour rollout the $K = 1$ CRPS (0.671) trails $K = 2$ (0.609) by $\approx 10\%$, and a residual $\approx 10\%$ gap against the ODE heads (0.55) remains even at $K = 2$ (Section 4.11). The second evaluation restores most of the spread, which is why $K = 2$ is the recommended operating point; a per-member rank-histogram diagnosis has not yet been run.

No distilled-diffusion comparator. We compare the flow heads against the full 35-NFE diffusion sampler, not against a distilled few-step diffusion student. MeanFlow demonstrates that the 1–2-NFE regime is reachable in a single run with no teacher; whether distillation could carry the diffusion baseline’s heavy-rain CSI into that same regime is an open question this thesis does not answer.

5.3 Future Work

The most promising directions, several of which are scoped concretely in the accompanying project to-do, are:

1. **Train to convergence and re-score.** Run all three heads to a matched, converged sample budget and repeat the full metric suite, smoothing over several validation snapshots to beat down the 10–30% noise. This is the single most important step to turn the provisional headline into a defensible claim.

- 
2. **Isolate the loss recipe.** Re-train the EDM diffusion baseline with the same rain-aware additions the flow heads use (channel weight w_{pr} , spectral log-PSD term), splitting the recipe's contribution from the generative objective's. The speed and NFE results are untouched by this experiment; the skill gaps are its target.
 3. **Close the heavy-rain gap.** Test an explicit tail-quantile or focal loss on qpepre, a higher w_{pr} paired with the spectral term, and a stronger $\text{asinh}/\log_{1p}$ dynamic-range transform, targeting the 5–10 mm h⁻¹ CSI deficit without inflating false alarms.
 4. **Restore low-NFE rollout spread.** MeanFlow under-disperses at $K = 1$ and the deficit compounds over autoregressive rollouts (a residual CRPS gap survives even at $K = 2$); candidate fixes — a stochastic single-segment sampler, temperature on the noise draw, per-step noise re-injection during rollout, or a small spread-aware loss term — would make the true 1–2-NFE regime probabilistically competitive end to end, diagnosed with per-member rank histograms.
 5. **Large-ensemble calibration.** Exploit the flow heads' low per-sample cost to run 50–100-member ensembles and report CRPS, rank histograms, and spread–skill — the uncertainty-quantification payoff that cheap sampling unlocks. At MeanFlow's two evaluations, a 100-member 24-hour cycle costs 4,800 network evaluations — about a ninth of what the 35-NFE diffusion sampler needs for a 50-member one — so ensemble size, not the sampler, becomes the budget knob.
 6. **Transfer to other domains.** Apply the same single-run flow recipes (FlowCast and MeanFlow) to another convection-permitting regression mean (e.g. an HRRR-trained StormCast) to measure how much of the pipeline is Taiwan-specific.

5.4 Closing Remarks



Generative modelling is almost certainly the right inductive bias for short-range convective-scale forecasting; the obstacle to deploying it has been the multi-step diffusion sampler. This thesis shows that, on a real regional dataset, a straight-line flow-matching residual can recover most of the diffusion baseline’s skill — and exceed it on winds, precipitation RMSE, and false alarms at the wider neighbourhoods — at roughly a quarter of the inference cost, trained in a single run with no teacher; and that an average-velocity residual trained under the same recipe compresses the sampler further still, to one or two network evaluations — a tenth of the diffusion cost — with the study’s best categorical precipitation hit rates and the best temperature accuracy of the flow heads, trading a modest spread-driven rollout-calibration gap. The path a generative model takes from noise to weather turned out to be a dial worth turning twice: first straightened, then jumped. The remaining gaps (heavy-rain hits, FlowCast’s τ_{2m} drift, one-evaluation spread, convergence at a larger budget) are concrete and addressable. Making sharp, probabilistic, regional convective forecasting fast enough to run between radar volumes is within reach, and single-run flow residuals — straight-line or average-velocity — are a credible path to it.



References

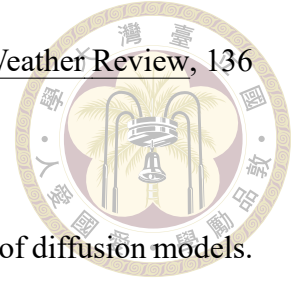
- [1] M. Andrychowicz, L. Espeholt, D. Li, S. Merchant, A. Merose, F. Zyda, S. Agrawal, and N. Kalchbrenner. Deep learning for day forecasts from sparse observations. arXiv preprint arXiv:2306.06079, 2023.
- [2] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian. Accurate medium-range global weather forecasting with 3D neural networks. Nature, 619:533–538, 2023.
- [3] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In Advances in Neural Information Processing Systems (NeurIPS), 2018.
- [4] L. Espeholt, S. Agrawal, C. K. Sønderby, M. Kumar, J. Heek, C. Bromberg, C. Gazeau, R. Carver, M. Andrychowicz, J. Hickey, A. Bell, and N. Kalchbrenner. Deep learning for twelve hour precipitation forecasts. Nature Communications, 13, 2022.
- [5] Z. Geng, M. Deng, X. Bai, J. Z. Kolter, and K. He. Mean flows for one-step generative modeling. arXiv preprint arXiv:2505.13447, 2025.
- [6] H. Hersbach, B. Bell, P. Berrisford, S. Hirahara, A. Horányi, J. Muñoz-Sabater, J. Nicolas, C. Peubey, R. Radu, D. Schepers, A. Simmons, C. Soci, S. Abdalla, X. Abellan, G. Balsamo, P. Bechtold, G. Biavati, J. Bidlot, M. Bonavita, G. D.

Chiara, P. Dahlgren, D. Dee, M. Diamantakis, R. Dragani, J. Flemming, R. Forbes, M. Fuentes, A. Geer, L. Haimberger, S. Healy, R. J. Hogan, E. Hólm, M. Janisková, S. Keeley, P. Laloyaux, P. Lopez, C. Lupu, G. Radnoti, P. de Rosnay, I. Rozum, F. Vamborg, S. Villaume, and J.-N. Thépaut. The ERA5 global reanalysis. Quarterly Journal of the Royal Meteorological Society, 146(730):1999–2049, 2020.

- [7] J. Ho and T. Salimans. Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598, 2022.
- [8] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [9] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. In Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [10] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Meroze, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia. Learning skillful medium-range global weather forecasting. Science, 382(6677):1416–1421, 2023.
- [11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In International Conference on Learning Representations (ICLR), 2023.
- [12] X. Liu, C. Gong, and Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In International Conference on Learning Representations (ICLR), 2023.

- 
- [13] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [14] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators. *arXiv preprint arXiv:2202.11214*, 2022.
- [15] J. Pathak, Y. Cohen, P. Garg, P. Harrington, N. Brenowitz, D. Durran, M. Mardani, A. Vahdat, S. Xu, K. Kashinath, and M. Pritchard. Kilometer-scale convection allowing model emulation using generative diffusion modeling. *arXiv preprint arXiv:2408.10958*, 2024.
- [16] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters, T. Ewalds, J. Stott, S. Mohamed, P. Battaglia, R. Lam, and M. Willson. Probabilistic weather forecasting with machine learning. *Nature*, 2024.
- [17] S. Ravuri, K. Lenc, M. Willson, D. Kangin, R. Lam, P. Mirowski, M. Fitzsimons, M. Athanassiadou, S. Kashem, S. Madge, R. Prudden, A. Mandhane, A. Clark, A. Brock, K. Simonyan, R. Hadsell, N. Robinson, E. Clancy, A. Arribas, and S. Mohamed. Skilful precipitation nowcasting using deep generative models of radar. *Nature*, 597:672–677, 2021.
- [18] B. P. Ribeiro and J. F. Pucer. FlowCast: Advancing precipitation nowcasting with conditional flow matching. *arXiv preprint*, 2025.
- [19] N. M. Roberts and H. W. Lean. Scale-selective verification of rainfall accumulations

from high-resolution forecasts of convective events. Monthly Weather Review, 136(1):78–97, 2008.



[20] T. Salimans and J. Ho. Progressive distillation for fast sampling of diffusion models. In International Conference on Learning Representations (ICLR), 2022.

[21] A. Sauer, D. Lorenz, A. Blattmann, and R. Rombach. Adversarial diffusion distillation. arXiv preprint arXiv:2311.17042, 2023.

[22] J. T. Schaefer. The critical success index as an indicator of warning skill. Weather and Forecasting, 5(4):570–575, 1990.

[23] C. K. Sønderby, L. Espenholt, J. Heek, M. Dehghani, A. Oliver, T. Salimans, S. Agrawal, J. Hickey, and N. Kalchbrenner. MetNet: A neural weather model for precipitation forecasting. arXiv preprint arXiv:2003.12140, 2020.

[24] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. In International Conference on Learning Representations (ICLR), 2021.

[25] Y. Song and P. Dhariwal. Improved techniques for training consistency models. arXiv preprint arXiv:2310.14189, 2023.

[26] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In International Conference on Learning Representations (ICLR), 2021.

[27] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency models. In International Conference on Machine Learning (ICML), 2023.

[28] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and

Y. Bengio. Improving and generalizing flow-based generative models with mini-batch optimal transport. Transactions on Machine Learning Research, 2024.







Appendix A — Hyperparameters and Training Configuration

A.1 EDM Diffusion Baseline Hyperparameters

The EDM diffusion baseline uses the following parameters. FlowCast and Mean-Flow share the data scale σ_{data} verbatim so that the standardised-residual statistics and per-channel loss scaling are identical across all three heads (Section 3.4):

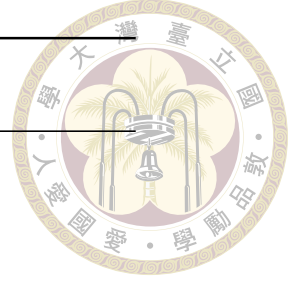
Parameter	Value
σ_{min}	0.002
σ_{max}	80
σ_{data}	0.5
ρ	7
Sampler	Heun
N (steps)	18 ($2N-1 = 35$ NFE)
Global batch size	64
Optimiser	AdamW

A.2 FlowCast (Conditional Flow Matching)



Hyperparameters copied from `stormcast/config/model/flowcast.yaml` and `stormcast/config/training/flowcast.yaml`:

Parameter	Value
σ_{data}	0.5
σ_{path} (I-CFM path std)	0.01
t_{ε} (time clamp)	10^{-5}
Time embedding scale τ	1000
diffusion_conditions	[state, regression, invariant]
regression_conditions	[state, background, invariant]
Backbone channel_mult	[1, 2, 2, 2, 2]
Optimiser	AdamW
Adam (β_1, β_2)	(0.9, 0.999)
Learning rate	5×10^{-4}
Weight decay	1×10^{-4}
Warmup steps (linear)	4,000
Min LR ratio (cosine floor)	10^{-2}
Total training steps	4×10^5
Global batch size	96
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	$(1, 1, 1, w_{\text{pr}})$, $w_{\text{pr}} = 2$ (headline)
Spectral weight λ_{spec}	0.1
Spectral channels	{qpepre}
Validation sampler	Euler, $K = 10$
Inference solvers	euler / midpoint

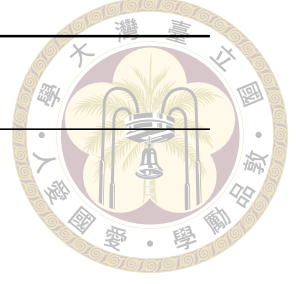


A.3 MeanFlow (Average-Velocity Flow Matching)



Hyperparameters copied from `stormcast/config/model/meanflow.yaml` and `stormcast/config/training/meanflow.yaml`. Everything shared with FlowCast (optimiser, schedule, conditioning, backbone, channel/spectral weights, EMA) is kept identical so the FlowCast-versus-MeanFlow comparison isolates the objective; only the rows below the divider are MeanFlow-specific:

Parameter	Value
σ_{data}	0.5
t_ε (time clamp)	10^{-5}
State-time embedding scale τ (on r)	1000
diffusion_conditions	[state, regression, invariant]
regression_conditions	[state, background, invariant]
Backbone channel_mult	[1, 2, 2, 2, 2]
Optimiser / Adam (β_1, β_2)	AdamW / (0.9, 0.999)
Learning rate / weight decay	5×10^{-4} / 1×10^{-4}
Warmup steps (linear) / min-LR ratio	4,000 / 10^{-2}
Gradient clip (global L2)	1.0
EMA decay (fixed)	0.999
Channel weights β	(1, 1, 1, 2)
Spectral weight λ_{spec} / channels	0.1 / {qpepre}
(r, t) sampler	two i.i.d. $\mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$, sorted $r \leq t$
MeanFlow ratio ρ_{MF} ($r < t$ fraction)	0.25
Adaptive weight exponent p	1.0
Adaptive weight stabiliser ε_w	10^{-3}
Interval embedding (on $t - r$)	32-dim sin / cos Fourier, freqs $\in [1, 10^3]$
σ_{path}	0 (pure interpolant)
Global batch size	112 (7 GPUs)
Validation sampler	average-velocity, $K = 2$
Inference NFE (headline)	$K \in \{1, 2\}$





A.4 Ablation Grids

The two ablations of Chapter 4 vary a single knob each, holding the rest of the Flow-Cast config fixed (the w_{pr} and encoding grids are FlowCast/EDM-only; MeanFlow adopts the winning settings, Sections 4.7–4.8):

Ablation	Values swept
Precipitation encoding (Section 4.7)	$\log_{10} p$ / raw mm h ⁻¹
Precipitation weight w_{pr} (Section 4.8)	{1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4}
Inference steps K , both flow heads (Section 4.4)	{1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 25, 32, 40, 50}; scoreb

A.5 Per-Channel Dataset Statistics

High-resolution target statistics computed on the 2019-08 – 2021-12 training split (original store; the cleaned store re-derives statistics after the $\log_{10} p$ encoding of qpepre):

Channel	Mean	Std
t2m (K)	295.98	5.61
u10 (m s ⁻¹)	-1.58	3.84
v10 (m s ⁻¹)	-2.65	5.66
qpepre (mm h ⁻¹ , raw)	0.182	9.12
qpepre (log(1 + x) space)	—	≈ 0.37



Appendix B — Reproducibility

B.1 Code Availability

The full training and evaluation code for this thesis lives in the Stormcast-Distillation repository. The key entry points are:

- `stormcast/train.py` — regression and EDM diffusion baseline training (Hydra config selects the mode).
- `stormcast/train_flowcast.py` — FlowCast (Conditional Flow Matching) training.
- `stormcast/train_meanflow.py` — MeanFlow (average-velocity flow matching) training.
- `stormcast/utils/trainer_flowcast.py` / `trainer_meanflow.py` — the two students' training loops with EMA.
- `stormcast/utils/flowcast_loss.py` — FlowCastLoss (I-CFM + per-channel weights + radial log-PSD).
- `stormcast/utils/meanflow_loss.py` — MeanFlowLoss (MeanFlow identity with JVP-bootstrapped stop-gradient target, adaptive weighting, same per-channel

weights + log-PSD term on the $t = r$ subset).

- `stormcast/utils/flowcast_precond.py/meanflow_precond.py` — FlowCastPrecond velocity wrapper and MeanFlowPrecond two-time average-velocity wrapper.
- `stormcast/utils/nn.py` — `get_preconditioned_architecture`, `build_network_condition` and the samplers `diffusion_model_forward/flowcast_model_forward/meanflow_model_for`
- `experiment_scripts/compare_diffusion_vs_flowcast.py` — the head-to-head rollout scoreboard harness used for Chapter 4 (the MeanFlow leg is enabled with `--meanflow-checkpoint`).
- `experiment_scripts/flowcast_nfe_sweep.py` — the NFE Pareto sweep behind Figure ?? and Table ??; `--method meanflow` runs the identical sweep for the MeanFlow head.
- `experiment_scripts/thesis_style.py` — the single source of truth for figure colour: `viridis` for all two-dimensional fields (matching the trainer-time validation plots), a diverging map for signed error fields, and a categorical per-method palette sampled from the `viridis` ramp.
- `experiment_scripts/make_thesis_figures.py/make_thesis_qualitative.sh`
`/ export_results_md.py` — regenerate every result figure of Chapter 4 and the consolidated `results.md` (see the regeneration commands below).

B.2 Dataset and Checkpoints

Zarr store. The cleaned Taiwan RWRF dataset used for the reported experiments is the local Zarr store

exp_3_..._full_cleaned_4_27_2026/



with sub-groups LowRes/, HighRes/, and invariants/, a 192×96 grid, and log1p-encoded qpepre (Section 3.2).

Checkpoints (cleaned, log1p, ≈ 2 M-sample anchor).

- Regression mean: StormCastUNet.0.8000.mdlus (shared by all three heads).
- EDM diffusion baseline: EDMPrecond.0.31000.mdlus.
- FlowCast: FlowCastPrecond.0.20000.mdlus; the sibling ema_state.pt holds the EMA shadow used by the in-training validation sampler. The canonical FlowCast run continues to FlowCastPrecond.0.140000.mdlus.
- MeanFlow: MeanFlowPrecond.0.20000.mdlus (≈ 2.24 M samples at batch 112) with its sibling ema_state.pt. The Chapter 4 harness loads the FlowCast and MeanFlow online checkpoints symmetrically.

B.3 Commands

All commands below launch from the stormcast/ directory. The shipped configs point at stale cluster paths and outdated valid_dates; override dataset.location, the regression weights, and dataset.valid_dates=['2022/01/01', '2022/12/31'] on the local workstation.

Regression mean (pre-train once).

```
python train.py \  
    dataset.location=.../zarr..._cleaned_4_27_2026 \  
    training.experiment_name=regression_main
```



EDM diffusion baseline.

```
python train.py --config-name=diffusion \  
    model.regression_weights=.../StormCastUNet.0.8000.mdlus \  
    dataset.location=.../zarr..._cleaned_4_27_2026 \  
    training.experiment_name=diffusion_main
```

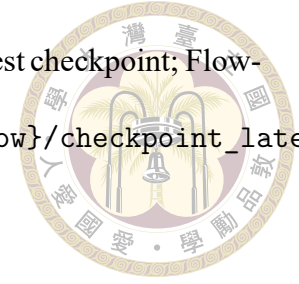
FlowCast.

```
python train_flowcast.py \  
    model.regression_weights=.../StormCastUNet.0.8000.mdlus \  
    dataset.location=.../zarr..._cleaned_4_27_2026 \  
    training.experiment_name=flowcast_main
```

MeanFlow.

```
python train_meanflow.py \  
    model.regression_weights=.../StormCastUNet.0.8000.mdlus \  
    dataset.location=.../zarr..._cleaned_4_27_2026 \  
    training.experiment_name=meanflow_main
```

Auto-resume. Re-running the same command picks up from the latest checkpoint; FlowCast and MeanFlow restore both checkpoints_{flowcast,meanflow}/checkpoint_latest.pt and ema_state.pt.



Head-to-head evaluation. The scoreboard, per-threshold, FSS, CRPS, rollout, and qualitative-panel outputs of Chapter 4 are produced by

```
bash experiment_scripts/run_main_experiment.sh
```

which runs compare_diffusion_vs_flowcast.py (with the MeanFlow leg at MEANFLOW_NFES="1 2" by default) and writes scoreboard.{md,csv}, per_threshold.csv, fss_p16.csv, rmse_per_step.csv, crps_per_channel.csv, and a panels/ tree in physical units (qpepr in mm h^{-1} after inverse-log1p).

NFE Pareto sweeps. The data behind Figure ?? and Tables ??–?? come from

```
python experiment_scripts/flowcast_nfe_sweep.py \  
    --flowcast-checkpoint .../FlowCastPrecond.0.20000.mdlus \  
    --nfes 1 2 3 4 5 6 8 10 12 16 20 25 32 40 50 \  
    --n-sequences 12 --n-steps 1 --ensemble 10 --seed 0  
python experiment_scripts/flowcast_nfe_sweep.py --method meanflow \  
    --meanflow-checkpoint .../MeanFlowPrecond.0.20000.mdlus \  
    --nfes 1 2 3 4 5 6 8 10 12 16 20 25 32 40 50 \  
    --n-sequences 12 --n-steps 1 --ensemble 10 --seed 0
```

The matched ≈ 2 M-sample checkpoints (step 20,000) are passed explicitly — the script's FlowCast default points at the longer-trained step-140,000 checkpoint, which would not

be comparable to the Chapter 4 scoreboard. `run_flowcast_nfe_sweep.sh` wraps both invocations (`METHODS="flowcast meanflow"`).



Figure and results regeneration. Every result figure of Chapter 4 and the consolidated results document are regenerated end-to-end by two commands:

```
bash experiment_scripts/make_thesis_figures.sh      # CPU only
bash experiment_scripts/make_thesis_qualitative.sh  # one GPU, ~2 min
```

The first rebuilds the data-driven figures (scoreboard, per-channel RMSE/CRPS, CSI/FAR/HSS by threshold, heavy-rain FSS, rollout RMSE, NFE Pareto with both flow heads) directly from the harness CSVs — no inference — writing them into the thesis figures/results/ tree, and then re-exports `experiment_scripts/results.md`, a single Markdown document collecting every scoreboard with its provenance. The second re-runs the cleaned-leg models for one generative step and re-renders the qualitative truth/diffusion/FlowCast/MeanFlow field panels. All plotting reads its colours from `thesis_style.py`, so the entire figure set stays on one palette by construction.



Appendix C — Detailed Derivations

This appendix collects line-by-line derivations of the identities the method of this thesis rests on, without passing through to the literature. The goal is that any reader with a working knowledge of multivariate Gaussians and basic Itô calculus can reconstruct, from scratch, the score–denoiser (Tweedie) identity and the probability-flow ODE that define the EDM diffusion baseline, the EDM preconditioning and noise schedule, the Conditional Flow Matching theorem and its straight-line Independent-CFM instantiation (the FlowCast baseline), and finally the MeanFlow average-velocity identity at the centre of the thesis. The discrete-time DDPM/DDIM machinery is omitted, as neither MeanFlow nor the continuous-time EDM baseline relies on it; the condensed summary in Chapter 2 suffices for context. Notation in each section is local; all symbols have the same meaning as in Chapters 2–3 unless otherwise specified.

C.1 Tweedie’s Formula: Score and Denoiser

Let $x_0 \sim p_{\text{data}}$ and $x = x_0 + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Write $p_\sigma(x) = \int p_{\text{data}}(x_0) \mathcal{N}(x; x_0, \sigma^2 I) dx_0$.

Differentiating under the integral,

$$\nabla_x p_\sigma(x) = \int p_{\text{data}}(x_0) \nabla_x \mathcal{N}(x; x_0, \sigma^2 I) dx_0 = \int p_{\text{data}}(x_0) \frac{x_0 - x}{\sigma^2} \mathcal{N}(x; x_0, \sigma^2 I) dx_0.$$

Dividing by $p_\sigma(x)$,

$$\nabla_x \log p_\sigma(x) = \frac{1}{\sigma^2} \mathbb{E}[x_0 - x | x] = \frac{\mathbb{E}[x_0 | x] - x}{\sigma^2}.$$



Identifying the MMSE denoiser $D^*(x; \sigma) = \mathbb{E}[x_0 | x]$ gives Tweedie's formula

$$\boxed{\nabla_x \log p_\sigma(x) = \frac{D^*(x; \sigma) - x}{\sigma^2}.} \quad (\text{C.1})$$

Because the Bayes-optimal denoiser is the conditional expectation, score matching and denoising regression are equivalent as learning objectives: fitting D_θ to minimise $\mathbb{E} \|D_\theta(x; \sigma) - x_0\|^2$ simultaneously learns an estimate of the score via (C.1).

C.2 Probability-Flow ODE from the VE-SDE

The variance-exploding forward SDE is

$$dx = g(\sigma) dW, \quad g(\sigma) = \sqrt{\frac{d\sigma^2}{dt}},$$

where we indexed time by σ . The corresponding Fokker–Planck equation for the marginal

$p_t(x) = p_\sigma(x)$ is

$$\frac{\partial p_\sigma}{\partial t} = \frac{1}{2} g(\sigma)^2 \Delta_x p_\sigma.$$

For the deterministic PF-ODE $dx/dt = f(x, \sigma)$ the continuity equation is

$$\frac{\partial p_\sigma}{\partial t} = -\nabla_x \cdot (f p_\sigma).$$

Requiring both to produce the same marginals and rearranging,

$$-\nabla_x \cdot (f p_\sigma) = \frac{1}{2} g^2 \Delta_x p_\sigma = \frac{1}{2} g^2 \nabla_x \cdot (\nabla_x p_\sigma) = \frac{1}{2} g^2 \nabla_x \cdot (p_\sigma \nabla_x \log p_\sigma),$$

which is solved by $f = -\frac{1}{2} g^2 \nabla_x \log p_\sigma$. Switching to σ as the time variable ($d/dt = (d\sigma/dt) d/d\sigma$ and $g^2 = 2\sigma d\sigma/dt$) yields the clean form

$$\boxed{\frac{dx}{d\sigma} = -\sigma \nabla_x \log p_\sigma(x)}. \quad (\text{C.2})$$

Combining (C.2) with Tweedie (C.1) gives the denoiser-parameterised PF-ODE

$$\frac{dx}{d\sigma} = \frac{x - D^*(x; \sigma)}{\sigma}, \quad (\text{C.3})$$

which is the form that first-order (Euler / DDIM) and second-order (Heun) integrators actually evaluate at inference.

C.3 EDM Preconditioning Derivation

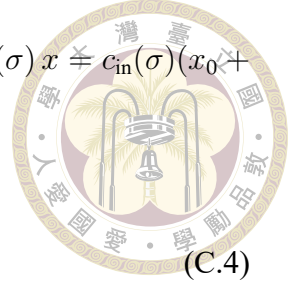
Karras et al. [9] parameterise the denoiser through an inner network F_θ with skip, input, output, and noise scalings:

$$D_\theta(x; \sigma) = c_{\text{skip}}(\sigma) x + c_{\text{out}}(\sigma) F_\theta(c_{\text{in}}(\sigma) x, c_{\text{noise}}(\sigma)).$$

We derive $(c_{\text{skip}}, c_{\text{in}}, c_{\text{out}})$ from three requirements.

Requirement 1: unit-variance input to F_θ . The network input $c_{\text{in}}(\sigma)x \doteq c_{\text{in}}(\sigma)(x_0 + \sigma\epsilon)$ has variance $c_{\text{in}}(\sigma)^2(\sigma_{\text{data}}^2 + \sigma^2)$. Setting this to 1 gives

$$c_{\text{in}}(\sigma) = \frac{1}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.4})$$



Requirement 2: unit-variance target for F_θ . The squared loss $\|D_\theta(x; \sigma) - x_0\|^2$ can be written in terms of the network's effective target:

$$F^*(x; \sigma) = \frac{x_0 - c_{\text{skip}}(\sigma)x}{c_{\text{out}}(\sigma)}.$$

For the F_θ -regression loss to remain well-conditioned across σ , $\text{Var}(F^*) = 1$:

$$\begin{aligned} \text{Var}(F^*) &= \frac{1}{c_{\text{out}}^2} \text{Var}(x_0 - c_{\text{skip}}x) \\ &= \frac{1}{c_{\text{out}}^2} \left[(1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2 \right]. \end{aligned}$$

Setting this equal to 1 is one equation in two unknowns ($c_{\text{skip}}, c_{\text{out}}$); we need the third requirement to pin them down.

Requirement 3: minimise the error amplification c_{out} . Requirement 2 fixed c_{out}^2 as a function of c_{skip} ; Karras et al. [9] pin c_{skip} down by minimising c_{out} , so the inner network's prediction error is amplified as little as possible on the way to D_θ :

$$c_{\text{out}}^2 = (1 - c_{\text{skip}})^2 \sigma_{\text{data}}^2 + c_{\text{skip}}^2 \sigma^2.$$

Differentiating in c_{skip} and setting to zero,

$$-2(1 - c_{\text{skip}})\sigma_{\text{data}}^2 + 2c_{\text{skip}}\sigma^2 = 0 \implies c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}.$$

Substituting back into the unit-variance requirement,

$$c_{\text{out}}^2 = \left(\frac{\sigma^2}{\sigma^2 + \sigma_{\text{data}}^2} \right)^2 \sigma_{\text{data}}^2 + \left(\frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2} \right)^2 \sigma^2 = \frac{\sigma^2 \sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}$$

so

$$c_{\text{skip}}(\sigma) = \frac{\sigma_{\text{data}}^2}{\sigma^2 + \sigma_{\text{data}}^2}, \quad c_{\text{out}}(\sigma) = \frac{\sigma \sigma_{\text{data}}}{\sqrt{\sigma^2 + \sigma_{\text{data}}^2}}. \quad (\text{C.5})$$

Noise scaling. $c_{\text{noise}}(\sigma) = \frac{1}{4} \log \sigma$ is an empirically-motivated choice to equalise the dynamic range of the time-embedding input across $\sigma \in [\sigma_{\min}, \sigma_{\max}]$ (the log compresses orders of magnitude; the $\frac{1}{4}$ puts typical values in a band that matches the embedding frequencies).

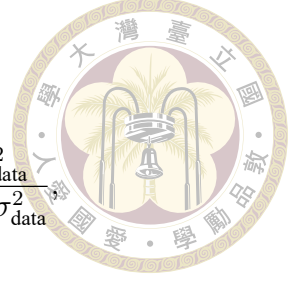
Loss weighting. The EDM loss is

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{\sigma} \left[\lambda(\sigma) \|D_{\theta}(x_0 + \sigma \epsilon; \sigma) - x_0\|^2 \right].$$

Substituting $D_{\theta} = c_{\text{skip}}x + c_{\text{out}}F_{\theta}(\cdot)$ and the target F^* , the inner squared error is $c_{\text{out}}^2 \|F_{\theta} - F^*\|^2$. To make the F_{θ} -regression effectively unit-weighted, we choose

$$\lambda(\sigma) = \frac{1}{c_{\text{out}}(\sigma)^2} = \frac{\sigma^2 + \sigma_{\text{data}}^2}{\sigma^2 \sigma_{\text{data}}^2}. \quad (\text{C.6})$$

With the Karras log-normal sampling $\log \sigma \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$, the training-time distribution of σ combined with (C.6) concentrates the learning signal at mid-range σ , which is where the denoiser has the highest leverage on the final sample quality. This concludes the derivation; the five preconditioning scalars $(c_{\text{skip}}, c_{\text{in}}, c_{\text{out}}, c_{\text{noise}}, \lambda)$ of Equation (2.10) of the main text are now all justified from first principles.





C.4 Karras ρ -Spaced Noise Schedule

The sampling schedule $\{\sigma_i\}$ is chosen to equalise truncation error per step of a first-order ODE integrator of Equation (C.3). Under the ansatz

$$\sigma_i = \left(\sigma_{\max}^{1/\rho} + \frac{i}{N-1}(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho}) \right)^\rho,$$

the change of variables $u = \sigma^{1/\rho}$ makes the index i linear in u , and the per-step truncation error of Euler integration is approximately constant in u for ρ tuned to the curvature of the trajectory. Karras et al. [9] show empirically that $\rho = 7$ gives the lowest terminal FID among $\rho \in \{1, 3, 5, 7, 9\}$ across diverse datasets; we adopt it unchanged for the StormCast diffusion baseline, following Pathak et al. [15].

C.5 CFM Theorem: Conditional = Marginal Gradient

Fix a Gaussian conditional probability path $p_t(x \mid z)$ indexed by a latent $z \sim q(z)$, with corresponding conditional vector field $u_t(x \mid z)$. The marginal flow-matching loss is

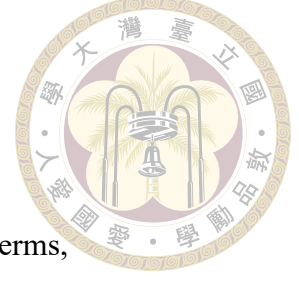
$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(x)} [\|v_\theta(x, t) - u_t(x)\|^2],$$

where $p_t(x) = \mathbb{E}_{q(z)}[p_t(x \mid z)]$ and $u_t(x) = \mathbb{E}[u_t(x \mid z) \mid x, t]$ is the marginal vector field.

This loss is intractable because $u_t(x)$ requires marginalising over z . The conditional flow-matching loss replaces the marginal target with the conditional one:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z, p_t(x|z)} [\|v_\theta(x, t) - u_t(x \mid z)\|^2].$$

Theorem. $\nabla_{\theta} \mathcal{L}_{\text{FM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{CFM}}(\theta)$.



Proof. Expanding the squared norms and dropping θ -independent terms,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(x)} [\|v_{\theta}(x, t)\|^2 - 2 v_{\theta}(x, t)^{\top} u_t(x)] + \text{const},$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z, p_t(x|z)} [\|v_{\theta}(x, t)\|^2 - 2 v_{\theta}(x, t)^{\top} u_t(x | z)] + \text{const}.$$

The first term is identical after applying the law of total expectation, $\mathbb{E}_{z, p_t(x|z)} = \mathbb{E}_{p_t(x)}$.

For the second term,

$$\begin{aligned} \mathbb{E}_{t, z, p_t(x|z)} [v_{\theta}(x, t)^{\top} u_t(x | z)] &= \mathbb{E}_t \mathbb{E}_{p_t(x)} [v_{\theta}(x, t)^{\top} \mathbb{E}[u_t(x | z) | x, t]] \\ &= \mathbb{E}_{t, p_t(x)} [v_{\theta}(x, t)^{\top} u_t(x)], \end{aligned}$$

using $u_t(x) = \mathbb{E}[u_t(x | z) | x, t]$. Hence $\mathcal{L}_{\text{FM}} = \mathcal{L}_{\text{CFM}} + \text{const}$, and their gradients coincide. $\square$

The upshot is that training against the conditional target is an unbiased (and computable) estimator of gradients of the marginal objective, and the latter is exactly the flow-matching objective whose ODE transports $p_0 \rightarrow p_1$.

C.6 Independent CFM: Straight-Line Path and Target

Take $z = (x_0, x_1)$ with $x_0 \sim \mathcal{N}(0, I)$ and $x_1 \sim p_{\text{data}}$, independent. Define the Gaussian probability path

$$p_t(x | x_0, x_1) = \mathcal{N}((1-t)x_0 + tx_1, \sigma_{\text{path}}^2 I), \quad t \in [0, 1].$$

Target vector field. Under the flow $\phi_t(x \mid x_0, x_1) = (1 - t)x_0 + tx_1 + \sigma_{\text{path}} \eta$ with η fixed, the associated vector field is

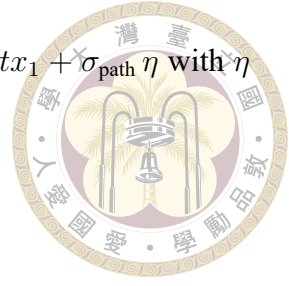
$$u_t(x \mid x_0, x_1) = \frac{d}{dt} \phi_t = x_1 - x_0,$$

which is independent of x (and of t) — a straight-line constant-velocity interpolation from noise to data. Plugging into $\mathcal{L}_{\text{CFM}}$,

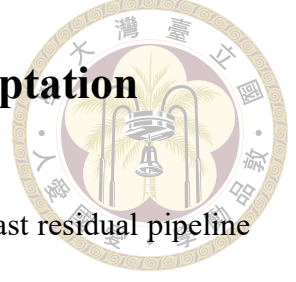
$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{t, x_0, x_1, \eta} [\|v_\theta((1 - t)x_0 + tx_1 + \sigma_{\text{path}}\eta, t) - (x_1 - x_0)\|^2]. \quad (\text{C.7})$$

Why $\sigma_{\text{path}} > 0$. At $\sigma_{\text{path}} = 0$ the conditional path is a Dirac at each time, supported on the line segment between x_0 and x_1 . In the marginalised view this is a measure-zero set, so the score of the marginal $p_t(x)$ is undefined on that set and the empirical loss gradient is extremely high-variance in high dimensions (Rectified-Flow pathology; see 12). Taking $\sigma_{\text{path}} > 0$ convolves the path with a spherical Gaussian of width σ_{path} , which regularises the target field onto a thin tube around the interpolant and smooths the loss landscape. Tong et al. [28] report $\sigma_{\text{path}} \in [10^{-3}, 10^{-1}]$ all work in practice; FlowCast and our adaptation use $\sigma_{\text{path}} = 0.01$.

Compared to diffusion. The straight-line ODE of I-CFM and the PF-ODE of a score-based diffusion both transport $\mathcal{N}(0, I) \rightarrow p_{\text{data}}$, but their trajectories differ. The diffusion PF-ODE is curved: its Jacobian sees the local score, which in turn depends on the local data density, and its integration error grows quickly as the step count shrinks. The I-CFM ODE has constant velocity $x_1 - x_0$ along each conditional trajectory, so the marginal field is the closest approximation (in L^2) to a linear transport available — which is exactly why a single Euler step is already competitive.



C.7 FlowCast’s Standardised-Residual Adaptation



The adaptation used in Chapter 3 applies I-CFM to the StormCast residual pipeline with the following changes to the textbook I-CFM of (C.7):

1. **Residual target.** x_1 is the standardised residual R_t/σ_{data} rather than the raw field. Standardisation absorbs the per-channel scale so the noise $x_0 \sim \mathcal{N}(0, I)$ is comparable in magnitude to the data target.
2. **Condition bundle.** v_θ is conditioned on $c_t = \text{concat}(X_{t-1}, M_t, I)$ (the low-resolution background S_t is omitted from the direct path and reaches the velocity field only through the regression mean M_t ; see Section 3.5). This keeps the FlowCast residual anchored to exactly the same M_t as the EDM diffusion baseline. No extra VAE encoder is inserted, unlike the original FlowCast which works in latent space.
3. **Time rescaling.** The flow time $t \in [0, 1]$ is multiplied by $\tau = 1000$ before entering the SongUNet time embedding, because the positional-embedding frequencies of the pretrained architecture were tuned for the EDM $\log \sigma$ range.
4. **Channel-weighted + spectral loss.** The pointwise MSE is the channel-weighted L2 norm $\|\cdot\|_{2,\beta}$ of Section 3.7, and an additional radial log-PSD regulariser is applied to the implied clean residual $\hat{x}_1 = x_t + (1 - t) v_\theta$ on the qpepre channel. Applying the regulariser on $\hat{x}_1$ rather than on v_θ keeps the spectrum interpretable as a physical field’s spectrum.
5. **Endpoint clamp.** Training samples $t \sim \mathcal{U}(t_\varepsilon, 1 - t_\varepsilon)$ with $t_\varepsilon = 10^{-5}$. At $t = 0$ or $t = 1$ the conditional path collapses to a point mass and the Monte-Carlo loss

gradient is unbounded; the clamp prevents numerical blow-ups at negligible bias cost.



The resulting training loop differs from I-CFM only in these five bookkeeping details; the underlying theorem (§C.5) and path derivation (§C.6) carry over unchanged, and the sampler (3.7) is the explicit Euler integrator of the straight-line ODE.

C.8 The MeanFlow Average-Velocity Identity

This is the derivation behind the method of this thesis: the MeanFlow identity [5], transcribed to the noise-at-0, data-at-1 convention used throughout (Section 3.6). It turns an intractable path integral into a pointwise relation the network can be regressed against.

Setup. Work in the standardised residual space of §C.7: $x_1 = R_t/\sigma_{\text{data}}$ is the data endpoint, $x_0 \sim \mathcal{N}(0, I)$ the prior, and the linear interpolant $z_s = (1 - s)x_0 + sx_1$ runs from noise at $s = 0$ to data at $s = 1$ along the trajectory $\dot{z}_s = v(z_s, s)$ with v the (marginal) velocity field of §C.6. For two times $r \leq t$ define the average velocity

$$u(z_r, r, t) \triangleq \frac{1}{t - r} \int_r^t v(z_\tau, \tau) d\tau, \quad (\text{C.8})$$

the displacement of the flow over $[r, t]$ per unit time. Two immediate properties follow. First, the limit as $t \rightarrow r$ is the instantaneous velocity, $\lim_{t \rightarrow r} u(z_r, r, t) = v(z_r, r)$, by the fundamental theorem of calculus. Second, multiplying by $(t - r)$ makes transport across the interval a single algebraic step,

$$z_t = z_r + (t - r) u(z_r, r, t), \quad (\text{C.9})$$

exactly and with no solver — one evaluation at $(r, t) = (0, 1)$ maps noise to data. The catch is that (C.8) is an intractable integral, so u cannot be regressed directly.



Differentiating away the integral. Clear the denominator in (C.8),

$$(t - r) u(z_r, r, t) = \int_r^t v(z_\tau, \tau) d\tau,$$

and differentiate both sides with respect to the state time r , holding the horizon time t fixed. Along the trajectory the state moves with $dz_r/dr = v(z_r, r)$ and $dt/dr = 0$. The left-hand side, by the product rule, is

$$\frac{d}{dr} [(t - r) u] = -u(z_r, r, t) + (t - r) \frac{d}{dr} u(z_r, r, t).$$

The right-hand side, by the fundamental theorem of calculus applied to the lower limit (hence the leading minus sign), is

$$\frac{d}{dr} \int_r^t v(z_\tau, \tau) d\tau = -v(z_r, r).$$

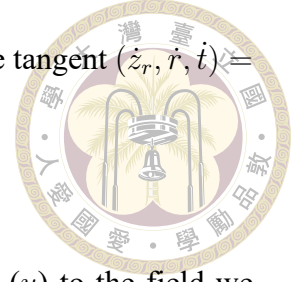
Equating the two and cancelling the $-u$ against $-v$ rearrangement gives the MeanFlow identity

$$\boxed{u(z_r, r, t) = v(z_r, r) + (t - r) \frac{d}{dr} u(z_r, r, t),} \quad (\text{C.10})$$

which is Equation (3.11) of the main text. The total derivative along the trajectory expands through the chain rule into partials,

$$\frac{d}{dr} u(z_r, r, t) = \partial_z u \cdot \dot{z}_r + \partial_r u = v(z_r, r) \partial_z u + \partial_r u, \quad (\text{C.11})$$

i.e. exactly the directional (Jacobian–vector) derivative of u along the tangent $(\dot{z}_r, \dot{r}, \dot{t}) = (v, 1, 0)$.



Why this is trainable. Equation (C.10) relates the field we want (u) to the field we can sample without bias ($v = x_1 - x_0$ along each conditional path, by §C.5) using only quantities available at a single point of a single trajectory — the path integral is gone. A network $u_\theta(z_r, r, t, c)$ is regressed onto the right-hand side of (C.10), with the derivative term (C.11) computed by one forward-mode Jacobian–vector product through u_θ along the tangent $(v, 1, 0)$ and the whole target held under stop-gradient. No higher-order gradients, no teacher, no integration, and no discretisation curriculum are required. A network achieving zero loss satisfies (C.10) everywhere, and therefore reproduces the average velocity of (C.8).

Boundary condition and the FlowCast special case. At $t = r$ the interval factor $(t - r)$ vanishes and (C.10) collapses to $u(z_r, r, r) = v(z_r, r)$: average velocity over a zero-length interval is instantaneous velocity. This is exactly the I-CFM/FlowCast regression of §C.6. MeanFlow therefore generalises the FlowCast baseline — the collapsed ($t = r$) subset of the batch trains the boundary that the $r < t$ subset propagates outward into the interval — which is what makes the comparison of Chapter 4 an A/B test on the objective alone.

Convention note. Geng et al. [5] place data at $s = 0$ and differentiate with respect to the horizon time t , obtaining $u = v - (t - r) \frac{d}{dt}u$. Here data sits at $s = 1$, so the differentiation runs through the lower limit of the integral and the sign of the interval term flips to a plus, as in (C.10). The two are the same identity in mirrored time; the loss and sampler of Section 3.6 are internally consistent with the convention used here.